

趣谷集项目面试宝典

项目背景：2024 年谷子经济爆发背景下，公司项目组为线下商场（万达、大悦城等）周边门店打造的线上聚合平台，采用 Spring Cloud Alibaba 拆分为 10+ 微服务模块，覆盖资讯发布、晒帖社区、全站搜索、IP 智能问答等核心业务。

技术栈：Spring Cloud Alibaba · Nacos · Sentinel · OpenFeign · Gateway · JWT · Redis · Elasticsearch(IK) · RabbitMQ · MySQL · MyBatis-Plus · Quartz

个人职责：主要参与登录认证、全站搜索、互动计数、AI 智能问答、内部服务稳定性和通用工程能力建设，以及日常单元测试与每日代码 Review。

说明：本文档基于原 wolf2w 项目代码实现整理，技术方案与趣谷集简历描述一一对应，每个知识点均可追溯到项目源码。

简历五大模块 → 面试题章节对照表

简历模块	简历要点	对应章节
① 微服务架构与登录认证	JWT + Redis 登录会话管理	第一章 1.1
	多端登录态隔离	第一章 1.1
	账号强制下线	第一章 1.1
	Token 滑动续期	第一章 1.1
	Gateway 网关鉴权	第一章 1.2
	统一登录入口与多端认证	第一章 1.3
	OpenFeign + Sentinel 跨服务调用降级	第一章 1.4
② AI 智能导购问答 (RAG)	多域向量知识库	第二章 2.1
	混合召回 (BM25 + kNN)	第二章 2.2
	IP 意图识别 + 检索路由	第二章 2.3
	角色化 Prompt 注入	第二章 2.4
	10 个核心技术点详解	第二章 2.5
③ Elasticsearch 全文搜索	IK 分词器	第三章 3.1
	ES 索引初始化	第三章 3.2
	RabbitMQ 异步同步 ES 索引	第三章 3.3
	瘦索引 + Feign 远程补全	第三章 3.4
	多索引独立容错	第三章 3.5
	ES 踩坑实录	第三章 3.6
④ Redis 高并发互动数据	Hash 聚合统计 + HINCRBY	第四章 4.1
	收藏 Set 去重	第四章 4.2
	点赞 String + INCR 限频	第四章 4.3
	三种数据结构对比	第四章 4.4
	Quartz 定时批量回写 MySQL	第四章 4.5
	游记模块 Redis 计数升级	第四章 4.6
⑤ 工程实践	枚举 RedisKeys 管理	第五章 5.1
	策略模式 + @ConditionalOnProperty 多存储切换	第五章 5.2
	递归构建层级树与面包屑导航	第五章 5.3
	MyBatis-Plus 分页与条件查询	第五章 5.4
附录	项目踩坑实录	附录 A
	Quartz 定时任务调度详解	附录 B
	游记模块完整实现参考	附录 C

使用方式：面试前先看「简历答题模板」小节（每个模块末尾），被追问细节时再翻对应章节展开。每个章节都有「面试怎么说」提示语。

一、枚举（Enum）

★ 所属简历模块：⑤ 工程实践

1.1 项目中的使用场景

场景一：RedisKeys 枚举统一管理 Redis 的 key（最典型）

在微服务里，多个服务都会往 Redis 写数据（验证码、登录 token、缓存等），如果 key 的前缀散落在各个业务代码里，很容易写错、重复、或者忘记设过期时间。本项目用一个枚举 `RedisKeys` 统一收口：

```
@Getter
public enum RedisKeys {

    // 枚举值 --> 因为外界无法创建对象,所以必须调用自己的构造方法先创建枚举对象
    VERIFY_CODE("verify_code:", 60 * 5);

    // key 前缀
    private String prefix;
    // 过期时间
    private long expire;

    // 构造方法私有,外界无法 new
    private RedisKeys(String prefix, long expire) {
        this.prefix = prefix;
        this.expire = expire;
    }

    // 对外提供方法 --> 外界拼接获取key
    public String join(String... values) {
        StringBuffer sb = new StringBuffer();
        sb.append(prefix);
        for (String value : values) {
            sb.append(":");
            sb.append(value);
        }
        return sb.toString();
    }
}
```

业务中两处真实调用：

① 发送验证码时写入 Redis（`SmsServiceImpl`）：

```
public void sendVerifyCode(String phone) {
    String verifyCode = String.format("%04d", new Random().nextInt(10000));
    // 用枚举拼 key: verify_code:13800000000
    String key = RedisKeys.VERIFY_CODE.join(phone);
    // 过期时间也由枚举统一管理: 60 * 5 = 300 秒
    redisService.setCacheObject(key, verifyCode,
        RedisKeys.VERIFY_CODE.getExpire(), TimeUnit.SECONDS);
}
```

② 注册时从 Redis 取出校验（`PcLoginService#register`）：

```
String key = RedisKeys.VERIFY_CODE.join(register.getUsername());
String verifyCode = redisService.getCacheObject(key);
if (StringUtils.isEmpty(verifyCode)) {
    throw new ServiceException("手机验证码不存在或者已失效");
}
if (!StringUtils.equals(verifyCode, register.getVerifyCode())) {
    throw new ServiceException("手机验证码错误");
}
```

这样做的好处：key 前缀和过期时间集中在一个枚举里管理，后续如果要新增一种缓存（比如登录 token、短信限流），只需加一个枚举值 `LOGIN_TOKEN("login_token:", 1800)`，所有调用点天然保持一致，不会出现 "verify_code:" 和 "verifyCode:" 两种写法并存的问题。

场景二：UserStatus 枚举描述用户状态

登录时需要判断用户是否被删除、是否被停用，本项目用枚举替代魔法值：

```
public enum UserStatus {
    OK("0", "正常"), DISABLE("1", "停用"), DELETED("2", "删除");

    private final String code;
    private final String info;

    UserStatus(String code, String info) {
        this.code = code;
        this.info = info;
    }
    // getCode / getInfo ...
}
```

在 `SysLoginService` 登录校验中：

```
if (UserStatus.DELETED.getCode().equals(user.getDelFlag())) {
    throw new ServiceException("对不起，您的账号：" + username + " 已被删除");
}
if (UserStatus.DISABLE.getCode().equals(user.getStatus())) {
    throw new ServiceException("对不起，您的账号：" + username + " 已停用");
}
```

好处：相比 `if ("2".equals(user.getDelFlag()))` 这种写法，`UserStatus.DELETED.getCode()` 语义清晰，且把状态码（0/1/2）和中文含义绑定，避免散落的魔法值。

场景三：BusinessType 等枚举配合 @Log 注解记录操作日志

后台管理每个增删改查接口都要记录操作日志，本项目用 `@Log` 注解 + 枚举来标记操作类型：

```
public enum BusinessType {
    OTHER, INSERT, UPDATE, DELETE, GRANT, EXPORT, IMPORT, FORCE, GENCODE, CLEAN
}
```

Controller 中的使用：

```
@Log(title = "区域", businessType = BusinessType.INSERT)
@PostMapping
public R<?> add(@RequestBody Region region) { ... }

@Log(title = "区域", businessType = BusinessType.DELETE)
@DeleteMapping("/{ids}")
public R<?> remove(@PathVariable Long[] ids) { ... }
```

AOP 切面拦截带 @Log 的方法，根据 businessType 字段把 "新增/删除/导出" 等操作类型写入 sys_oper_log 表，便于审计。此外还有 BusinessStatus (SUCCESS/FAIL)、OperatorType (后台/手机端用户)、Logical (权限校验 AND/OR) 等枚举，思路一致。

1.2 面试可能追问的问题

Q1: Java 枚举的本质是什么?

A: Java 枚举本质上是一个继承自 java.lang.Enum 的特殊 final 类。每个枚举常量都是该类的 public static final 实例，在类加载的 <clinit> 阶段被创建。所以枚举是 "单例的集合"，每个枚举值在 JVM 中只有一个实例。

Q2: 为什么枚举构造方法默认是 private?

A: 因为枚举实例只能由 JVM 在类加载时创建 (在 static 块中)，外部不能 new，也不能被继承 (final)。这样就保证了枚举值的固定性和唯一性。本项目的 RedisKeys 构造方法显式写成 private，外部只能通过 RedisKeys.VERIFY_CODE 拿到唯一实例。

Q3: 枚举相比常量类 (public static final String) 有什么优势?

A:

- 类型安全**: 枚举是独立类型，方法参数声明为 BusinessType 就只能传入合法的枚举值，不会传错字符串。
- 语义丰富**: 枚举可以带属性和方法 (如 UserStatus 同时持有 code 和 info，RedisKeys 持有 prefix 和 expire 并提供 join 方法)，常量类只能放值。
- 可遍历**: values() 可以枚举所有取值，便于做下拉框、校验。
- switch 支持**: 编译器会对枚举的 switch 做穷尽性检查。

Q4: 枚举能实现单例模式吗? 为什么推荐?

A: 能，而且是最推荐的单例实现方式 (Effective Java 推荐)。因为枚举实例的创建由 JVM 在类加载时保证线程安全，且天然防御反射攻击 (反射创建枚举会抛 IllegalArgumentException) 和反序列化破坏 (枚举的序列化机制特殊，不会产生新对象)。本项目 RedisKeys.VERIFY_CODE 实际就是一个单例。

Q5: 枚举可以用在 switch 中吗?

A: 可以，并且 Java 1.7 起 switch 支持枚举。case 后面直接写枚举常量名 (不带类名前缀)，编译器会做类型推断和穷尽性检查。

Q6: 枚举可以继承类或被继承吗?

A: 枚举隐式继承 java.lang.Enum，由于 Java 单继承，不能再继承别的类；但可以实现接口。枚举类默认是 final 的，不能被继承。如果想让每个枚举值有不同行为，可以让枚举实现接口，或在枚举里定义 abstract 方法，由每个枚举值各自实现 (类似策略模式)。

Q7: 枚举的 values() 方法是怎么实现的?

A: values() 是编译器在编译期自动插入的静态方法 (不在 Enum 基类里)，它返回一个由所有枚举常量组成的新数组 (每次调用都 new 新数组，所以修改不会影响内部)。valueOf(String) 也是编译器插入的，通过 name 匹配。

二、递归 (Recursion)

✳ 所属简历模块: ⑤ 工程实践

2.1 项目中的使用场景

场景一: 菜单树构建 (SysMenuServiceImpl)

后台菜单是典型的树形结构 (一级目录 → 二级菜单 → 三级按钮)。数据库里存的是扁平结构 (每个菜单有 menu_id 和 parent_id)，查出来后需要组装成树返回给前端渲染侧边栏。

核心递归方法 `recursionFn` :

```
/**
 * 递归列表：为某个节点挂载它的子节点，并继续向下递归
 */
private void recursionFn(List<SysMenu> list, SysMenu t) {
    // 1. 找当前节点的所有直接子节点
    List<SysMenu> childList = getChildList(list, t);
    // 2. 挂到 children 属性上
    t.setChildren(childList);
    // 3. 对每个子节点继续递归（如果它还有孩子）
    for (SysMenu tChild : childList) {
        if (hasChild(list, tChild)) {
            recursionFn(list, tChild);
        }
    }
}

/**
 * 从全量列表中挑出 parentId == t.menuId 的节点
 */
private List<SysMenu> getChildList(List<SysMenu> list, SysMenu t) {
    List<SysMenu> tlist = new ArrayList<>();
    for (SysMenu n : list) {
        if (n.getParentId().longValue() == t.getMenuId().longValue()) {
            tlist.add(n);
        }
    }
    return tlist;
}

/**
 * 判断是否有子节点（递归出口辅助方法）
 */
private boolean hasChild(List<SysMenu> list, SysMenu t) {
    return getChildList(list, t).size() > 0;
}
```

入口方法 `getChildPerms` 从 parentId=0 的根节点开始递归：

```
public List<SysMenu> getChildPerms(List<SysMenu> list, int parentId) {
    List<SysMenu> returnList = new ArrayList<>();
    for (SysMenu t : list) {
        if (t.getParentId() == parentId) {
            recursionFn(list, t);
            returnList.add(t);
        }
    }
    return returnList;
}
```

调用处（ `selectMenuTreeByUserId` ）：

```
List<SysMenu> menus = menuMapper.selectMenuTreeByUserId(userId);
return getChildPerms(menus, 0); // 从根节点开始递归构建树
```

场景二：部门树构建 (SysDeptServiceImpl)

部门管理同样是树形结构，逻辑几乎完全一样（recursionFn / getChildList / hasChild 三件套）：

```
public List<SysDept> buildDeptTree(List<SysDept> depts) {
    List<SysDept> returnList = new ArrayList<>();
    List<Long> tempList = depts.stream().map(SysDept::getDeptId).collect(Collectors.toList());
    for (SysDept dept : depts) {
        // 找出顶级节点（其 parentId 不在当前列表中的就是根）
        if (!tempList.contains(dept.getParentId())) {
            recursionFn(depts, dept);
            returnList.add(dept);
        }
    }
    if (returnList.isEmpty()) {
        returnList = depts;
    }
    return returnList;
}
```

场景三：目的地吐司导航（向上递归找父节点）

与场景一、二的"向下递归找子节点"不同,吐司导航是"向上递归找父节点"的典型例子。攻略页面顶部会显示一条面包屑(如 中国 / 广东 / 广州),数据库里只存了广州的 parentId=广东id ,需要沿着 parentId 一路向上找到根,再反序输出。

核心递归方法 createToast :

```
@Override
public List<Destination> queryToastsByDestId(Long destId) {
    // 递归实现
    List<Destination> list = new ArrayList<>();
    createToast(list, destId);
    // ★ 注意: Collections.reverse 已注释掉, 因为 createToast 用的是后序递归
    // (先递归到底再 add), 天然得到正序 (中国→广东→广州), 无需反转
    // Collections.reverse(list);
    return list;
}

// 自定义递归方法: 从当前节点出发, 沿 parentId 向上找
private void createToast(List<Destination> list, Long destId) {
    // 递归出口: destId 为 null, 说明已经走到根节点之上, 停止
    if (destId == null) {
        return;
    }
    Destination destination = lambdaQuery()
        .eq(Destination::getId, destId)
        .one();
    // ★ 后序递归: 先递归找父节点, 再加入当前节点
    // 执行流程: 广州→广东→中国→null(触底), 回返时逐层 add: 中国→广东→广州
    createToast(list, destination.getParentId());
    list.add(destination); // "归"阶段才 add, 天然正序
}
```

与菜单树递归的关键区别:

对比维度	菜单树/部门树(场景一、二)	吐司导航(场景三)
递归方向	向下找子节点(父→子)	向上找父节点(子→父)
遍历方式	先序遍历,每层可能有多个子节点	单链向上,每层只有一个父节点
数据结构	树(N 叉树)	链表(单链)
递归出口	节点没有子节点(hasChild=false)	parentId 为 null(走到根)
结果顺序	天然是从根到叶子,无需反转	后序递归天然正序(中国→广东→广州),无需 reverse
查询次数	每层查一次 getChildList	每层查一次当前节点

面试题:吐司导航这种"沿父引用向上回溯到根"的递归,本质是链表遍历。因为每层只有一个父节点,所以不会出现菜单树那种 N 叉分支,递归深度就等于节点层级,非常安全。结果反序是常见的技巧 —— 递归天然"逆序收集"(先处理当前,再处理父级),而业务需要"正序展示"(从根到当前),所以最后 reverse 一次。

性能优化:当前实现每递归一层就查一次数据库,层级深时会有 N 次 SQL。优化方案是一次性把整张目的地表查出来转成 `Map<id, Destination>`,之后查父节点就是内存 O(1) 查询,SQL 次数从 N 降到 1。代码里已注释保留了优化版本(详见下面 2.4 节)。

场景四：目的地二级树展示（按区域筛选 + 向下组装子节点）

项目位置：`DestinationServiceImpl.queryDestinationsByRegionId()`。首页或目的地列表页，点击某个区域（如“东南亚”），要展示该区域下的所有国家，以及每个国家的前 10 个热门城市。

数据结构：`Destination` 有 `parentId` 字段和 `children` 集合（`@TableField(exist=false)`，非数据库字段），天然支持父子层级。

核心实现：

```
public List<Destination> queryDestinationsByRegionId(Long regionId) {
    List<Destination> list = new ArrayList<>();
    if (regionId == -1) {
        // 热门：查所有 parentId=1 的一级目的地（国家）
        list = lambdaQuery().eq(Destination::getParentId, 1).list();
    } else {
        // 指定区域：通过 region 的 refIds 关联字段，逐个查目的地
        Region region = regionMapper.selectById(regionId);
        String[] split = region.getRefIds().split(",");
        for (String id : split) {
            list.add(lambdaQuery().eq(Destination::getId, id).one());
        }
    }

    // 遍历每个一级目的地，查它的前 10 个子目的地（城市）
    list.forEach(destination -> {
        List<Destination> children = lambdaQuery()
            .eq(Destination::getParentId, destination.getId())
            .last("LIMIT 10")
            .list();
        destination.setChildren(children);
    });
    return list;
}
```

特点：

- 只做了**两层**组装（国家 → 城市），不是完全递归到底层——业务上首页只需要展示两级，更深层级点击进入详情页再加载。
- 子级用 `LIMIT 10` 控制数量，避免一次返回太多数据。
- 这是“**广度优先，按需加载**”的思路：先展示一级和少量二级，用户点击再展开更深层。

面试点：树结构不一定非要一次性递归到底。根据业务场景做“层级截断 + 按需加载”，既满足前端展示，又控制数据量。这是工程思维——不追求技术上的完美递归，而是做合适的取舍。

和菜单树递归的关系：菜单树是 N 层全递归，因为后台管理系统需要看到完整的菜单结构；目的地只展示两级，是因为 C 端首页只需要展示国家 + 热门城市。两种都是树结构，只是**递归深度控制策略不同**。

2.2 递归三要素（结合项目讲解）

1. **递归出口（终止条件）：**当前节点没有子节点时停止。本项目通过 `hasChild(list, tChild)` 判断，返回 `false` 就不再进入下一层 `recursionFn`。
2. **递归体（向下一层）：**`recursionFn(list, tChild)` —— 把子节点当作新的父节点继续处理。
3. **参数收敛（问题规模缩小）：**每递归一层，处理的节点层级就更深一层，子节点集合越来越小，最终必然走到叶子节点退出。

2.3 面试可能追问的问题

Q1: 递归和迭代（循环）有什么区别？各自的优缺点？

A:

- 递归：代码简洁，符合树形/分治问题的思维模型；但每次调用都要压栈，层级深时有栈溢出风险，性能略差。
- 迭代：需要自己维护栈/队列，代码稍繁琐；但没有栈溢出风险，性能更好。
- 本项目菜单/部门树层级通常很浅（3~5 层），用递归可读性最好，是合理选择。如果数据量大、层级深（如百万级评论楼中楼），应改成迭代 + 显式栈。

Q2: 你这个递归会不会有问题？比如性能？栈溢出？

A:

- **性能**：`getChildList` 每次都要遍历整个 list 找孩子，时间复杂度 $O(n)$ ，整体 $O(n^2)$ 。如果菜单数量大（几百上千），可以先用 `Map<parentId, List<子节点>>` 做一次分组 ($O(n)$)，之后查孩子就是 $O(1)$ 。
- **栈溢出**：菜单/部门树层级有限，不会触发；但若是任意深度的数据（如组织架构、族谱），应加最大深度限制或改迭代。
- **重复计算**：`hasChild` 内部又调了一次 `getChildList`，相当于每个节点查两次孩子，可优化为只查一次。

Q3: 如何把这段递归改成迭代？

A: 用显式栈或队列。思路：把所有节点按 parentId 分组到 Map 中，然后从根节点入栈，循环出栈、挂载子节点、子节点入栈，直到栈空。这样避免了方法调用栈，可处理任意深度。

Q4: 递归和回溯是一回事吗？递归和回溯有什么区别和联系？

一、本质定义

- **递归(Recursion)** 是一种**执行机制**:方法自己调用自己,把大问题拆成同结构的子问题,直到触底(出口)再逐层返回。它是一种"代码组织方式",关注的是"怎么调用"。
- **回溯(Backtracking)** 是一种**算法思想**:通过"试探 + 撤销"穷举所有可能的解,走不通就回退一步换条路走,核心是"选择、递归、撤销选择"三步曲。它关注的是"怎么搜索解空间"。

二、联系:回溯通常用递归来实现

回溯需要维护一条"当前路径",并在失败时回退到上一个分叉点 —— 这种"进入下一层 + 返回上一层"的栈结构,天然适合用递归的方法调用栈来表达。经典回溯模板:

```
void backtrack(路径, 选择列表) {
    if (满足结束条件) {
        结果.add(路径的拷贝);    // 注意是拷贝,因为路径后面还要撤销
        return;
    }
    for (选择 : 选择列表) {
        做选择;                // 前置:把当前选择加入路径
        backtrack(路径, 选择列表); // 递归进入下一层
        撤销选择;              // 后置:把当前选择移出路径(回溯的核心!)
    }
}
```

模板里的"撤销选择"就是回溯的精髓 —— 也是它和普通递归最直观的区别。

三、核心区别对比

维度	递归	回溯
定位	执行机制(代码怎么写)	算法思想(问题怎么解)
目的	把大问题拆成同结构小问题	穷举所有可能解,找满足条件的
是否撤销	一般 不撤销 状态,结果在递归过程中累积	必须撤销 选择,保证每个分支起点干净
结果收集	通常在出口处 return,或通过副作用累积	在叶子节点拷贝当前路径加入结果集
典型场景	树遍历、阶乘、斐波那契、构建树	全排列、组合、N 皇后、数独、走迷宫
复杂度关注点	递归深度(栈溢出)	剪枝(减少搜索空间)

四、用项目例子说明区别

本项目的三个递归场景,都不是回溯:

- **菜单树/部门树**:递归向下挂载子节点,一旦挂上就不会撤销,最终所有节点都留在树里。这是"遍历型递归",不是"搜索型回溯"。
- **吐司导航**:沿 parentId 向上找,路径唯一(每个节点只有一个父),不存在"走不通要换条路"的情况,也就不需要撤销。

对比一个真正的回溯例子(面试常考):求数组的所有全排列:

```
// 回溯: 每次选一个数加入路径, 递归后再撤销, 换下一个数选
void permute(int[] nums, List<Integer> path, boolean[] used, List<List<Integer>> res) {
    if (path.size() == nums.length) {
        res.add(new ArrayList<>(path));    // 拷贝路径, 因为后面还要撤销
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        path.add(nums[i]);                // ① 做选择
        used[i] = true;
        permute(nums, path, used, res);    // ② 递归进入下一层
        used[i] = false;
        path.remove(path.size() - 1);      // ③ 撤销选择(回溯的核心!)
    }
}
```

注意第 ③ 步:递归返回后必须把刚加入的数移出 path,否则下一个分支的起点就不干净了。这种"撤销"操作在本项目的树构建递归里是**没有的**—— 因为树构建是"累积型"的,所有节点都要保留;而全排列是"搜索型"的,每条路径只能保留一种选择。

五、一句话总结

递归是"自己调自己"的执行机制,回溯是"试探+撤销"的搜索思想。回溯通常用递归实现,但递归不一定是回溯 —— 关键看**有没有"撤销选择"这一步**。本项目构建树/吐司导航是普通递归,不涉及回溯。

六、易混淆点:后序递归 ≠ 回溯算法

这是个非常容易被误判的地方。以本项目的吐司导航为例,有两种写法都能得到正确结果,但经常被误认为是"回溯":

写法一:前序递归(操作在"递"阶段,原版)

```
private void createToast(List<Destination> list, Long destId) {
    if (destId == null) return;
    Destination destination = lambdaQuery().eq(Destination::getId, destId).one();
    list.add(destination);                // ★ 先 add
    createToast(list, destination.getParentId());    // 再递归
}
// 结果: [广州, 广东, 中国] → 需要 Collections.reverse
```

写法二:后序递归(操作在"归"阶段,巧妙版)

```
private void createToast(List<Destination> list, Long destId) {
    if (destId == null) return;
    Destination destination = lambdaQuery().eq(Destination::getId, destId).one();
    createToast(list, destination.getParentId()); // 先递归到底
    list.add(destination);                       // ★ "归"时才 add
}
// 结果: [中国, 广东, 广州] → 天然正序, 不用 reverse
```

写法二很巧妙,执行流程是"一路递到底(广州→广东→中国→null),再在回返时逐层 add",所以天然得到正序。但它不是回溯算法,理由是:

判断维度	写法二(后序递归)	真正的回溯算法(如全排列)
有没有撤销操作	✗ add 之后从不 remove	✓ 递归返回后必须 remove
是否在搜索解空间	✗ 路径唯一,不存在换条路	✓ 多个候选选择,逐个试探
"回溯"指什么	指"归"阶段(递归返回,自然过程)	指"撤销选择退回分叉点"(主动操作)

关键辨析:"回溯"有两个层面的含义,不能混为一谈

层面	含义	写法二
递归执行阶段的"回溯"	指"归"的过程(递归返回),是递归的自然组成部分	✓ 利用了这个阶段
算法分类的"回溯算法"	指"试探 + 撤销选择"穷举解空间的算法	✗ 没有"撤销"步骤

判断标准(一句话):

有没有 remove / 撤销操作,是判断"回溯算法"的唯一标志。有撤销才是回溯算法,没有就是普通递归(无论前序还是后序)。

前序 vs 后序递归的本质区别:

维度	前序递归	后序递归
操作位置	递归调用之前	递归调用之后
处理顺序	先处理当前,再处理子/父	先处理子/父,再处理当前
典型对应	树的前序遍历、构建树(父先入结果)	树的后序遍历、计算目录大小(子先算完)
吐司导航结果	逆序,需 reverse	正序,无需 reverse

面试时如果被问"你这个吐司导航能不能用回溯写?",可以回答:吐司导航路径唯一,没有分叉可选,用前序或后序递归即可,不需要回溯。回溯适用于"有多条候选路径需要试探"的场景,如全排列、N 皇后。把"后序递归"误叫成"回溯"是常见错误,核心区别在于有没有"撤销选择"这一步。

Q5: 递归方法的返回值和副作用怎么设计的?

A: 本项目的 recursionFn 是 void, 通过副作用 (t.setChildren(childList)) 直接修改传入对象的状态, 不返回值。这种写法在树构建场景常见, 因为树是"原地组装"。如果追求函数式风格, 可以改成返回新节点的方式。

Q6: 除了构建树, 递归还常用在哪些场景?

A: 文件目录遍历、JSON/XML 解析、JSON 序列化 (对象图含嵌套)、分治算法 (快排、归并)、回溯算法 (全排列、N 皇后)、树的遍历 (前/中/后序)、计算斐波那契/阶乘等。

2.4 性能优化：N+1 查询 vs 全量 + Map 内存组装

这是递归/树结构场景下最常被追问的性能问题。项目代码里吐司导航的“注释掉的优化版本”就是这个思路，面试主动提出来非常加分。

2.4.1 什么是 N+1 问题？

以吐司导航为例：

```
// 原始实现：每层递归查一次 DB
private void createToast(List<Destination> list, Long destId) {
    if (destId == null) return;
    Destination dest = lambdaQuery().eq(Destination::getId, destId).one(); // 1 次 SQL
    createToast(list, dest.getParentId()); // 递归，每层都查一次
    list.add(dest);
}
```

如果目的地有 4 层（国 → 省 → 市 → 区），就要执行 **4 次 SQL**。1 个 + N 层 = N+1 次查询，这就是经典的 **N+1 问题**。

- 层级越深，查询次数越多。
- 每次查询都有网络 I/O + 数据库解析优化的开销。
- 高并发下，这 N 次查询的累积延迟会被放大。

2.4.2 优化方案：一次查全量 + Map 内存组装

```
public List<Destination> queryToastsByDestId(Long destId) {
    // 1. 一次性查询全部数据（或按业务条件过滤）
    List<Destination> allList = lambdaQuery().list(); // 1 次 SQL

    // 2. 转成 Map<id, 对象>，O(1) 查找
    Map<Long, Destination> idMap = allList.stream()
        .collect(Collectors.toMap(Destination::getId, Function.identity()));

    // 3. 从当前节点往上追溯（循环代替递归，内存操作）
    List<Destination> result = new ArrayList<>();
    Long currentId = destId;
    while (currentId != null) {
        Destination dest = idMap.get(currentId);
        if (dest == null) break;
        result.add(dest);
        currentId = dest.getParentId();
    }
    Collections.reverse(result); // 逆序 → 正序
    return result;
}
```

核心思路：把“多次查数据库”换成“一次查数据库 + 多次查内存 Map”。SQL 次数从 N 降到 **1**。

2.4.3 两种方案对比

对比维度	N+1 递归查询	全量 + Map 组装
SQL 次数	N 次（N=层级数）	1 次
内存占用	小（每次只装一条）	大（整张表加载到内存）
适合数据量	大表、层级浅	小表、层级深
代码复杂度	简单直观	需要先组装 Map
并发性能	差（多次 DB 往返）	好（DB 只查一次）

面试点：不能说"全量 Map 一定更好"，要看场景。

- **目的地表**一般只有几千到几万条，全量加载完全没问题，优化收益大。
- **菜单表**数据量更小，SysMenuServiceImpl 的菜单树也是先查全量再递归组装，用的就是这个思路。
- 如果是**百万级的分类表**，全量加载就不合适了，得用其他方案（如 Redis 缓存、SQL 递归 CTE、闭包表等）。

2.4.4 扩展：还有哪些解决 N+1 的思路？

面试可以主动展开，体现知识面：

- 1. **一次性 JOIN 查出所有层级**（SQL 层面）：MySQL 8.0 支持 `WITH RECURSIVE` 递归 CTE，一条 SQL 查出整棵子树。
- 2. **闭包表（Closure Table）**：额外维护一张 ancestor-descendant 关系表，查子树只需要一次 JOIN。写入多存关系，读取极快。读多写少的场景合适。
- 3. **路径枚举（Path Enumeration）**：每个节点存 `ancestor_path` 字段，如 `1/5/23/101`，用 `LIKE '1/5/%'` 查所有后代。简单但不支持引用完整性。
- 4. **嵌套集（Nested Set）**：用 left/right 两个数描述子树范围。查询快，但插入/移动节点成本极高，几乎不用了。
- 5. **Redis 缓存树结构**：把整棵树序列化后缓存到 Redis，读的时候直接拿。

一句话总结：项目里用的是"全量 + Map 内存组装"，是数据量不大时最简单有效的优化方案。数据量大了再考虑 SQL 递归 CTE 或闭包表等更重的方案。

三、OSS 对象存储

✦ 所属简历模块：⑤ 工程实践（与第四章合并阅读）

项目背景:文件上传模块(`travel-modules-file`)需要存储用户头像、游记图片等资源。传统本地磁盘存储存在单机容量受限、无法水平扩展、不支持高并发访问等问题,所以项目引入了对象存储服务(OSS)。本项目支持 5 种存储方式切换:本地磁盘 / MinIO / 阿里云 OSS / 七牛云 / FastDFS,通过配置 `store.type` 决定使用哪一种。

3.1 什么是对象存储

对象存储(Object Storage) 是一种以"对象"为基本单位的分布式存储架构,专门用来存海量非结构化数据(图片、视频、文档等)。与传统文件系统、数据库存储形成对比:

存储方式	存储单位	特点	典型场景
本地文件系统	文件+目录树	路径层级深、单机容量受限、无冗余	小项目临时存储
数据库(BLOB)	表字段	读写慢、占连接、影响业务查询	极少用
对象存储	对象(扁平,Key-Value)	海量扩展、HTTP 直接访问、高可用	图片/视频/文档

对象存储的核心概念:

- **Bucket(桶/存储空间)**:最顶层的容器,类似"文件夹"。每个对象必须属于某个 Bucket。命名全局唯一。
- **Object(对象)**:存储的基本单元,由 **Key(对象名)** + **Data(数据)** + **Metadata(元数据)** 组成。Key 类似文件路径,但对象存储是**扁平的**,没有真正的目录层级(目录是通过 Key 前缀模拟的)。
- **Endpoint(接入点)**:访问 OSS 的域名,如 `https://oss-cn-hangzhou.aliyuncs.com`。

3.2 为什么不用本地存储而用 OSS

维度	本地磁盘存储	对象存储(OSS)
扩展性	单机容量上限,加盘要停机	近乎无限扩展,按需扩容
高可用	服务器宕机即丢失(除非做 RAID)	多副本/纠删码,数据可靠性 99.9999999999%(12 个 9)
访问方式	需要服务器中转(Nginx 静态映射)	HTTP 直接访问,带 CDN 加速
跨服务共享	微服务间无法直接访问彼此磁盘	所有服务通过同一 Endpoint 访问
成本	硬件成本 + 运维	按用量付费,小项目几乎免费额度

本项目微服务架构下,如果用本地存储,文件存到 file 服务的磁盘上,其他服务(member/strategy)要访问就得走 HTTP 再中转一次,效率低且耦合。用 OSS 后,所有服务拿到 URL 就能直接访问,彻底解耦。

3.3 项目支持的 5 种存储方案对比

本项目通过 `store.type` 配置切换:

方案	havingValue	类型	说明
本地磁盘	<code>local</code>	自建	存到服务器本地磁盘 + Spring 静态资源映射
MinIO	<code>minio</code>	自建	开源对象存储,可私有化部署,兼容 S3 协议
阿里云 OSS	<code>alioss</code>	公有云	阿里云提供的对象存储服务
七牛云	<code>qiniu</code>	公有云	七牛云对象存储(Kodo)
FastDFS	<code>fdfs</code>	自建	早期开源分布式文件系统

方案一:MinIO(项目主推)

MinIO 是开源的对象存储,兼容 Amazon S3 API,适合私有化部署(自己搭服务器,数据不出内网)。本项目 `MinioSysFileServiceImpl` 的上传流程:

```
@Override
public String uploadFile(MultipartFile file) throws Exception {
    String fileName = FileUploadUtils.extractFilename(file);
    InputStream inputStream = file.getInputStream();
    PutObjectArgs args = PutObjectArgs.builder()
        .bucket(minioConfig.getBucketName()) // 桶名
        .object(fileName) // 对象 Key
        .stream(inputStream, file.getSize(), -1)
        .contentType(file.getContentType())
        .build();
    client.putObject(args); // 上传到 MinIO
    return minioConfig.getUrl() + "/" + minioConfig.getBucketName() + "/" + fileName;
}
```

配套配置类 `MinioConfig` ,用 `@ConfigurationProperties(prefix="minio")` 绑定连接参数,并 `@Bean` 创建 `MinioClient` :

```

@Configuration
@ConfigurationProperties(prefix = "minio")
public class MinioConfig {
    private String url;           // 服务地址, 如 http://127.0.0.1:9000
    private String accessKey;     // 账号
    private String secretKey;     // 密码
    private String bucketName;   // 桶名

    @Bean
    public MinioClient getMinioClient() {
        return MinioClient.builder()
            .endpoint(url)
            .credentials(accessKey, secretKey)
            .build();
    }
}

```

nacos 配置示例:

```

store:
  type: minio          # 切换存储方式
minio:
  url: http://127.0.0.1:9000
  access-key: minioadmin
  secret-key: minioadmin
  bucket-name: wolf2w

```

方案二:阿里云 OSS

阿里云提供的公有云对象存储,数据存在阿里云机房,按用量付费。 `AliOssSysFileServiceImpl` 的特点是**每次上传都 new 一个 OSSClient**,用完 **shutdown**(与 MinIO 的单例 client 不同):

```

@Override
public String uploadFile(MultipartFile file) throws Exception {
    String fileName = UUID.randomUUID() + "." + FilenameUtils.getExtension(file.getOriginalFilename());
    // 每次创建客户端(适合低频上传场景;高频应复用)
    OSS ossClient = new OSSClientBuilder().build(endpoint, accessKeyId, accessKeySecret);
    try {
        PutObjectRequest request = new PutObjectRequest(bucketName, fileName, file.getInputStream());
        ossClient.putObject(request);
        return url + fileName;
    } finally {
        ossClient.shutdown(); // 必须关闭,否则连接泄漏
    }
}

```

方案三:七牛云

与阿里云 OSS 类似,也是公有云存储,SDK 不同。上传需要先用 AccessKey/SecretKey 生成 `upToken` (上传凭证),再用 `UploadManager.put` 上传:

```

Auth auth = Auth.create(accessKey, secretKey);
String upToken = auth.uploadToken(bucket);
Response response = uploadManager.put(file.getInputStream(), key, upToken, null, null);

```

七牛云对个人开发者有免费额度(10GB 存储 + 10GB/月 流量),适合学习项目。

方案四:FastDFS

FastDFS 是早期开源的分布式文件系统(C 语言编写),架构上分 Tracker(调度)和 Storage(存储)两组节点。本项目用 `tobato` 开源客户端:

```

StorePath storePath = storageClient.uploadFile(inputStream, file.getSize(),
    FileTypeUtils.getExtension(file), null);
return domain + "/" + storePath.getFullPath(); // group1/M00/00/00/xxx.jpg

```


FastDFS 返回的 URL 带 `group` 前缀(如 `group1/M00/...`),这是它独有的分组机制。现在新项目更倾向用 MinIO, FastDFS 逐渐被淘汰。

方案五:本地磁盘(兜底方案)

存到服务器本地磁盘,用 Spring 的 `ResourceHandlerRegistry` 做 URL 到文件路径的映射:

```
// LocalSysFileServiceImpl
String name = FileUploadUtils.upload(localFilePath, file);
String url = domain + localFilePrefix + name; // http://localhost:9300/static/xxx.jpg

// ResourcesConfig 做静态资源映射
registry.addResourceHandler(localFilePrefix + "/*")
        .addResourceLocations("file:" + localFilePath + File.separator);
```

本地存储还带了 `matchIfMissing = true` 参数,作为配置缺失时的兜底首选实现(详见下一节策略模式)。

3.4 面试可能追问的问题

Q1:为什么要用对象存储,不用本地文件系统?

A:本地文件系统在微服务架构下有三个硬伤:① 单机容量有限,扩展要停机加盘;② 服务间无法直接访问彼此磁盘,必须 HTTP 中转,耦合且低效;③ 没有多副本,服务器宕机数据丢失。对象存储提供 HTTP 直接访问、近乎无限扩展、多副本高可用,微服务拿到 URL 就能共享访问。本项目 file 服务专门做上传,其他服务(member/strategy)只存 URL,彻底解耦。

Q2:MinIO 和阿里云 OSS 有什么区别?怎么选?

A:

- **部署方式**:MinIO 是开源的,自己搭服务器私有化部署,数据在内网;OSS 是阿里云托管的,数据在阿里云机房。
- **成本**:MinIO 硬件成本一次性投入,数据量大时便宜;OSS 按用量付费,小项目有免费额度,量大时贵。
- **协议**:MinIO 兼容 S3 API, OSS 是自家协议(也兼容 S3)。
- **选择**:对数据安全要求高、有运维能力的公司选 MinIO;创业公司/个人项目图省事选 OSS。本项目两种都支持,通过配置切换。

Q3:MinIO 的 Bucket 和 Object 是什么关系?

A:Bucket 是最顶层的容器, Object 是存储在 Bucket 里的对象。一个 Bucket 可以存无数个 Object,每个 Object 有唯一的 Key(类似文件路径)。访问 URL 的格式是 `endpoint/bucket/key`,如 `http://127.0.0.1:9000/wolf2w/2026/08/11/xxx.jpg`。Bucket 命名全局唯一,创建后不能改名。

Q4:上传文件时,文件名怎么生成?为什么要重命名?

A:本项目用 `FileUploadUtils.extractFilename(file)` 生成基于日期目录 + UUID 的文件名(如 `2026/08/11/uuid.jpg`),原因有三:① 避免同名文件覆盖(用户都传 "avatar.jpg" 会冲突);② UUID 防止猜测 URL 恶意访问;③ 日期目录方便管理和清理。阿里云实现里用的是 `UUID.randomUUID() + 扩展名`,思路一致。

Q5:OSS 上传大文件(如 1GB 视频)怎么处理?

A:大文件不能一次性上传(内存扛不住、网络中断要重传),用**分片上传**(Multipart Upload):① 客户端把文件切成多个小片(如 5MB 一片);② 并发上传各片, OSS 端暂存;③ 全部上传完调 `CompleteMultipartUpload` 合并。MinIO/OSS SDK 都提供了 `uploadObject` 的分片版本。本项目只演示了普通上传,生产环境传大文件应改用分片。

Q6:对象存储是扁平的,怎么实现"目录"效果?

A:对象存储**没有真正的目录**,目录是通过 Key 的前缀模拟的。比如 Key 是 `2026/08/11/xxx.jpg`,OSS 控制台会显示成 `2026 > 08 > 11 > xxx.jpg` 的层级,但底层其实就是个扁平的 Key。这样设计的好处是 Key 查找是 O(1) 的哈希查找,不像文件系统要逐级遍历目录。

Q7:文件上传的安全性怎么保证?

A:

1. **鉴权**:AccessKey/SecretKey 不暴露在前端,上传请求走后端 file 服务中转。
2. **文件名校验**:重命名为 UUID,防止路径穿越攻击(如 `../../etc/passwd`)。
3. **类型校验**: `FileUploadUtils` 校验扩展名和 `contentType`,防止上传 .exe 等可执行文件。

4. **大小限制**:Spring 配置 `spring.servlet.multipart.max-file-size` 限制单文件大小。
5. **私有 Bucket**:敏感数据用私有 Bucket,访问时生成带签名的临时 URL(预签名 URL),过期失效。

Q8:FastDFS 为什么逐渐被 MinIO 淘汰?

A:

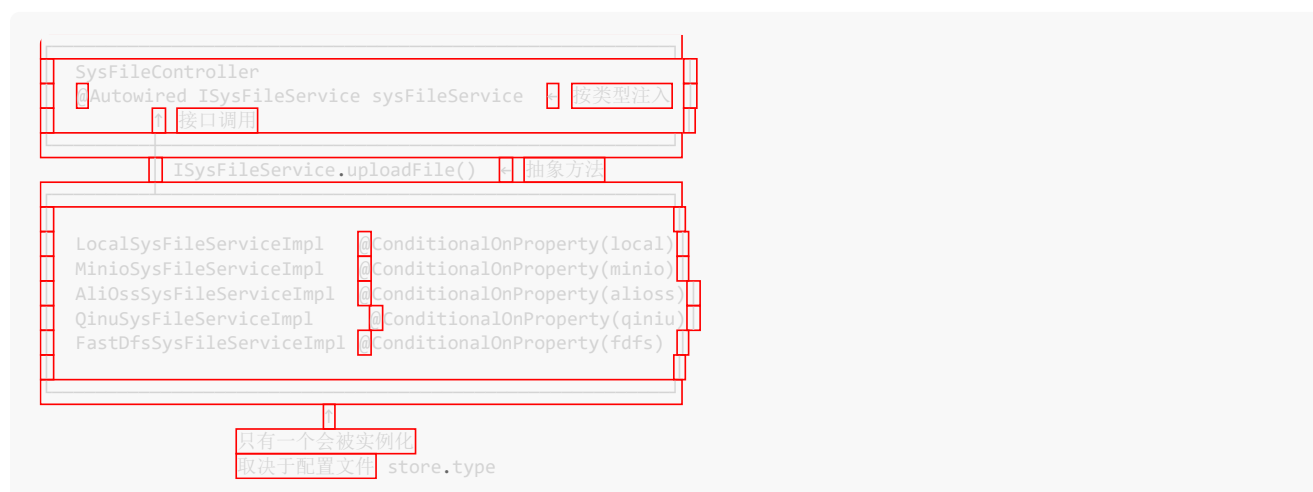
1. **协议**:FastDFS 是私有协议,生态封闭;MinIO 兼容 S3 协议,生态丰富(各种工具、SDK 通用)。
 2. **部署**:FastDFS 要装 Tracker + Storage 两组服务,部署复杂;MinIO 单一二进制,开箱即用。
 3. **运维**:FastDFS 是 C 语言写的,二次开发难;MinIO 是 Go 写的,跨平台易维护。
 4. **功能**:MinIO 支持版本控制、生命周期管理、Lambda 计算,FastDFS 功能单一。
- 所以新项目基本都选 MinIO,FastDFS 多见于老项目。

四、策略模式 + @ConditionalOnProperty 实现多态切换

✦ 所属简历模块: ⑤ 工程实践 (与第三章 OSS 存储方案合并阅读)

项目背景:文件上传模块(`travel-modules-file`)需要支持 5 种存储方式切换 —— 本地磁盘 / MinIO / 阿里云 OSS / 七牛云 / FastDFS。如果用 if-else 硬编码,每加一种存储就要改业务代码,违反开闭原则。本项目用 **接口多实现 + @ConditionalOnProperty 条件装配** 实现了运行时多态切换,调用方零感知。

4.1 整体架构图



4.2 核心实现

第一步:定义统一接口(抽象契约)

`ISysFileService` 规定所有存储方式都必须实现"传 MultipartFile,返回访问 URL"的方法:

```

public interface ISysFileService {
    public String uploadFile(MultipartFile file) throws Exception;
}

```

这是多态的基础 —— 调用方永远只面向接口编程,不感知具体实现类。

第二步:每个存储方式写一个实现类 + 条件注解

5 个实现类都实现同一接口,类上标 `@ConditionalOnProperty` 做条件筛选:

实现类	havingValue	底层技术	备注
LocalSysFileServiceImpl	local	本地磁盘 + Spring 资源映射	matchIfMissing=true，配置缺失时兜底
MinioSysFileServiceImpl	minio	MinIO 对象存储 SDK	
AliOssSysFileServiceImpl	alioss	阿里云 OSS SDK	
QiniuSysFileServiceImpl	qiniu	七牛云 SDK	
FastDfsSysFileServiceImpl	fdfs	FastDFS Java 客户端	

以本地实现为例:

```
@Service
@ConditionalOnProperty(prefix = "store", name = "type", havingValue = "local", matchIfMissing = true)
public class LocalSysFileServiceImpl implements ISysFileService {

    @Value("${file.prefix}")
    public String localFilePrefix;

    @Value("${file.domain}")
    public String domain;

    @Value("${file.path}")
    private String localFilePath;

    @Override
    public String uploadFile(MultipartFile file) throws Exception {
        String name = FileUploadUtils.upload(localFilePath, file);
        String url = domain + localFilePrefix + name;
        return url;
    }
}
```

第三步:Spring 启动时按配置注册 Bean

@ConditionalOnProperty 底层走的是 @Conditional + OnPropertyCondition 匹配器,匹配逻辑:

读取配置项 store.type = xxx

遍历 5 个 Service 实现类

havingValue == "local"	匹配?	✓	注册 Bean
havingValue == "minio"	匹配?	✗	跳过
havingValue == "alioss"	匹配?	✗	跳过
havingValue == "qiniu"	匹配?	✗	跳过
havingValue == "fdfs"	匹配?	✗	跳过

Spring 容器里只有 1 个 ISysFileService Bean

注解参数解读:

```
@ConditionalOnProperty(
    prefix = "store", // 配置前缀
    name = "type", // 属性名,最终匹配 store.type
    havingValue = "minio" // 当配置值等于此字符串时,才注册 Bean
)
```

关键行为:

场景	行为
<code>store.type=minio</code>	只有 <code>MinioSysFileServiceImpl</code> 实例化
<code>store.type=local</code>	只有 <code>LocalSysFileServiceImpl</code> 实例化
配置不存在	<code>LocalSysFileServiceImpl</code> 实例化 (<code>matchIfMissing = true</code> 兜底)
配置值写错	全部跳过,找不到 Bean,启动报 <code>NoSuchBeanDefinitionException</code>

注: `LocalSysFileServiceImpl` 加了 `matchIfMissing = true`,配置缺失时默认匹配成功,作为兜底实现。其他 4 个实现类没有此参数。

第四步:Controller 面向接口注入,零 if-else

```
@RestController
public class SysFileController {

    @Autowired
    private ISysFileService sysFileService;    // 接口类型注入!

    @PostMapping("upload")
    public R<SysFile> upload(MultipartFile file) {
        // 直接调接口方法,JVM 通过虚方法表分派到具体实现
        String url = sysFileService.uploadFile(file);
        SysFile sysFile = new SysFile();
        sysFile.setName(FileUtils.getName(url));
        sysFile.setUrl(url);
        return R.ok(sysFile);
    }
}
```

这就是多态的精髓——调用方完全不感知具体实现类,只跟接口打交道。切换存储方案时:

1. Controller 一行代码不用改
2. Service 实现类也不用改
3. 只需改配置文件里的 `store.type` 即可

对比不用多态的写法(反模式)

```
// ✗ 糟糕: 调用方写死 if-else, 每加一种存储都要改这段代码
@Value("${store.type}")
private String storeType;

public String upload(MultipartFile file) {
    if ("local".equals(storeType)) {
        return localService.uploadFile(file);
    } else if ("minio".equals(storeType)) {
        return minioService.uploadFile(file);
    } else if ("alioss".equals(storeType)) {
        return alioSsService.uploadFile(file);
    } ... // 每加一种都要加 else if
}
```

用 `@ConditionalOnProperty` + 接口多实现 + `@Autowired` 接口注入,本质上就是把"运行时 if-else 分支"搬到了"Spring 启动时的 Bean 注册阶段",既做到了策略切换灵活,又保持了调用代码的简洁。

4.3 两层条件装配:Service + 配套 Config

`@ConditionalOnProperty` 不只用在 Service 层,项目还通过配置类自动绑定做了第二层的条件化准备。以 MinIO 为例:

- `MinioConfig` 用 `@ConfigurationProperties(prefix="minio")` 绑定连接参数

- 同时 `@Bean` 方法实例化 `MinioClient` (SDK 客户端)
- `MinioSysFileServiceImpl` 里直接 `@Autowired private MinioClient client`

```
store.type = minio
MinioConfig 被扫描到(ConfigurationProperties,无条件,但值可以为 null)
MinioClient Bean 注册
MinioSysFileServiceImpl 条件匹配成功
@Autowired MinioConfig minioConfig (取参数)
@Autowired MinioClient client (用 SDK 客户端上传)
```

这里 `MinioConfig`/`AliOssConfig`/`QiniuConfig` 本身没有加 `@ConditionalOnProperty`,所以不管 `store.type` 设什么,它们都会被 Spring 加载(只是值可能用不到)。如果想追求极致,可以给 `Config` 也加同一条 `@ConditionalOnProperty`,避免无用配置类实例化。

4.4 matchIfMissing 的配合细节(防御性设计)

`LocalSysFileServiceImpl` 上除了条件注解,还有一个 `matchIfMissing = true` 参数:

```
@Service
@ConditionalOnProperty(prefix = "store", name = "type", havingValue = "local", matchIfMissing = true)
//                                     ↑ 注意这个
public class LocalSysFileServiceImpl implements ISysFileService
```

作用:当配置文件中 `store.type` 没有配置(缺失)时, `matchIfMissing = true` 让该 Bean 默认匹配成功,作为兜底实现被注册。这样即使忘记配 `store.type`,项目也能正常启动并使用本地存储,不会报 `NoSuchBeanDefinitionException`。

`@ConditionalOnProperty` 保证"只注册一个", `matchIfMissing = true` 保证"配置缺失时也有默认实现",两者配合是双重保险。

4.5 面试题模板(结合项目)

面试官问:在项目里你是怎么用多态的?/怎么根据配置切换实现?

答:我项目里文件上传模块做了 5 种存储方式的切换(本地/MinIO/阿里云 OSS/七牛云/FastDFS),用的是 **接口 + 多实现类 + `@ConditionalOnProperty` 条件装配** 的方案。

第一步:定义统一接口 `ISysFileService`,规定 `uploadFile(MultipartFile)` 方法,所有存储都遵守这个契约。这样调用方(Controller)永远只面对接口编程。

第二步:每个存储方式写一个实现类,实现同一接口,然后在类上标 `@ConditionalOnProperty(prefix="store", name="type", havingValue="具体类型")`,比如 MinIO 实现就写 `havingValue="minio"`。

第三步:Spring 启动时会读配置文件的 `store.type`,匹配哪个 `havingValue`,就只实例化哪个 Service Bean 注册到容器里,其他 4 个全部跳过不加载。

第四步:Controller 直接 `@Autowired ISysFileService` 注入。因为容器里永远只有 1 个 Bean,所以不会报多 Bean 冲突,调用 `uploadFile()` 时 JVM 会通过虚方法表分派到具体实现类,这就是运行时多态。

它的好处是:切换存储方案时,Controller 和业务代码一行都不用改,只改配置文件里的 `store.type` 即可;新增一种存储(比如腾讯云 COS),只需要新增一个实现类加注解,完全符合开闭原则。这比写一堆 if-else 判断存储类型的方案优雅太多。

4.6 面试可能追问的问题

Q1:@ConditionalOnProperty 的作用是什么?底层怎么实现的?

A:它是 Spring Boot 提供的条件装配注解,根据配置文件中的属性值决定是否注册 Bean。底层走的是 `@Conditional` 机制,匹配器是

`OnPropertyCondition` 在 Spring 容器启动注册 Bean 定义阶段触发:读取 `prefix.name` 对应的配置值,跟 `havingValue` 比较,相等才让该 Bean 定义通过,否则直接跳过,该类不会被实例化。

Q2:@ConditionalOnProperty 和 @Profile 有什么区别?

A:

- `@Profile` 按环境切换(dev/test/prod),粒度粗,一个环境通常只激活一套配置。
- `@ConditionalOnProperty` 按任意配置项切换,粒度细,可以在同一环境里通过多个配置项组合控制多组 Bean 的加载。本项目 5 种存储方式在同一环境里通过 `store.type` 切换,用 `@Profile` 就不合适。

Q3:如果配置项没写,会怎样?怎么兜底?

A:默认情况下配置项不存在时, `@ConditionalOnProperty` 匹配失败,Bean 不会被注册,启动时会报 `NoSuchBeanDefinitionException`。兜底方案有两种:

1. 加 `matchIfMissing = true`,让配置缺失时默认匹配成功(适合做兜底实现)。本项目 `LocalSysFileServiceImpl` 用的就是这个方案。
2. 给某个实现类同时加 `@Primary` + `matchIfMissing = true`,作为默认实现(更防御性,但本项目未使用)。

Q4:多个实现类都没加条件注解,Autowired 注入接口会怎样?

A:Spring 会报 `NoUniqueBeanDefinitionException`,因为找到多个候选 Bean 无法决定注入哪个。解决方法:@Primary 指定首选、@Qualifier 按名字指定、或者用条件注解保证只有一个被注册。本项目用 `@ConditionalOnProperty` 保证唯一,同时 `LocalSysFileServiceImpl` 加 `matchIfMissing = true` 做兜底,即使配置缺失也能正常启动。

Q5:这种方案和策略模式有什么关系?

A:这就是策略模式的 Spring 实现版。经典策略模式是:接口 + 多个实现类 + 一个 Context 持有引用 + 客户端手动选择策略。Spring 版把"手动选择"换成了"容器根据配置自动选择",Context 就是 Spring 容器本身,客户端只需 `@Autowired` 接口。优点是更解耦,缺点是策略切换需要重启(因为是启动时注册),不能运行时动态切换。

Q6:能运行时动态切换存储方式吗?

A:用 `@ConditionalOnProperty` 不行,因为它是启动时决定 Bean 注册的。如果需要运行时动态切换,可以用:① `@Autowired Map<String, ISysFileService>` 注入所有实现,按 key 动态选;② 自己维护一个策略 Map,根据配置实时路由;③ 用 Spring Event 监听配置变更热替换 Bean(复杂)。但文件存储一般不需要频繁切换,启动时固定一种就够用了。

Q7:这和 Spring Cloud 的自动装配(spring.factories / AutoConfiguration.imports)有什么区别?

A:

- `@ConditionalOnProperty` 是在已被扫描到的类上加条件,决定要不要实例化。
- `AutoConfiguration.imports` 是解决类本身都没被扫描到的问题,把外部 jar 里的配置类引入容器,通常会再配合 `@ConditionalOnProperty` 做条件装配。
- 本项目的 5 个 Service 在本模块内,能被扫描到,所以只用 `@ConditionalOnProperty` 就够了;如果是做成独立 starter 给别人用,就需要配合 `AutoConfiguration.imports`。

五、游记模块 (Note)

✦ 所属简历模块: 附录 C — 游记模块完整实现参考 (面试选讲)

项目背景: `travel-modules-note` 是 2026-08-10 新增的微服务模块,负责游记的发布、详情展示、评论、搜索、收藏/点赞等完整业务链路。该模块涉及三个核心知识点: **大字段分离** (NoteContent 独立存富文本)、**微服务数据聚合** (Feign 调 member 服务查作者)、**Redis 缓存计数** (HINCRBY 原子操作 + 定时持久化)。

5.1 整体架构



5.2 表结构设计 —— 大字段分离

三张表：

表	实体	说明
ta_note	Note.java	游记主表（标题/概要/封面/旅游时间/花费/天数/统计字段）
ta_note_content	NoteContent.java	游记富文本内容（大字段独立存储，id 与 note.id 1:1）
ta_note_comment	NoteComment.java	游记评论（关联用户信息，冗余 noteTitle）

大字段分离的核心代码：

```
// NoteContent: id 非自增，与 ta_note.id 保持 1:1
@TableName("ta_note_content")
public class NoteContent {
    @TableId // 非自增!
    private Long id; // 与 ta_note.id 一致
    private String content; // 富文本 HTML，可能几十 KB
}
```

为什么这么设计？

对比	存一张表	大字段分离
列表查询性能	全表扫描加载富文本，慢	只查主表，不碰大字段
索引效率	大字段占空间，索引页可用的行数少	主表紧凑，索引效率高
更新灵活性	修改标题也会锁住大字段行	互不影响
缓存策略	做不到差异化	主表可全量缓存，内容按需加载

本项目 Strategy 模块的 `StrategyContent` 也是同样思路。面试时可以说：“主表存高频查询字段，大字段独立存储，避免全表扫描时加载大文本影响性能”。

5.3 分页查询 —— 多条件动态筛选

`NoteServiceImpl.queryPage` 是本模块最核心的查询方法，通过 `LambdaQueryWrapper` 动态拼条件：

```
public IPage<Note> queryPage(NoteQuery qo) {
    IPage<Note> page = new Page<>(qo.getCurrentPage(), qo.getPageSize());
    LambdaQueryWrapper<Note> wrapper = new LambdaQueryWrapper<>();

    // 1. 前后台分离: admin=true 不过滤 is_public, 可查草稿/待审核
    if (!Boolean.TRUE.equals(qo.getAdmin())) {
        wrapper.in(Note::getIsPublic, "1", "true"); // 兼容历史数据
    }

    // 2. 目的地筛选
    if (qo.getDestId() != null) {
        wrapper.eq(Note::getDestId, qo.getDestId());
    }

    // 3. 出发时间段筛选 (1-6 → 1-2月、3-4月...)
    // MONTH(travel_time) BETWEEN {0} AND {1}
    if (qo.getTravelTimeType() != null) {
        wrapper.apply("MONTH(travel_time) BETWEEN {0} AND {1}", monthStart, monthEnd);
    }

    // 4. 人均花费筛选 (1:1-999 / 2:1K-6K / 3:6K-20K / 4:20K+)
    if (qo.getConsumeType() != null) {
        wrapper.between(Note::getAvgConsume, min, max);
    }

    // 5. 排序: viewnum 最热 / create_time 最新
    if ("viewnum".equals(qo.getOrderBy())) {
        wrapper.orderByDesc(Note::getViewnum);
    } else {
        wrapper.orderByDesc(Note::getCreateTime);
    }
    return page(page, wrapper);
}
```

涉及的知识点：

知识点	代码体现
前后台分离查询	<code>admin</code> 标识控制 <code>is_public</code> 过滤
兼容历史数据	<code>wrapper.in(isPublic, "1", "true")</code>
MySQL 函数查询	<code>MONTH(travel_time) BETWEEN</code>
动态排序	前端传 <code>orderBy</code> 切换排序字段
Lambda 安全引用	<code>Note::getDestId</code> 避免字段名拼写错误

5.4 游记详情 —— 微服务数据聚合 + 降级兜底

`NoteServiceImpl.detail` 是典型的微服务数据聚合场景，聚合了 4 个数据源：

```
detail(id)
├─ 1. 查 Note 主表（本服务 DB）
├─ 2. BeanUtils.copyProperties(note, vo) → 自动拷贝 20+ 字段
├─ 3. 查 NoteContent 表 → 富文本内容（本服务 DB）
├─ 4. Feign 调 member 服务 → 作者信息（跨服务 RPC）
│   └─ 失败降级：用 Note 冗余字段兜底
├─ 5. person 数字 → 中文映射（1→亲子出游，2→情侣出行...）
└─ 6. favorFlag / isFollow 默认 false
```

VO 继承式设计 —— 不扁平展开：

```
// 继承 Note，只补 VO 特有字段，不用手写 20+ 个 setXxx()
public class NoteDetailVO extends Note {
    private NoteContent content; // 富文本
    private UserInfo author; // 作者信息
    private Integer day; // 前端模板别名
    private String personDisplay; // 中文映射
    private Boolean favorFlag; // 收藏状态
    private Boolean isFollow; // 关注状态
}

// 使用：自动拷贝 20+ 基础字段
NoteDetailVO vo = new NoteDetailVO();
BeanUtils.copyProperties(note, vo); // 一行代码替代 20+ 行 setXxx()
```

Feign 调用的降级策略（容错设计）：

```
// 尝试调 member 服务查作者
try {
    R<UserInfo> r = remoteUserInfoService.getOne(note.getAuthorId(), "inner");
    if (r != null && r.getCode() == R.SUCCESS) {
        vo.setAuthor(r.getData());
    }
} catch (Exception ignored) {
    // member 服务挂了，静默降级
}

// 降级兜底：用 Note 冗余字段
if (vo.getAuthor() == null) {
    UserInfo fallback = new UserInfo();
    fallback.setNickname(note.getAuthorNickname() != null
        ? note.getAuthorNickname() : "匿名用户");
    fallback.setHeadImgUrl(note.getAuthorHeadImgUrl());
    vo.setAuthor(fallback);
}
```

核心思想：即使 member 服务挂了，游记详情页仍能正常展示，只是作者信息用冗余字段兜底，不会直接 500。这是微服务架构中典型的服务降级模式。

5.5 评论系统 —— 多表关联 + 计数联动

发表评论时做三件事，保证数据一致性：

```
public void addContent(NoteComment comment) {
    // 1. 设置当前登录用户
    Long userId = SecurityUtils.getUserId();
    comment.setUserId(userId);
    comment.setCreateTime(new Date());
    comment.setStatus("0");

    // 2. 冗余存储游记标题（避免列表页跨表 JOIN）
    Note note = noteService.getById(comment.getNoteId());
    if (note != null) {
        comment.setNoteTitle(note.getTitle());
    }
    baseMapper.insert(comment);

    // 3. 更新游记回复数（SQL 原子操作，避免并发问题）
    noteService.lambdaUpdate()
        .eq(Note::getId, comment.getNoteId())
        .setSql("replynum = replynum + 1") // ★ SQL 原子操作
        .update();
}
```

为什么用 `setSql("replynum = replynum + 1")` 而不是 "先查后更新"？

✖

错误方案：先查后更新（存在竞态条件）
int count = note.getReplnum(); // 线程A读到 5
note.setReplnum(count + 1); // 线程B也读到 5，写 6
updateById(note); // 两个线程都写 6，实际应该是 7

✔

正确方案：SQL 原子操作
setSql("replnum = replnum + 1") // MySQL 行锁保证原子性

5.6 点赞/收藏 —— DB 原子操作 + 演进思路

```
// 点赞
@PostMapping("/star/{id}")
public R<?> star(@PathVariable Long id) {
    noteService.lambdaUpdate()
        .eq(Note::getId, id)
        .setSql("thumbsupnum = COALESCE(thumbsupnum, 0) + 1")
        .update();
    return R.ok();
}

// 取消收藏（带防负数保护）
@PostMapping("/uncollect/{id}")
public R<?> uncollect(@PathVariable Long id) {
    noteService.lambdaUpdate()
        .eq(Note::getId, id)
        .gt(Note::getFavornum, 0) // ★ 防负数
        .setSql("favornum = favornum - 1")
        .update();
    return R.ok();
}
```

代码注释明确标注了：“后续迁移到 Redis 异步刷 DB”。这是项目演进思路：先用 DB 快速上线验证业务，再逐步优化到 Redis 异步方案（参考 Strategy 模块的 `incrementCount` + `statisHashMapPersist` 组合）。

5.7 路径别名兼容 —— 解决前端路由不一致

`TravelsAliasController` 不设类级别 `@RequestMapping`，直接映射多个不同前缀的路径：

```
@RestController // 注意：没有 @RequestMapping 类级别注解
public class TravelsAliasController {

    @GetMapping("/travels/user/query") // 我的游记
    public R<IPage<Note>> myTravels(NoteQuery qo) { ... }

    @GetMapping("/destinations/regions") // 写游记- 区域列表
    public R<List<Region>> regions(...) { ... }
}
```

为什么需要这个 Controller?

因为网关统一前缀 `/note` 将所有 `/note/**` 请求路由到 `note` 模块，但前端定义的路径种类多样：

前端路径	实际映射	来源
<code>/note/notes/query</code>	NoteController	<code>@RequestMapping("notes")</code>
<code>/note/travels/user/query</code>	TravelsAliasController	路径别名
<code>/note/destinations/regions</code>	TravelsAliasController	路径别名
<code>/note/notes/star/{id}</code>	NoteController	<code>@RequestMapping("notes")</code>

如果不用别名 Controller，要么在网关层做复杂的路由规则，要么改前端路径。别名 Controller 是最轻量的方案，一个类解决所有路径兼容问题。

5.8 Feign API 包分离设计

本项目采用 "API 包 + 业务实现包" 分离模式：

```

travel-api-note (API 包)
├── domain/Note.java
├── domain/NoteContent.java
├── domain/NoteComment.java
├── RemoteNoteService.java
│   (Feign 声明 + 降级工厂)
└──

travel-modules-note (业务实现包)
├── service/impl/NoteServiceImpl.java
└── controller/NoteController.java
  
```

其他模块如何使用？只需引入 API 包：

```

<!-- 比如 strategy 模块要查游记数据 -->
<dependency>
  <groupId>cn.wolfcode.wolf2w</groupId>
  <artifactId>travel-api-note</artifactId>
</dependency>
  
```

然后直接注入 Feign 接口调用：

```

@Autowired
private RemoteNoteService remoteNoteService;

// 查目的地阅读量 top 游记
R<List<Note>> r = remoteNoteService.viewnumTop(destId);
  
```

好处：调用方不依赖业务实现，只依赖 API 定义。API 包轻量（只有 Domain + Feign 接口），业务实现包可以独立部署、独立扩缩容。

5.9 面试题模板（结合项目）

面试官问：介绍下你做的游记模块？

答：我做的游记模块是旅游平台的一个独立微服务，包含游记的发布、详情展示、评论、收藏、搜索等功能。

表设计上，我用了**大字段分离**模式。游记主表 `ta_note` 存标题、概要、统计字段等高频查询字段，富文本内容单独存 `ta_note_content` 表，主键和游记 id 保持 1:1。这样列表查询时不会加载几十 KB 的富文本，索引效率也更高。

查询上，分页接口支持多条件动态筛选：目的地、出行时间、人均花费、出行天数，通过 `LambdaQueryWrapper` 动态拼条件，`admin` 标识控制前后台数据隔离。

详情聚合上，用 Feign 跨服务调 member 服务查作者信息，同时做了**降级兜底**：member 服务挂了时，用 Note 表的冗余字段（nickname、headImgUrl）填充，保证详情页不报 500。

评论系统的回复数更新用了 `setSql("replynum = replynum + 1")` 的 SQL 原子操作，避免"先查后更新"的并发竞态。

VO 设计上，`NoteDetailVO` 继承 `Note`，用 `BeanUtils.copyProperties` 自动拷贝 20+ 基础字段，只补 VO 特有的扩展字段，避免了手动扁平展开的冗余代码。

架构上，API 包和业务实现包分离，其他服务只需引入 `travel-api-note` 就能通过 Feign 调用，不依赖业务实现。

5.10 面试可能追问的问题

Q1: 大字段分离有什么好处? 什么场景下应该用?

A:

- **好处**: ① 列表查询不加载大文本, 性能提升; ② 索引页能容纳更多行, 索引效率高; ③ 更新主表字段不锁大字段行, 并发更好; ④ 可以做差异化缓存 (主表全量缓存, 内容按需加载)。
- **适用场景**: 当某列的数据量远大于其他列 (如富文本 HTML、JSON 大对象、BLOB 文件), 且列表查询通常不需要该列时, 就应该分离。
- **本项目**: Note 的 `content` 字段 (富文本) 和 Strategy 的 `StrategyContent` 都是这个思路。

Q2: 为什么 NoteContent 的 id 不用自增, 而是和 Note.id 保持一致?

A:

- 保证 1:1 关联, `NoteContent.id = Note.id`, 查询时直接 `selectById(noteId)`, 不需要 JOIN 或额外字段。
- 如果 NoteContent 用自增 id, 就需要一个外键字段关联 Note, 多一次关联查询。
- 1:1 关系通常用共享主键, 这是数据库设计中的常见模式。

Q3: Feign 调用失败时, 你是怎么处理的?

A: 项目中做了两层防护:

1. **代码级 try-catch**: `NoteServiceImpl.detail` 中 try-catch 包裹 Feign 调用, 失败时静默降级, 用冗余字段兜底。
2. **框架级降级**: `RemoteNoteService` 上声明了 `fallbackFactory = RemoteNoteFallbackFactory.class`, 通过 Sentinel 熔断, 服务不可用时返回空列表, 不会拖垮调用方。

Q4: 评论回复数 +1 为什么用 setSql 而不是先查后更新?

A: 高并发场景下, "先查后更新"存在竞态条件 (两个线程同时读到 5, 都写 6, 实际应该是 7)。 `setSql("replynum = replynum + 1")` 利用 MySQL 的行锁机制, `UPDATE ... SET col = col + 1` 是原子操作, 保证并发安全。

Q5: 收藏/点赞后续怎么优化?

A: 当前是直接写 DB, 高并发下 DB 压力大。后续计划:

1. 学 Strategy 模块的 `incrementCount` 方案, 用 **Redis Hash + HINCRBY** 做原子计数。
2. 定时任务 `statisHashMapPersist` 批量刷回 DB。
3. 这样每次点赞只操作 Redis, DB 只做定期持久化, 性能提升 10 倍以上。

Q6: NoteDetailVO 为什么继承 Note, 而不是组合一个 Note 字段?

A:

- **继承**: `BeanUtils.copyProperties(note, vo)` 自动拷贝所有同名字段, 一行代码搞定 20+ 字段的赋值, 不用手写 `vo.setXxx(note.getXxx())`。
- **组合**: 需要 `vo.setNote(note)`, 前端访问时要 `vo.note.title` 多一层嵌套, 不如 `vo.title` 直观。
- 继承的缺点是 VO 和实体耦合, 如果 Note 加了不需要暴露的字段, VO 也会带上。但本项目是内部系统, 这个代价可以接受。

Q7: 为什么 Note 里 authorNickname 和 authorHeadImgUrl 标了 @TableField(exist = false)?

A: 这两个字段是冗余字段, `ta_note` 表里暂时还没加对应的列。标注 `exist = false` 告诉 MP 这两个字段不对应数据库列, SELECT 时不映射, INSERT/UPDATE 时忽略。后续通过 ALTER TABLE 加列后, 去掉这个注解即可。

六、Redis 计数 + 异步持久化

★ 所属简历模块: ④ Redis 高并发互动数据

项目背景: `travel-modules-strategy` 模块中, 攻略的阅读量、评论数、点赞数、收藏数、分享数属于**高频写、弱一致性**场景。如果每次点赞都直接 `UPDATE DB`, 高并发下数据库会成为瓶颈。本项目采用 **Redis Hash + HINCRBY 原子计数 + 定时任务批量刷回 DB** 的方案, 将 DB 写入压力降低 N 倍 (N=持久化周期内的请求次数)。

6.1 整体架构流程



6.2 通用增量引擎:incrementCount

这是整个 Redis 计数的核心方法,设计为**通用方法**,支持任意字段、任意增量(+/-),只需一行就能扩展新计数字段。

```

/**
 * @param sid 攻略ID
 * @param field 统计字段名 "viewnum"/"thumbsupnum"/"replynum"/"sharenun"/"favornum"
 * @param delta 增量(正数加,负数减),如点赞=+1,取消点赞=-1
 */
private Map<String, Object> incrementCount(Long sid, String field, int delta) {
    // 1. 拼 Redis Key
    String key = RedisKeys.STRATEGY_STATIS_HASH.join(sid.toString());

    // 2. 缓存不存在或不完整 → 从 DB 预热
    Map<String, Object> map = redisService.getCacheMap(key);
    if (!isMapComplete(map)) {
        map = loadAndCacheFromDB(sid, key);
    }

    // 3. HINCRBY 原子增加(并发安全)
    redisService.incrementCacheMapValue(key, field, delta);

    // 4. 返回最新完整数据
    return redisService.getCacheMap(key);
}
  
```

上层调用:

```

// 阅读量+1 (简单计数, 直接复用 incrementCount)
public Map<String, Object> viewnumIncr(Long sid) {
    return incrementCount(sid, "viewnum", 1);
}

// 评论数+1 (简单计数, 直接复用 incrementCount)
public Map<String, Object> replynumIncr(Long sid) {
    return incrementCount(sid, "replynum", 1);
}

// 点赞: 有每日限 5 次的业务规则, 所以独立实现, 不走 incrementCount
public Map<String, Object> thumbsup(Long sid) {
    // 1. 拼限流 key, 首次设置过期时间为今天剩余时间
    // 2. INCR +1, 判断返回值 > 5 则拒绝
    // 3. 未超限 → HINCRBY thumbsupnum +1
    // ... 详见 6.6 节
}

// 收藏: 是 toggle 切换 (已收藏→取消, 未收藏→收藏), 所以独立实现
public Map<String, Object> favor(Long sid) {
    // 1. SISEMEMBER 判断是否已收藏
    // 2. 已收藏 → HINCRBY favornum -1 + SREM
    // 3. 未收藏 → HINCRBY favornum +1 + SADD
    // ... 详见 6.7 节
}

```

面试题:开闭原则的体现 —— 简单计数 (阅读量/评论数) 复用 `incrementCount` 核心逻辑,只需一行调用;而点赞 (每日限次) 和收藏 (toggle 切换) 有独立业务规则,需单独实现。参数化替代硬编码,避免简单计数的代码重复。

6.3 缓存预热:loadAndCacheFromDB

当 Redis 中不存在该攻略的 Hash 时,从 DB 加载完整数据一次性写入:

```

private Map<String, Object> loadAndCacheFromDB(Long sid, String key) {
    Strategy strategy = baseMapper.selectById(sid);
    Map<String, Object> map = new HashMap<>();
    map.put("id", strategy.getId());
    map.put("viewnum", strategy.getViewnum().intValue());
    map.put("thumbsupnum", strategy.getThumbsupnum().intValue());
    map.put("replynum", strategy.getReplynum().intValue());
    map.put("sharenum", strategy.getSharenum().intValue());
    map.put("favornum", strategy.getFavornum().intValue());
    // ★ 一次性写入所有 6 个字段,而不是只写当前用到的 viewnum
    redisService.setCacheMap(key, map);
    return map;
}

```

为什么一次性写所有字段,而不是只写当前用到的?

`isMapComplete` 校验的是全部 6 个字段,如果只写 `viewnum`,下次有人点赞时会发现缓存不完整,又触发一次 DB 查询。一次性预热全部字段,后续所有操作都直接走 Redis,避免多次重复加载。

6.4 定时持久化:statisHashMapPersist

把 Redis 中的计数批量刷回 DB,保证数据最终落盘:

```
public void statisHashMapPersist() {
    // 1. 匹配所有攻略的 Key
    String key = RedisKeys.STRATEGY_STATIS_HASH.join("*");
    Collection<String> keys = redisService.keys(key);

    for (String k : keys) {
        Map<String, Object> cacheMap = redisService.getCacheMap(k);
        Long id = (Long) cacheMap.get("id");
        Integer viewnum = (Integer) cacheMap.get("viewnum");
        Integer thumbsupnum = (Integer) cacheMap.get("thumbsupnum");
        Integer replynum = (Integer) cacheMap.get("replynum");
        Integer sharenun = (Integer) cacheMap.get("sharenun");
        Integer favornum = (Integer) cacheMap.get("favornum");

        // ★ 5 个字段一次 UPDATE, 而不是每个字段一条 SQL
        lambdaUpdate()
            .eq(Strategy::getId, id)
            .set(Strategy::getViewnum, viewnum)
            .set(Strategy::getThumbsupnum, thumbsupnum)
            .set(Strategy::getReplynum, replynum)
            .set(Strategy::getSharenun, sharenun)
            .set(Strategy::getFavornum, favornum)
            .update();
    }
}
```

设计要点:5 个字段合成一条 UPDATE SQL,而不是每个字段一条。假设 1000 条攻略,1 次持久化只执行 1000 条 SQL,而不是 5000 条。

6.5 Redis 存储结构

Key: strategy_status_hash:1001 [前缀] + [冒号] + [攻略ID]

Type: Hash

TTL: -1 [永不过期] [定时任务管理]

Fields:

id

viewnum

thumbsupnum

replynum

sharenun

favornum

1001

1523

89

45

12

67

攻略ID

阅读量

点赞数

评论数

分享数

收藏数

底层 Redis 命令映射:

业务操作	Redis 命令	Java 代码
缓存预热	HSET key id 1001 viewnum 0 ...	redisTemplate.opsForHash().putAll(key, map)
阅读量+1	HINCRBY key viewnum 1	redisTemplate.opsForHash().increment(key, "viewnum", 1)
获取全部	HGETALL key	redisTemplate.opsForHash().entries(key)
批量查找	KEYS strategy_status_hash:*	redisService.keys(pattern)

6.6 为什么选 Hash 而不是 String(JSON)?

对比维度	String(JSON)方案	Hash 方案(本项目)
单字段修改	GET → 反序列化 → 改字段 → 序列化 → SET(多步,非原子)	直接 <code>HINCRBY</code> (一步,原子)
并发安全	✗ 竞态风险(两个请求同时读到相同值)	✓ Redis 单线程, <code>HINCRBY</code> 天然原子
网络开销	每次传整个 JSON(5 个字段都传)	只传改变的字段(1 个 field + delta)
内存占用	每个攻略一个独立 String,无法共享编码	Hash 内部 ziplist/listpack 紧凑编码,内存更省
单字段读取	反序列化整个 JSON 才能拿到 viewnum	<code>HGET key viewnum</code> 一步到位
扩展性	加字段要改 JSON 结构,可能兼容问题	加字段直接 <code>HSET key newfield value</code> ,零侵入

6.7 面试题模板(结合项目)

面试官问:点赞/收藏这种高频计数你是怎么实现的?

答:我项目里攻略模块的 5 种计数(阅读量、点赞、评论、分享、收藏)用的是 **Redis Hash + HINCRBY 原子操作 + 定时任务批量刷回 DB** 的方案。

写路径上,用户每次点赞时,先查 Redis 中是否存在该攻略的 Hash 缓存,如果不存在就从 DB 加载完整数据做一次**缓存预热**(一次性写入 5 个字段,避免后续操作重复触发 DB)。然后用 `HINCRBY` 命令对对应字段做原子加一,这一步是 Redis 单线程保证并发安全的,不会有竞态问题。

通用方法设计上,我把计数逻辑抽成了一个 `incrementCount(sid, field, delta)` 方法,field 可以是 5 个字段中的任意一个,delta 支持正负(点赞+1,取消收藏-1)。这样新增一个计数字段只需在上层调用一行,不用写重复代码,符合开闭原则。

持久化上,我用了定时任务 `statisHashMapPersist`,每 X 分钟把 Redis 里所有攻略的计数**批量 UPDATE**回 MySQL。注意是 5 个字段合成一条 SQL,而不是每个字段一条。这样做把 N 次 DB 写入合并成了 1 次,DB 压力降低 N 倍。

一致性上,我选择的是**最终一致性**而非强一致性。对于点赞和阅读量,丢失几秒的数据可以接受,这个 trade-off 换来了极高的写入性能。如果业务需要强一致性(比如钱包余额),就不能用这个方案,得走实时双写加分布式事务。

6.8 面试可能追问的问题

Q1:为什么用 HINCRBY 而不是 GET → +1 → SET?

- A:
- `HINCRBY` 是 Redis 内置的**原子操作**,Redis 单线程模型保证不会有并发问题。
 - `GET → +1 → SET` 是三步操作,非原子:两个请求同时读到 100,都设置为 101,就会丢失一次计数。
 - 类比 Note 模块的 `setSql("replynum = replynum + 1")`,原理完全一样,MySQL 靠行锁保证原子性,Redis 靠单线程。

Q2:缓存预热时为什么一次性写所有 5 个字段,而不是只写当前用到的?

- A:
- 因为 `isMapComplete` 校验的是全部字段是否都存在。如果只写 `viewnum`,下次有人点赞时会发现缓存不完整,又触发一次 DB 查询。
 - 一次性预热所有字段,后续所有操作都直接走 Redis,避免多次重复加载。
 - 这也是**写放大换读效率**的思想:第一次多写几个字段,后面所有读都省了。

Q3:Redis 和 DB 数据不一致怎么办?

- A:
- 本项目采用**最终一致性**方案:Redis 是实时数据,DB 是定时持久化的历史数据。两者允许短暂不一致。
 - **读操作优先走 Redis**,Redis 挂了才回源 DB(代码里 `loadAndCacheFromDB` 就是回源逻辑)。
 - **宕机恢复**:如果 Redis 重启清空了数据,下一次请求会自动从 DB 预热回来,只是那一次会走 DB。
 - **Trade-off**:牺牲了强一致性,换来了高并发下的写入性能。对于点赞/阅读量,丢失几秒的数据可以接受。

Q4:KEYS 命令在生产环境有性能风险,怎么优化?

A:

- **KEYS *** 会阻塞 Redis(单线程),如果有 100 万条 key,可能卡顿几百毫秒,期间 Redis 无法响应其他请求。
- **生产环境应改用 SCAN 命令**:游标遍历,分批次读取,不会阻塞 Redis。
- **SCAN 0 MATCH strategy_status_hash:* COUNT 1000**,循环直到游标返回 0。
- 当前项目数据量小, **KEYS** 可以接受,但面试时要主动提这个优化方向,体现生产意识。

Q5:为什么不用实时双写(Redis 和 DB 同时写)?

A:

- **性能**:实时双写每次请求要写两个存储,不仅慢(磁盘 I/O + 网络),还会引入分布式事务问题(一个成功一个失败怎么办?)。
- **定时持久化的好处**:把 N 次 DB 写合并成 1 次批量更新,DB 压力降低 N 倍。
- **什么时候用实时双写**:如果业务要求强一致性(如订单状态、钱包余额),不能丢数据,那就得实时双写+分布式事务。对于点赞/阅读量,用最终一致性更合适。

Q6:如果 Redis 挂了,计数功能会怎么样?

A:

- 本项目的 **loadAndCacheFromDB** 里有 DB 回源机制:如果 Redis 里取不到,就从 DB 加载。
- 但是**写操作目前还是只写 Redis**,如果 Redis 挂了, **incrementCacheMapValue** 会抛异常,计数功能就不可用了。
- **优化方案**(面试可以主动提):
 1. 加熔断降级:Redis 不可用时,降级成直接 **UPDATE DB SET x = x + 1** 的 SQL 原子操作。
 2. Redis 恢复后,再把这段时间 DB 的计数同步回 Redis(或直接以 DB 为准,下次请求时自动预热)。
- 这就是 Note 模块当前用的方案,作为兜底完全可行。

Q7:这个方案和 Note 模块的直接 DB UPDATE 有什么关系?为什么两个模块方案不一样?

A:

- **Strategy 模块**:业务上计数频率高(攻略详情页每次打开都加 viewnum),且攻略数量相对少,适合 Redis Hash 方案。
- **Note 模块**:当前是 MVP 快速上线阶段,直接用 **setSql("replynum = replynum + 1")** 原子 UPDATE,代码简单,快速验证业务。
- Note 模块的点赞/收藏代码里**明确标注了"后续迁移到 Redis 异步刷 DB"**,这就是演进路线。
- **核心思想**:业务不同阶段选不同方案。早期简单快速上线,后期用户量上来再迁移到 Redis 方案。不要过早优化,也不能不考虑优化空间。

Q8:如果要记录"哪个用户给哪篇攻略点过赞"(防止重复点赞),怎么扩展?

A:

- 当前方案只记录"每篇攻略被点赞多少次",不管是谁点的,所以用户可以无限点赞。
- 要做防重复点赞,需要存**用户-攻略的关联关系**,常见方案:
 1. **Redis Set**:Key = **strategy_thumbsup:1001**,Value = Set of userId。点赞时 **SADD**,判断 **SISMEMBER** 是否已存在。缺点:内存占用大,数据无法持久化。
 2. **Redis Bitmap**:Key = **strategy_thumbsup_bitmap:1001**,offset = userId。 **SETBIT** 设置, **GETBIT** 判断。优点:极度节省内存(1 亿用户只要 12.5MB)。缺点:userId 必须是连续整数,稀疏 ID 浪费空间。
 3. **DB 持久化表**:新建 **ta_strategy_thumbsup(strategy_id, user_id, create_time)** 表,联合唯一索引防重复。
- 通常是**Redis + DB 组合**:Redis 做实时防重判断(Set/Bitmap),异步写 DB 做持久化,DB 冷数据定时删除。

6.9 收藏功能: Redis Set 自动去重

项目位置: **StrategyServiceImpl.favor()**。用户点击收藏按钮时,需要保证同一用户对同一篇攻略**只能收藏一次**,再次点击就是取消收藏。

6.9.1 为什么用 Set 而不是 List / Hash?

数据结构	能否去重	判断是否存在的复杂度	适合收藏吗
List	✗ 不能，元素可重复	O(n)，要遍历	✗ 不合适
Hash	✓ 可以，field 唯一	O(1)	⚠ 可以但浪费（value 位没用上）
Set	✓ 可以，元素唯一	O(1)	✓ 最合适

Set 的 `SADD` 天然去重，`SISMEMBER` 一步判断是否已收藏，是这个场景的最佳选择。

6.9.2 项目实现

Key 设计: `strategy_favor_set:{用户id}` → Value 是该用户收藏的所有攻略 id 集合。

```
public Map<String, Object> favor(Long sid) {
    Long userId = SecurityUtils.getUserId();
    String key = RedisKeys.STRATEGY_FAVOR_SET.join(userId.toString());
    String statishashKey = RedisKeys.STRATEGY_STATIS_HASH.join(sid.toString());

    boolean exists = redisService.isCacheSetContains(key, sid);
    if (exists) {
        // 已收藏 → 取消收藏
        redisService.removeCacheSet(key, sid);
        redisService.incrementCacheMapValue(statishashKey, "favornum", -1);
    } else {
        // 未收藏 → 收藏
        redisService.setCacheSet(key, sid);
        redisService.incrementCacheMapValue(statishashKey, "favornum", 1);
    }
    // 返回最新统计数据 + 收藏状态
    Map<String, Object> cacheMap = redisService.getCacheMap(statishashKey);
    cacheMap.put("favor", !exists);
    return cacheMap;
}
```

核心逻辑: 先 `SISMEMBER` 判断是否已收藏 → 已收藏就 `SREM` 取消 + 计数 -1 → 未收藏就 `SADD` 添加 + 计数 +1。一次点击，切换状态。

面试点: 收藏用 Set 的两个理由: ① `SADD` 天然去重，不用担心重复收藏; ② `SISMEMBER` O(1) 判断状态，性能极高。Key 以用户 id 为维度（不是攻略 id），因为“查我收藏了哪些攻略”比“查这篇攻略被谁收藏了”更常见。

6.10 点赞限次: String + INCR 计数器 + 当日过期

项目位置: `StrategyServiceImpl.thumbsup()`。业务规则: 每个用户每天最多给同一篇攻略点赞 5 次，到次数后拒绝，第二天自动重置。

6.10.1 为什么用 String 而不是 Hash / Set?

这个场景的核心需求是: 记录“某用户对某攻略”当天点了几次，超过阈值就拒绝。这是典型的“计数器 + 过期时间”场景。

- Hash 适合“多个字段聚合在一起”（比如攻略的 5 个统计字段），这里只有一个数字，用 Hash 有点重。
- Set 适合去重，不适合计数。
- String + INCR 最简单: value 就是数字，`INCR` 原子自增，`EXPIRE` 设置过期，完美匹配。

6.10.2 项目实现

Key 设计: `user_strategy_thumbsup:{攻略id}:{用户id}` → Value 是今天已点赞次数。

```
public Map<String, Object> thumbsup(Long sid) {
    Long userId = SecurityUtils.getUserId();
    String key = RedisKeys.USER_STRATEGY_THUMBSUP.join(sid.toString(), userId.toString());

    // 第一次点赞 → 初始化计数器, 过期时间= 今天剩余秒数
    if (!redisService.hasKey(key)) {
        Date now = new Date();
        Date endTime = DateUtil.getEndDate(now); // 当天 23:59:59
        long expireTime = DateUtil.getBetweenDate(now, endTime);
        expireTime = expireTime == 0 ? 1 : expireTime;
        redisService.setCacheObject(key, 0, expireTime, TimeUnit.SECONDS);
    }

    // 原子自增 +1
    Long ret = redisService.incrementCacheObjectValue(key, 1);
    String statiskKey = RedisKeys.STRATEGY_STATIS_HASH.join(sid.toString());

    if (ret > 5) {
        // 超过 5 次 → 点赞失败
        result = false;
    } else {
        // 点赞成功 → 攻略点赞总数 +1
        result = true;
        redisService.incrementCacheMapValue(statiskKey, "thumbsupnum", 1);
    }

    Map<String, Object> cacheMap = redisService.getCacheMap(statiskKey);
    cacheMap.put("result", result);
    return cacheMap;
}
```

- 关键点：
- 1. 过期时间精确到当天结束，不是固定 24 小时。固定 24 小时会跨年（比如晚上 10 点点的，到第二天晚上 10 点才过期），业务上"每天 5 次"应该是自然日清零。
 - 2. INCR 是原子操作，Redis 单线程保证并发安全，不用担心两个人同时点赞数错。
 - 3. 点赞成功后，再去 Hash 统计里 HINCRBY 把总数 +1。两套 Redis 结构各司其职：String 管"用户单日限次"，Hash 管"攻略总计数"。

面试点：被问"点赞限次怎么做的？"答三点：① String + INCR 原子计数器；② Key 按「攻略:用户」组合，过期时间设为当天剩余秒数（自然日清零）；③ 超过阈值直接拒绝，成功了再去 Hash 里加总数。注意说清楚"过期时间是到当天结束，不是固定 24h"，这个细节能加分。

6.11 一张表搞清楚：三个功能各用什么 Redis 结构

功能	Redis 结构	Key 示例	核心命令	作用
攻略统计（阅读/点赞/评论/收藏/分享）	Hash	strategy_statisk_hash:{sid}	HINCRBY / HGETALL	聚合一篇攻略的所有计数字段
用户收藏	Set	strategy_favor_set:{uid}	SADD / SREM / SISMEMBER	自动去重，判断是否已收藏
点赞单日限次	String	user_strategy_thumbsup:{sid}:{uid}	INCR / EXPIRE	计数器 + 当日过期，限制每天 5 次

持久化统一出口：Hash 里的统计数据通过 Quartz 定时任务（StrategyTask.statiskHashMapPersist）批量回写 MySQL。Set 和 String 属于用户行为数据，不回写 DB（业务上不需要持久化限次记录）。

七、游记模块 Redis 计数升级 (Note → Redis)

✦ 所属简历模块：④ Redis 高并发互动数据（进阶版）

7.1 升级背景

游记模块 (travel-modules-note) 原先的观看/点赞/评论/收藏功能全部直接走数据库 SQL +1:

```
// 旧代码: 直接 DB +1, 每次操作都打数据库
noteService.lambdaUpdate()
    .eq(Note::getId, id)
    .setSql("thumbsupnum = COALESCE(thumbsupnum, 0) + 1")
    .update();
```

问题: 高并发场景下 (如热门游记被频繁浏览/点赞), 每次操作都打 DB, 数据库压力大; 且没有收藏关系记录, 无法判断用户是否已收藏; 点赞无限制, 可无限刷。

解决方案: 参照同项目 strategy 模块的成熟模式, 引入 **Redis Hash + HINCRBY 原子计数 + 定时任务异步刷库**。

7.2 Redis 数据结构设计

Redis Key 枚举	前缀	过期时间	数据结构	用途
NOTE_STATUS_HASH	note_status_hash	-1 (永不过期)	Hash	每篇游记的统计计数 (viewnum/thumbsupnum/replynum/sharenum/favornum)
NOTE_FAVOR_SET	note_favor_set	-1 (永不过期)	Set	记录每个用户收藏了哪些游记 (userId → Set of notelId)
USER_NOTE_THUMBSUP	user_note_thumbsup	86400 (24 小时)	String	每个用户对每篇游记的每日点赞次数 (nid:uid → count)

Key 拼接示例:

```
note_status_hash:101      {id:101, viewnum:500, thumbsupnum:30, replynum:15, sharenum:5, favornum:20}
note_favor_set:1001       {101, 105, 200}  (用户1001收藏了游记101, 105, 200)
user_note_thumbsup:101:1001 3  (用户1001今天对游记101点赞了3次)
```

7.3 核心流程

流程一：浏览/评论计数 +1（通用 incrementCount 方法）

```

前端请求 → Controller → Service.incrementCount(nid, field, delta)

├── 1. 从 Redis Hash 获取当前游记的统计 Map
├── 2. 检查 Map 是否完整（包含 id + 5 个计数字段）
│   ├── 完整 → 直接 HINCRBY 原子 +1
│   └── 不完整/不存在 → 从 DB 加载完整数据写入缓存，再 HINCRBY
└── 3. 返回更新后的完整统计 Map
  
```

核心代码（NoteServiceImpl）：

```

private Map<String, Object> incrementCount(Long nid, String field, int delta) {
    String key = RedisKeys.NOTE_STATIS_HASH.join(nid.toString());

    // 1. 检查缓存是否完整，不完整则从 DB 加载（懒加载模式）
    Map<String, Object> map = redisService.getCacheMap(key);
    if (!isMapComplete(map)) {
        map = loadAndCacheFromDB(nid, key);
    }

    // 2. 原子增加（底层 HINCRBY，线程安全）
    redisService.incrementCacheMapValue(key, field, delta);

    // 3. 返回最新数据
    return redisService.getCacheMap(key);
}
  
```

流程二：收藏/取消收藏（独立接口）

```

前端 POST /notes/collect/{id} → Service.favor(nid) // 收藏
前端 POST /notes/uncollect/{id} → Service.unFavor(nid) // 取消收藏

├── favor(nid):
│   ├── 1. 从 Redis Set 检查用户是否已收藏（SISMEMBER）
│   ├── 2. 未收藏 → HINCRBY favornum +1 + SADD 记录收藏关系
│   └── 3. 已收藏 → 幂等跳过（不做任何操作）
├── unFavor(nid):
│   ├── 1. 从 Redis Set 检查用户是否已收藏（SISMEMBER）
│   ├── 2. 已收藏 → HINCRBY favornum -1 + SREM 移除收藏关系
│   └── 3. 未收藏 → 幂等跳过（不做任何操作）
└── 4. 持久化到 DB（调用 statisHashMapPersist）
  
```

设计差异：与 Strategy 模块的 toggle 设计不同，Note 模块采用 favor/unFavor 两个独立接口。favor 只做收藏（已收藏则幂等跳过），unFavor 只做取消（未收藏则幂等跳过），语义更清晰。

流程三：点赞（每日限 5 次）

```

前端 POST /notes/star/{id} → Service.thumbsup(nid)

├── 1. 拼接限流 key: user_note_thumbsup:{nid}:{uid}
├── 2. key 不存在 → 设置初始值 0，过期时间为今天剩余时间
├── 3. INCR 原子 +1，判断返回值
│   ├── > 5 → 点赞失败（今日已达上限）
│   └── ≤ 5 → HINCRBY thumbsupnum +1
└── 4. 返回 {result: true/false, ...统计Map}
  
```

流程四：定时持久化 (Redis → DB)

```
Quartz 定时任务 → NoteTask.statisHashMapPersist()  
├─ Feign 调用 → NoteController /feign/statisHashMapPersist  
├─ 1. KEYS note_statis_hash:* 获取所有游记的统计 key  
├─ 2. 遍历每个 key, HGETALL 获取完整统计 Map  
└─ 3. 批量 UPDATE ta_note SET viewnum=?, thumbsupnum=?, ...
```

流程五：应用启动预热

```
ApplicationReadyEvent → NoteStatisHashInitListener  
├─ 1. 查询 DB 中所有游记  
├─ 2. 遍历每篇游记, 拼接 key  
├─ 3. 判断 key 是否已存在 (避免覆盖已有缓存)  
└─ 4. 不存在 → 将统计数据写入 Redis Hash
```

7.4 改动文件清单

文件	改动类型	说明
RedisKeys.java	修改	新增 NOTE_STATIS_HASH 、 NOTE_FAVOR_SET 、 USER_NOTE_THUMBSUP 三个枚举值
pom.xml (note)	修改	新增 travel-common-redis 依赖
DateUtil.java	新增	点赞限流用的日期工具类（计算今天剩余秒数）
INoteService.java	修改	新增 7 个接口方法 (viewnumIncr/replynumIncr/favor/unFavor/isUserFavor/thumbsup/statisHashMapPersist)
NoteServiceImpl.java	修改	注入 RedisService，实现全部 Redis 计数逻辑 + detail() 中浏览量自增 + 收藏状态查询
NoteController.java	修改	star/collect 改为走 Redis；新增 isUserFavor 接口；新增 feign/statisHashMapPersist 接口
NoteCommentServiceImpl.java	修改	评论时 replynum +1 改为走 Redis (noteService.replynumIncr)
NoteStatisHashInitListener.java	新增	应用启动时预加载 DB 统计数据到 Redis
RemoteNoteService.java	修改	新增 statisHashMapPersist Feign 接口
RemoteNoteFallbackFactory.java	修改	新增 statisHashMapPersist 降级实现
NoteTask.java	新增	job 模块的定时任务类，Feign 调用 note 服务刷库
pom.xml (job)	修改	新增 travel-api-note 依赖

7.5 与 Strategy 模块的对比优化

对比项	Strategy 模块	Note 模块（升级后）	优化说明
实体字段类型	Long	Integer	loadAndCacheFromDB 中做了 null 安全处理 (!= null ? value : 0)
detail 接口	浏览量需前端单独调接口	detail 时自动 viewnumIncr	减少前端请求次数，打开详情即自动统计浏览量
收藏状态查询	detail 中不查	detail 中自动查 Redis Set 返回 favorFlag	用户打开详情页即可看到收藏按钮状态
收藏接口	toggle 设计（一个 favor 接口切换）	独立的 favor/unFavor 两个接口	Note 模块语义更清晰，各自幂等
缓存预热	StatisHashInitListener	NoteStatisHashInitListener	完全一致的懒加载 + 启动预热模式

7.6 面试问答模板

Q: 游记模块的浏览/点赞/评论/收藏是怎么实现的？

A: 我们采用了 **Redis Hash + HINCRBY 原子计数 + 定时任务异步刷库** 的方案。每篇游记在 Redis 中对应一个 Hash key (`note_statis_hash:{id}`)，包含 viewnum、thumbsupnum、replynum、sharenum、favornum 五个字段。用户操作时通过 HINCRBY 原子增减，不直接打数据库。定时任务 (Quartz + Feign) 每隔一段时间扫描所有 key，批量 UPDATE 回数据库。

Q: 为什么要用 Redis 而不是直接操作数据库？

A: 三个原因：
1. **性能**：浏览/点赞是高频操作，直接打 DB 会造成行锁竞争和 IO 瓶颈。Redis 全内存操作，HINCRBY 是 O(1) 的原子命令。
2. **原子性**：HINCRBY 本身是原子的，不需要额外加锁，并发安全。
3. **削峰**：高并发时 Redis 做缓冲，定时任务批量刷库，DB 压力从 N 次写降为 1 次写。

Q: 缓存数据怎么初始化的？如果 Redis 挂了怎么办？

A: 两层保障：
1. **启动预热**：应用启动时 (ApplicationReadyEvent)，NoteStatisHashInitListener 把 DB 中所有游记的统计数据预加载到 Redis，避免冷启动。
2. **懒加载兜底**：incrementCount 方法中检查 Map 是否完整，如果不完整 (Redis 重启后丢失)，自动从 DB 加载并写入缓存。即使 Redis 挂了恢复后，下次访问也能自动重建缓存。

Q: 点赞怎么防刷？

A: 用 Redis 的 String 类型做每日限流。Key = `user_note_thumbsup:{游记id}:{用户id}`，过期时间设为今天剩余时间 (到 23:59:59)。每次点赞 INCR +1，超过 5 次就拒绝。第二天 key 自动过期，重新计数。

Q: 收藏功能怎么实现的？

A: 用 Redis Set 记录用户的收藏关系。Key = `note_favor_set:{用户id}`，Value = Set of 游记id。收藏时 SADD + HINCRBY favornum +1，取消收藏时 SREM + HINCRBY favornum -1。查详情时用 SISMEMBER 判断是否已收藏，设置 favorFlag 返回前端。

Q: 定时任务怎么配置的？

A: 在 job 模块 (travel-modules-job) 中创建 NoteTask 类，通过 Feign 调用 note 服务的 `/feign/statisHashMapPersist` 接口。然后在管理后台的 sys_job 表中配置 Quartz 定时任务，invokeTarget 设为 `noteTask.statisHashMapPersist`，设置 cron 表达式 (如每 10 分钟执行一次)。

A1、JWT + Redis 登录会话管理

✦ 所属简历模块：① 微服务架构与登录认证

A1.1 整体架构

```

用户登录 → Auth 服务 → 生成 JWT + UUID → 存入 Redis (login_tokens:{uuid})
                                     ↓
用户请求 → Gateway → 解析 JWT → 检查 Redis 是否存在 → 放行/拒绝
                                     ↓
Token 即将过期 → 剩余 < 120 分钟 → 自动滑动续期
                                     ↓
账号下线 → 删除 Redis 中的 login_tokens:{uuid} → 后续请求被 Gateway 拦截
  
```

A1.2 Token 生成：UUID + JWT 双层设计

核心文件：TokenService.java 、 JwtUtils.java

```

public Map<String, Object> createToken(LoginUser loginUser) {
    // 1. 生成 UUID 作为 Redis 中的唯一标识
    String token = IdUtils.fastUUID();
    loginUser.setToken(token);

    // 2. 滑动续期: 设置过期时间并存入 Redis
    refreshToken(loginUser);

    // 3. JWT Claims 封装
    Map<String, Object> claimsMap = new HashMap<>();
    claimsMap.put(SecurityConstants.USER_KEY, token);           // UUID
    claimsMap.put(SecurityConstants.DETAILS_USER_ID, userId);
    claimsMap.put(SecurityConstants.DETAILS_USERNAME, userName);

    // 4. JWT 签名: HS512 算法
    Map<String, Object> rspMap = new HashMap<>();
    rspMap.put("access_token", JwtUtils.createToken(claimsMap));
    rspMap.put("expires_in", expireTime);
    return rspMap;
}
  
```

为什么用 UUID + JWT 双层？

- JWT 本身携带 user_id/username 信息，但不存登录态，JWT 只是“通行证”
- UUID 是 Redis Key 的一部分，真正的登录态（过期时间、用户完整信息）存在 Redis 里
- Gateway 解析 JWT 拿到 UUID → 去 Redis 查是否存在 → 存在才放行
- 好处：Token 过期由 Redis TTL 控制，不受 JWT 签名时间限制；强制下线只需删 Redis Key

A1.3 Redis 存储设计

```

// Key 格式: login_tokens:{uuid}
// Value: LoginUser 对象（含用户信息、登录时间、过期时间）
// TTL: 720 分钟（12 小时）

public void refreshToken(LoginUser loginUser) {
    loginUser.setLoginTime(System.currentTimeMillis());
    loginUser.setExpireTime(loginUser.getLoginTime() + expireTime * MILLIS_MINUTE);
    String userKey = getTokenKey(loginUser.getToken()); // login_tokens:{uuid}
    redisService.setCacheObject(userKey, loginUser, expireTime, TimeUnit.MINUTES);
}
  
```

面试怎么说：“Token 存在 Redis 里，Key 是 login_tokens:{uuid}，Value 是 LoginUser 对象，TTL 设为 720 分钟。JWT 只携带 UUID 和基本用户信息，真正的登录态在 Redis。这样做的好处是：可以主动控制 Token 过期（删 Redis Key 即可），不像纯 JWT 方案无法中途失效。”

A1.4 多端登录态隔离

项目支持两个登录入口：

端	入口	用户来源	登录服务
管理后台 (admin)	/auth/login type=admin	SysUser （系统用户表）	SysLoginService
前台用户 (pc)	/auth/login type=pc	UserInfo （会员用户表）	PcLoginService

```
// TokenController 统一入口，按 type 分流
@PostMapping("login")
public R<?> login(@RequestBody LoginBody form) {
    LoginUser userInfo = null;
    if (form.getType().equals(TokenConstants.ADMIN)) {
        userInfo = sysLoginService.login(form.getUsername(), form.getPassword());
    } else {
        userInfo = pcLoginService.login(form.getUsername(), form.getPassword());
    }
    return R.ok(tokenService.createToken(userInfo));
}
```

隔离机制：

- admin 和 pc 走不同的登录 Service，查不同的用户表
- LoginUser 对象中区分 sysUser（后台）和 userInfo（前台）
- 每次登录生成独立 UUID，同一用户多端登录各自持有独立 Token

面试怎么说： "后台管理员和前台用户共用一个登录接口，通过 type 参数分流到不同的 LoginService。管理员查 sys_user 表，前台用户查 user_info 表。每次登录生成独立 UUID Token 存入 Redis，多端各自的 Token 互不影响。 "

A1.5 Token 滑动续期

```
public void verifyToken(LoginUser loginUser) {
    long expireTime = loginUser.getExpireTime();
    long currentTime = System.currentTimeMillis();
    // 剩余时间不足 120 分钟 → 自动续期
    if (expireTime - currentTime <= MILLIS_MINUTE_TEN) { // MILLIS_MINUTE_TEN = 120 分钟
        refreshToken(loginUser); // 重新设置 720 分钟 TTL
    }
}
```

机制说明：

- Token 初始有效期 720 分钟（12 小时）
- 用户每次请求时，Gateway 解析 Token 后检查 Redis 中 LoginUser 的过期时间
- 如果剩余时间不足 120 分钟（2 小时），自动调用 refreshToken 重新设置 720 分钟 TTL
- 效果：活跃用户永不过期，非活跃用户 12 小时后自动登出

面试怎么说： "我们用了滑动续期机制。Token 初始 720 分钟，每次请求时检查剩余时间，不足 120 分钟就自动续回 720 分钟。活跃用户不会感知到 Token 过期，非活跃用户 12 小时后自然失效。这比固定过期时间体验好很多。 "

A1.6 账号强制下线

```
// AuthLogic.java
public void logoutByToken(String token) {
    tokenService.delLoginUser(token);
}

// TokenService.java
public void delLoginUser(String token) {
    if (StringUtils.isEmpty(token)) {
        String userkey = JwtUtils.getUserKey(token); // 从 JWT 中提取 UUID
        redisService.deleteObject(getTokenKey(userkey)); // 删除 login_tokens:{uuid}
    }
}
```

原理：删除 Redis 中的登录态 Key，后续请求到 Gateway 时，JWT 能解析（签名没过期），但 Redis 中已无此 Key → 判定为“登录状态已过期” → 拒绝访问。

面试怎么说：“强制下线的实现很简单——删 Redis Key。用户持有的 JWT 仍然能被解析（签名没变），但 Gateway 查 Redis 发现 Key 不存在了，直接返回‘登录状态已过期’。这也是我们不用纯 JWT 而要加 Redis 的核心原因——纯 JWT 无法中途失效。”

A1.7 网关鉴权流程 (AuthFilter)

```
// AuthFilter.java - 网关过滤器
private Mono<Void> checkToken(String token, ServerWebExchange exchange, ...) {
    // 1. 解析 JWT，验证签名
    Claims claims = JwtUtils.parseToken(token);
    if (claims == null) return unauthorizedResponse(exchange, "令牌已过期或验证不正确！");

    // 2. 从 JWT 中提取 UUID
    String userkey = JwtUtils.getUserKey(claims);

    // 3. 检查 Redis 中是否存在登录态
    boolean islogin = redisService.hasKey(getTokenKey(userkey));
    if (!islogin) return unauthorizedResponse(exchange, "登录状态已过期");

    // 4. 提取用户信息，放入请求 Header 传递给下游服务
    String userid = JwtUtils.getUserId(claims);
    String username = JwtUtils.getUserName(claims);
    addHeader(mutate, SecurityConstants.USER_KEY, userkey);
    addHeader(mutate, SecurityConstants.DETAILS_USER_ID, userid);
    addHeader(mutate, SecurityConstants.DETAILS_USERNAME, username);

    return chain.filter(exchange.mutate().request(mutate.build()).build());
}
```

鉴权三步：解析 JWT（验签）→ 查 Redis（验活）→ 注入 Header（传递用户信息）

白名单： /auth/login、/auth/register、/auth/logout、/* /v2/api-docs、/csrf 路径直接放行，不走鉴权。

A1.8 面试题模板

"登录认证这块我们用的是 **JWT + Redis** 方案。用户登录后，Auth 服务生成一个 UUID 作为 Redis Key，把登录态（LoginUser 对象）存入 Redis，TTL 设为 720 分钟。同时用 JWT 把 UUID 和用户信息签名后返回给前端。

前端每次请求携带 JWT，Gateway 的 AuthFilter 解析 JWT 拿到 UUID，去 Redis 查是否存在——存在才放行，不存在返回‘登录状态已过期’。这样做的好处是：**Token 过期由 Redis TTL 控制，可以主动失效**（删 Redis Key 即可强制下线），不像纯 JWT 方案签发后无法中途撤销。

还做了**滑动续期**：每次请求检查 Token 剩余时间，不足 120 分钟就自动续回 720 分钟，活跃用户不会感知到 Token 过期。后台管理员和前台用户共用一个登录接口，通过 type 参数分流，查不同的用户表，实现多端隔离。"

A1.9 面试可能追问的问题

Q1：为什么不用纯 JWT？为什么要加 Redis？

A：纯 JWT 最大问题是**签发后无法中途失效**——Token 直到签名过期前一直有效。加 Redis 后，真正的登录态存在 Redis 里，删 Key 就能强制下线，还可以做滑动续期。

Q2：JWT 密钥怎么管理的？

A：密钥存在 `TokenConstants.SECRET` 常量类中，生产环境应该迁移到环境变量或 Nacos 配置中心。签名算法用的是 HS512（HMAC-SHA512）。

Q3：滑动续期的"120 分钟"是怎么定的？

A：Token 总有效期 720 分钟（12 小时），续期阈值 120 分钟（2 小时）。意思是用户最后一次请求后 10 小时内再请求都能自动续期，超过 12 小时不活跃才真正过期。这个值可以根据业务调，活跃度高的系统可以缩短。

Q4：多端登录怎么隔离的？

A：共用一个 `/auth/login` 接口，通过 `type` 参数区分 admin 和 pc，分别走 `SysLoginService` 和 `PcLoginService`，查不同的用户表。每次登录生成独立 UUID Token，多端各自独立，互不影响。

A2、OpenFeign + Sentinel 跨服务调用降级

所属简历模块：① 微服务架构与登录认证

A2.1 整体架构



A2.2 Feign 客户端定义

项目共定义了 16 个 Feign 远程服务接口，覆盖所有微服务模块：

Feign 接口	目标服务	用途
RemoteUserService	travel-system	后台用户查询、注册
RemoteUserInfoService	travel-member	前台会员查询
RemoteStrategyService	travel-strategy	攻略查询
RemoteNoteService	travel-note	游记查询
RemoteDestinationService	travel-destination	目的地查询
RemoteFileService	travel-file	文件上传
RemoteBannerService	travel-adv	轮播图
...	...	(共 16 个)

接口定义模式：

```

@FeignClient(
    contextId = "remoteStrategyService",    // Bean 名称
    value = ServiceNameConstants.STRATEGY_SERVICE, // Nacos 服务名
    fallbackFactory = RemoteStrategyFallbackFactory.class // 降级工厂
)
public interface RemoteStrategyService {

    @GetMapping("/strategies/inner/{id}")
    R<Strategy> getOne(@PathVariable Long id, @RequestHeader String source);

    @GetMapping("/strategies/inner/list")
    R<List<Strategy>> list(@RequestParam String source);
}

```

A2.3 FallbackFactory 降级工厂

16 个 Feign 接口对应 16 个 FallbackFactory，实现模式完全统一：

```

@Component
public class RemoteStrategyFallbackFactory implements FallbackFactory<RemoteStrategyService> {

    private static final Logger log = LoggerFactory.getLogger(RemoteStrategyFallbackFactory.class);

    @Override
    public RemoteStrategyService create(Throwable cause) {
        log.error("攻略服务调用失败: {}", cause.getMessage());

        // 返回匿名内部类，所有方法返回 R.fail()
        return new RemoteStrategyService() {
            @Override
            public R<Strategy> getOne(Long id, String source) {
                return R.fail("获取攻略详情失败:" + cause.getMessage());
            }

            @Override
            public R<List<Strategy>> list(String source) {
                return R.fail("获取攻略列表失败:" + cause.getMessage());
            }
        };
    }
}

```

降级模式四步走：

1. 实现 `FallbackFactory<XxxService>` 泛型接口
2. `@Component` 注册为 Spring Bean
3. `create(Throwable cause)` 方法中记录错误日志
4. 返回匿名内部类，每个方法返回 `R.fail(...)` 降级结果

为什么用 FallbackFactory 而不是 Fallback?

- `Fallback` 只能无参构造，拿不到异常信息
- `FallbackFactory` 的 `create(Throwable cause)` 能拿到异常原因，可以在降级信息中携带错误详情，方便排查

A2.4 Sentinel Gateway 限流配置

Feign + Sentinel 集成：

```
# application-dev.yml (Nacos 配置中心)
feign:
  sentinel:
    enabled: true          # 开启 Sentinel 对 Feign 的支持
  okhttp:
    enabled: true          # 使用 OkHttp 替代默认 HttpURLConnection
  client:
    config:
      default:
        connectTimeout: 10000 # 连接超时 10s
        readTimeout: 10000   # 读取超时 10s
  compression:
    request:
      enabled: true        # 请求压缩
    response:
      enabled: true        # 响应压缩
```

Gateway 层 Sentinel 限流 (Nacos 配置) :

```
[
  { "resource": "travel-auth", "count": 500 },
  { "resource": "travel-system", "count": 1000 },
  { "resource": "travel-gen", "count": 200 },
  { "resource": "travel-job", "count": 300 }
]
```

- `travel-auth` : 认证服务限流 500 QPS
- `travel-system` : 系统服务限流 1000 QPS
- 限流触发后统一返回: "请求超过最大数, 请稍候再试"

限流异常处理:

```
// GatewayConfig.java
@Bean
@Order(Ordered.HIGHEST_PRECEDENCE)
public SentinelFallbackHandler sentinelGatewayExceptionHandler() {
    return new SentinelFallbackHandler();
}

// SentinelFallbackHandler.java - 统一返回限流提示
public Mono<Void> handle(ServerWebExchange exchange, Throwable ex) {
    if (!BlockException.isBlockException(ex)) {
        return Mono.error(ex);
    }
    return writeResponse(response, exchange); // "请求超过最大数, 请稍候再试"
}
```

A2.5 面试题模板

"跨服务调用这块, 我们用 **OpenFeign + Sentinel** 做了统一的降级方案。项目里有 16 个 Feign 远程服务接口, 每个都配了对应的 `FallbackFactory` 降级工厂。

降级工厂的实现模式是统一的: 实现 `FallbackFactory` 接口, 在 `create(Throwable cause)` 方法里记录错误日志, 然后返回一个匿名内部类, 每个方法都返回 `R.fail(...)` 而不是抛异常。这样调用方拿到的永远是一个 `R` 响应对象, 不会因为远程服务挂了导致整个链路崩溃。

Sentinel 还在 Gateway 层做了限流, 限流规则存在 Nacos 配置中心, 比如认证服务限流 500 QPS、系统服务限流 1000 QPS。触发限流后统一返回'请求超过最大数'的提示。

选 `FallbackFactory` 而不是 `Fallback`, 是因为 `FallbackFactory` 的 `create` 方法能拿到异常原因, 降级信息里可以携带具体错误, 方便排查。"

A2.6 面试可能追问的问题

Q1: Feign 降级 Fallback 和 FallbackFactory 的区别？

A: `Fallback` 需要无参构造，拿不到异常信息；`FallbackFactory` 的 `create(Throwable cause)` 能拿到异常原因，可以在降级返回中携带错误详情。生产中一般用 `FallbackFactory`。

Q2: Sentinel 和 Hystrix 的区别？

A: Sentinel 是阿里开源的，核心优势是**控制台可视化 + 动态规则配置**（规则存 Nacos，修改实时生效）；Hystrix 已经停更。Sentinel 支持限流、熔断、系统自适应等多种规则，比 Hystrix 功能更丰富。

Q3: Feign 超时时间怎么设的？

A: 全局配置 `connectTimeout` 和 `readTimeout` 都是 10 秒。也可以按服务单独配置，在 `feign.client.config.{service-name}` 下覆盖。

Q4: 降级返回 `R.fail()`，调用方怎么处理？

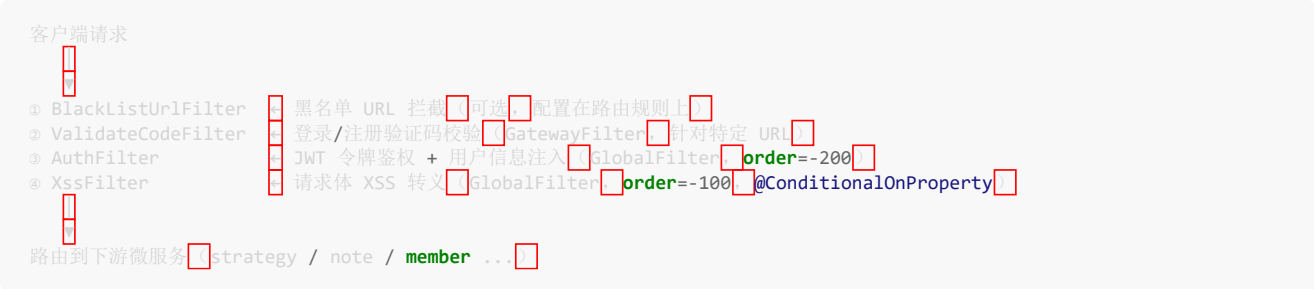
A: 调用方拿到 `R` 对象后用 `R.isError(result)` 或 `result.getCode() != 200` 判断是否失败。如果失败就走自己的兜底逻辑（比如返回空列表、缓存数据等），不会让异常向上传播。

八、Gateway 网关过滤器体系

所属简历模块：① 微服务架构与登录认证

项目背景：`travel-gateway` 是微服务架构的统一入口，所有外部请求都先经过网关。网关承担了“交通警察”的角色，通过层层过滤器（Filter）实现**鉴权、验证码校验、XSS 防护、黑名单拦截**等横切关注点，让各业务微服务专注于自身逻辑。

8.1 整体架构：四层过滤器



过滤器分类与执行顺序：

过滤器	类型	顺序	作用
<code>BlackListUrlFilter</code>	GatewayFilterFactory（路由级）	默认	黑名单 URL 正则匹配拦截
<code>ValidateCodeFilter</code>	GatewayFilterFactory（路由级）	默认	登录/注册请求验证码校验
<code>AuthFilter</code>	GlobalFilter（全局）	-200	JWT 令牌解析、白名单放行、请求头注入用户信息
<code>XssFilter</code>	GlobalFilter（全局，条件装配）	-100	请求体 JSON 中 HTML/JS 脚本转义

面试点：`Ordered` 接口的 `getOrder()` 返回值越小优先级越高。`AuthFilter` 先执行（-200），未登录直接 401 返回，后面的 `XssFilter` 就不会白忙活。

8.2 AuthFilter: 网关鉴权核心

这是整个网关最关键的过滤器，承担三件事：**白名单放行** → **令牌解析** → **注入用户信息到请求头**。

核心代码流程：

```
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    String url = request.getURI().getPath();

    // ① 白名单直接放行（如 /auth/login、/auth/register）
    if (StringUtils.matches(url, ignoreWhite.getWhites())) {
        return chain.filter(exchange);
    }

    // ② 前后台差异化鉴权：
    // 前台非 /admin/** 路径：有 token 就解析并注入，没有也放行（游客可浏览）
    // 后台 /admin/** 路径：必须有 token，否则 401
    if (!request.getURI().getPath().startsWith("/admin/")) {
        String token = getToken(request);
        if (StringUtils.isNotEmpty(token)) {
            return checkToken(token, exchange, mutate, chain); // 解析+注入
        }
        return chain.filter(exchange); // 游客放行
    }

    // ③ 后台强制校验 token
    String token = getToken(request);
    if (StringUtils.isEmpty(token)) {
        return unauthorizedResponse(exchange, "令牌不能为空");
    }
    return checkToken(token, exchange, mutate, chain);
}
```

checkToken 的三步校验：

```
private Mono<Void> checkToken(String token, ...) {
    // 1. JWT 签名解析（解析失败 = 伪造或过期）
    Claims claims = JwtUtils.parseToken(token);
    if (claims == null) { return unauthorizedResponse(...); }

    // 2. Redis 校验是否仍在登录态（防止已注销但 JWT 未过期）
    String userkey = JwtUtils.getUserKey(claims);
    boolean islogin = redisService.hasKey(getTokenKey(userkey));
    if (!islogin) { return unauthorizedResponse("登录状态已过期"); }

    // 3. 从 JWT 中提取 userid/username，注入到下游请求头
    String userid = JwtUtils.getUserId(claims);
    String username = JwtUtils.getUserName(claims);
    addHeader(mutate, SecurityConstants.DETAILS_USER_ID, userid);
    addHeader(mutate, SecurityConstants.DETAILS_USERNAME, username);
    // 清除外部伪造的 FROM_SOURCE 头（安全：防止外部伪装成内部调用）
    removeHeader(mutate, SecurityConstants.FROM_SOURCE);

    return chain.filter(exchange.mutate().request(mutate.build()).build());
}
```

为什么要“从请求头注入用户信息”而不是让各服务自己解析 JWT？

因为：

1. **解耦**：各业务服务不感知 JWT 的存在，只从请求头读 `userId / username`，换认证方案（如 Session、OAuth）时业务零改动。
2. **安全**：JWT 密钥只在网关和 auth 服务保管，下游服务不接触密钥，降低泄露面。
3. **统一**：认证逻辑只有一份，不会出现“这个服务校验了、那个服务没校验”的情况。

8.3 ValidateCodeFilter: 验证码校验

关键点：它是 `AbstractGatewayFilterFactory`（路由级过滤器），不是 `GlobalFilter`，需要在路由配置中显式指定。只拦截 `/auth/login` 和 `/auth/register`。

```

public class ValidateCodeFilter extends AbstractGatewayFilterFactory<Object> {
    private final static String[] VALIDATE_URL = new String[] { "/auth/login", "/auth/register" };

    @Override
    public GatewayFilter apply(Object config) {
        return (exchange, chain) -> {
            ServerHttpRequest request = exchange.getRequest();
            // 非登录/注册 或 验证码关闭 → 跳过
            if (!StringUtils.containsAnyIgnoreCase(url, VALIDATE_URL) || !captchaProperties.getEnabled()) {
                return chain.filter(exchange);
            }
            // 读取请求体 JSON → 取出 code 和 uuid → 校验
            String rspStr = resolveBodyFromRequest(request);
            JSONObject obj = JSON.parseObject(rspStr);
            validateCodeService.checkCaptcha(obj.getString(CODE), obj.getString(UUID));
            return chain.filter(exchange);
        };
    }
}

```

面试追问：Gateway 是 WebFlux（响应式），请求体是 `Flux<DataBuffer>` 只能读一次，为什么这里读了之后下游服务还能读到？
 → 本项目用了 `CacheRequestFilter`（未贴代码）先把请求体缓存起来，这是 Spring Cloud Gateway 的标准套路。

8.4 XssFilter：跨站脚本防护

特点：

- `@ConditionalOnProperty(value = "security.xss.enabled", havingValue = "true")` → 配置驱动，可随时开关
- 只过滤 POST/PUT 等带 JSON body 的请求（GET/DELETE 跳过）
- 可配置 `excludeUrls` 排除某些 URL（如富文本编辑接口，本身就允许 HTML）

核心实现（请求体装饰器模式）：

```

private ServerHttpRequestDecorator requestDecorator(ServerWebExchange exchange) {
    return new ServerHttpRequestDecorator(exchange.getRequest()) {
        @Override
        public Flux<DataBuffer> getBody() {
            Flux<DataBuffer> body = super.getBody();
            return body.buffer().map(dataBuffers -> {
                // 1. 聚合 DataBuffer → byte[] → String
                byte[] content = ...;
                String bodyStr = new String(content, UTF_8);
                // 2. HTML/JS 转义 (<script> → &lt;script>)
                bodyStr = EscapeUtil.clean(bodyStr);
                // 3. 写回新的 DataBuffer
                byte[] bytes = bodyStr.getBytes();
                ...
            });
        }
        @Override
        public HttpHeaders getHeaders() {
            // ★ 注意：修改了 body 长度后必须删除 Content-Length，
            // 改为 Transfer-Encoding: chunked，否则长度对不上会截断
            httpHeaders.remove(HttpHeaders.CONTENT_LENGTH);
            httpHeaders.set(HttpHeaders.TRANSFER_ENCODING, "chunked");
        }
    };
}

```

面试点：Content-Length 与实际 body 不匹配是 Gateway 修改 body 时最常见的坑，本项目用 Transfer-Encoding: chunked（分块传输）解决，体现了对 HTTP 协议的理解。

8.5 面试题模板（结合项目）

面试官问：微服务网关做了哪些事情？

答：网关是整个系统的统一入口，我项目里通过四层过滤器实现了横切关注点的下沉：

第一层 AuthFilter（全局过滤器，order=-200）做统一鉴权。首先和 Nacos 配置白名单比对（如登录、注册、验证码接口直接放行）。然后前后台差异化处理：非 `/admin/**` 的前台接口有 token 就解析注入、没有也放行（游客可浏览）；后台 `/admin/**` 接口强制要求 token，否则返回 401。校验通过后把 `userId`、`username` 编码后注入到下游请求头，同时清除外部伪造的 `FROM_SOURCE` 内部调用标识，各业务服务不接触 JWT，只从请求头读用户信息，解耦且安全。

第二层 ValidateCodeFilter（路由级过滤器）只拦截登录和注册接口，从响应式的 `Flux<DataBuffer>` 中读取请求体 JSON，取出验证码和 uuid，和 Redis 存储的验证码比对，防止暴力破解登录。

第三层 XssFilter（全局过滤器，`@ConditionalOnProperty` 条件装配），用 `ServerHttpRequestDecorator` 装饰请求，把请求体中的 `<script>` 等危险 HTML/JS 字符通过 `EscapeUtil.clean()` 转义，预防跨站脚本攻击。还可配置 `excludeUrls` 排除富文本编辑这类允许 HTML 的接口。

第四层 BlackListUrlFilter（路由级）通过正则匹配拦截黑名单 URL。

8.6 面试可能追问的问题

Q1: Gateway 的 GlobalFilter 和 GatewayFilter 有什么区别？

A:

- **GlobalFilter**：全局过滤器，对所有路由生效，实现类加 `@Component` 就自动生效。本项目的 AuthFilter、XssFilter 都是 GlobalFilter，因为要对几乎所有请求生效。
- **GatewayFilter**：路由级过滤器，需要在路由配置中显式关联（`spring.cloud.gateway.routes.filters`），或通过 `AbstractGatewayFilterFactory` 编程式绑定。本项目的 ValidateCodeFilter、BlackListUrlFilter 属于这一类，因为只对特定 URL 生效。
- 共同点：两者执行时都会经过相同的过滤器链，都通过 `Ordered` 接口排优先级。

Q2: 为什么要把用户信息放到请求头里传下去，而不是用共享上下文？

A:

1. **协议通用**：请求头是 HTTP 标准协议的一部分，不受微服务框架升级影响。
2. **跨线程传递**：WebFlux 响应式编程下 ThreadLocal 会失效，而请求头会随着请求自然传递。
3. **无状态可观测**：链路追踪（SkyWalking、Zipkin）可以直接看到请求头中的 userId，方便排查。
4. **安全分层**：下游业务服务不接触 JWT 密钥，降低密钥泄露面。

Q3: XSS 攻击的原理是什么？怎么防护？

A:

- **原理**：攻击者把恶意 JS/HTML 脚本（如 `<script>stealCookie()</script>`）通过输入框提交到数据库，其他用户访问页面时浏览器把这些字符串当成真正的 HTML 执行，就会窃取 Cookie、跳转到钓鱼网站等。

- **本项目防护**：

1. **网关层转义**：XssFilter 在所有 JSON POST 请求进入业务之前，把 `<`、`>`、`"` 等特殊字符转成 HTML 实体（`<`、`>`、`"`），存到数据库里的就是安全文本。
2. **白名单排除**：对确实需要保存 HTML 的富文本接口（如攻略发布），排除在 XSS 过滤之外，改用前端渲染时的白名单标签净化。
3. **配置可开关**：`@ConditionalOnProperty` 支持线上临时关闭 XSS 过滤，便于排查问题。

Q4: @ConditionalOnProperty 在网关中的应用有哪些好处？

A: XssFilter 加了 `@ConditionalOnProperty(value = "security.xss.enabled", havingValue = "true")`，好处：

1. **热切换**：Nacos 改配置即可开关，不需要重启网关。
2. **灰度验证**：新环境上线先关 XSS 过滤跑通主流程，再逐步打开。
3. **排障方便**：线上怀疑 XSS 转义导致业务异常时，临时关闭快速定位。
4. **环境隔离**：开发/测试环境关闭过滤，便于调试；生产环境打开。

Q5: 前后台为什么要做差异化鉴权? /admin/ 有 token 强制校验, 非 /admin/ 有 token 就解析没有也放行?

- A: 业务场景不同:
- 后台管理端 (/admin/) 是内部人员操作, 所有接口必须登录, 强制 token 校验。
 - 前台用户端 (非 /admin/) 有公开内容 (首页、攻略列表、攻略详情游客都能看), 不需要登录; 但如果用户已登录 (携带 token), 就需要解析出用户信息注入, 以便展示"我已收藏"等个性化状态。
 - 这种设计避免了"公开接口必须传 token"的不合理要求, 同时保持了登录态用户的个性化体验。

九、统一登录入口与多端认证 (Auth 服务)

✦ 所属简历模块: ① 微服务架构与登录认证

项目背景: 项目分后台管理端 (Admin) 和前台用户端 (PC) 两套用户体系, 但不想暴露两个登录接口给前端。travel-auth 服务通过同一个 /login 接口 + type 字段路由的方式, 实现了一个入口服务于两端。

9.1 统一入口设计

TokenController 是 Auth 服务唯一的对外控制器, 包含 4 个接口:

接口	说明
POST /auth/login	统一登录入口, type 分流
POST /auth/register	统一注册入口, type 分流
DELETE /auth/logout	统一注销, 清除 Redis 登录态
POST /auth/refresh	刷新 token 过期时间

登录分流核心代码:

```
@RestController
public class TokenController {
    @Autowired private SysLoginService sysLoginService; // 后台管理员登录
    @Autowired private PcLoginService pcLoginService; // 前台用户登录

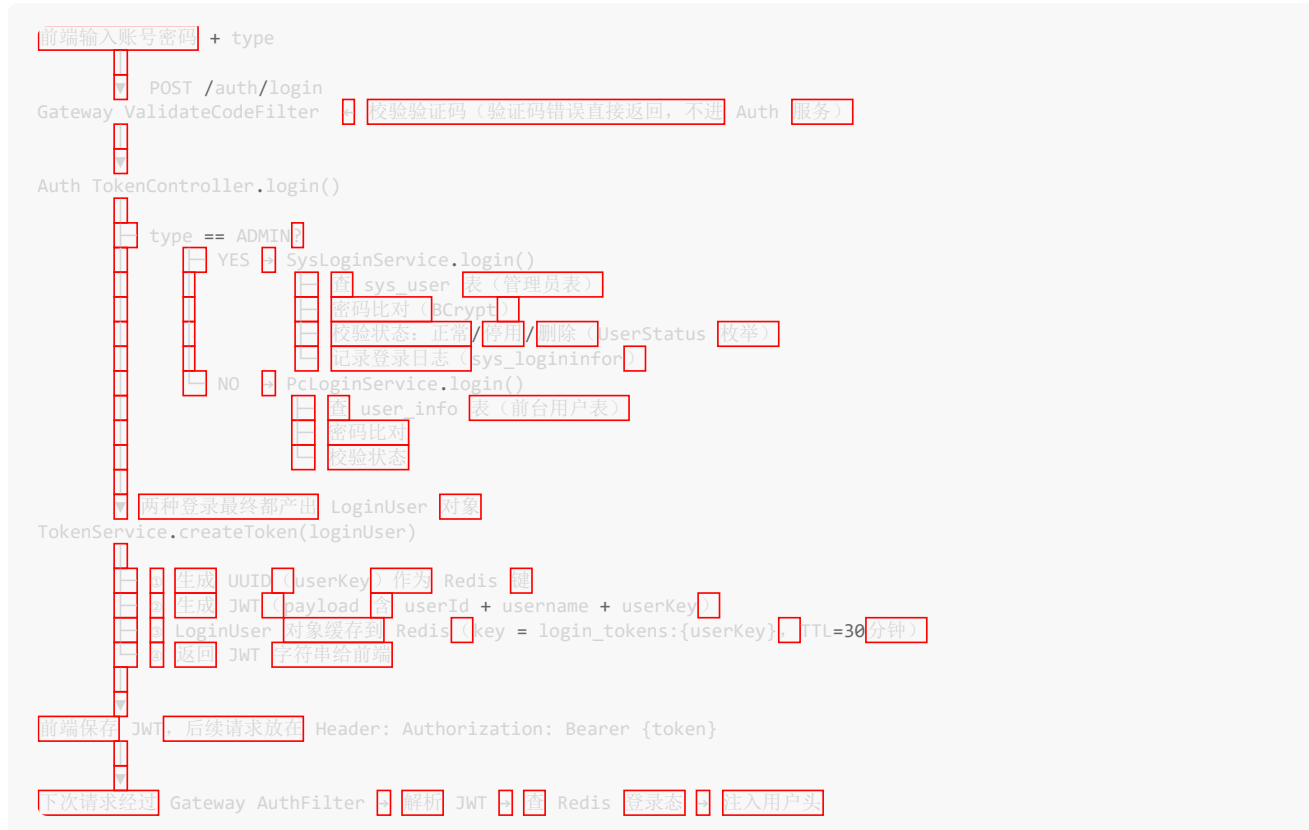
    @PostMapping("login")
    public R<?> login(@RequestBody LoginBody form) {
        LoginUser userInfo = null;
        // 同一个接口, 按 type 字段路由到不同的登录实现
        if (form.getType().equals(TokenConstants.ADMIN)) {
            userInfo = sysLoginService.login(form.getUsername(), form.getPassword());
        } else {
            userInfo = pcLoginService.login(form.getUsername(), form.getPassword());
        }
        // 统一颁发 token (不管是管理员还是前台用户, 走同一套 JWT 生成逻辑)
        return R.ok(tokenService.createToken(userInfo));
    }
}
```

LoginBody 表单对象:

```
public class LoginBody {
    private String username; // 账号
    private String password; // 密码
    private String type; // 登录类型: ADMIN (管理端) / PC (用户端)
}
```

面试题：这就是策略模式在 **Controller 层的轻量应用**——同一个接口通过参数路由到不同的业务 Service。比写两个 Controller `/auth/admin/login` 和 `/auth/pc/login` 更简洁，前端对接也只需要一个 URL。

9.2 整体认证流程



两层存储的设计：JWT 是自包含的（客户端携带），但“用户是否仍处于登录态”由 Redis 控制。这样即使用户没有点击注销、JWT 本身还有效，管理员也能通过删除 Redis 中的 key 强制用户下线，解决了“JWT 一旦签发无法撤销”的经典问题。

9.3 面试题模板（结合项目）

面试官问：你的系统登录是怎么设计的？后台管理员和前台用户是同一个表吗？

答：不是同一个表，我项目里后台管理员和前台用户分了两张表（`sys_user` 和 `user_info`），但**登录接口是同一个**，通过 `type` 字段路由。

接口设计：只有一个 `/auth/login`，接收 `{username, password, type}`，`type=ADMIN` 就走后台管理员登录（`SysLoginService`，查 `sys_user` 表），`type=PC` 就走前台用户登录（`PcLoginService`，查 `user_info` 表）。登录成功后统一由 `TokenService` 生成 JWT 返回，两套用户体系共用同一套 JWT 格式。

Token 机制：我用了 **JWT + Redis 双存储** 的方案。JWT 保存在前端，后续请求放 Header 里；Redis 里存一份完整的 `LoginUser` 对象作为“在线用户池”。这样做的好处是可以**强制用户下线**：即使 JWT 还没过期，只要把 Redis 对应的 key 删掉，网关 `AuthFilter` 校验时发现 Redis 里没有这个登录态，就直接返回 401“登录状态已过期”，解决了 JWT 无法撤销的问题。

安全方面：登录前经过网关的 `ValidateCodeFilter` 校验图形验证码，防止暴力破解；密码用 `BCrypt` 加密存储；登录结果写入 `sys_logininfor` 表留痕，方便审计。

9.4 面试可能追问的问题

Q1: JWT 和 Session 有什么区别？你为什么选择 JWT？

A:

| 维度 | Session | JWT |

|-----|-----|-----|

| 存储位置 | 服务端（内存/Redis） | 客户端（Cookie/LocalStorage） |

| 服务端状态 | 有状态，每次请求查 Session | 无状态（自包含），但本项目配合 Redis 做了半有状态 |

| 跨服务共享 | 需做 Session 粘滞或统一存储 | 天然跨服务，微服务友好 |

| 可撤销 | 直接删 Session | 纯 JWT 无法撤销，所以我加了 Redis 校验层 |

- 选 JWT 是因为微服务部署在多台机器上，Session 共享需要额外配置；JWT 天然支持水平扩展，配合 Redis 解决撤销问题。

Q2: JWT 的 payload 里放了哪些信息？敏感信息为什么不能放？

A: JWT 的 payload 只是 Base64 编码，**不是加密**，任何人拿到 token 都能解码看到内容。本项目 payload 只放 `userId`、`username`、`userKey` (UUID) 三个字段，不放密码、手机号、身份证号等敏感信息。如果一定要放敏感数据，应该用 JWE (JSON Web Encryption) 做加密后再签发。

Q3: 为什么不用两个登录接口 /admin/login 和 /pc/login？

A:

1. **前端对接简单**：前端写一套登录逻辑，只需要改 type 参数，不用维护两个 URL。
2. **统一安全策略**：验证码校验、限流、日志记录等横切逻辑只需要写在同一个入口，不会出现"后台接口加了验证码、前台忘了加"的情况。
3. **可扩展性**：后续再加 App 端登录 (type=APP)，还是同一个接口，新增一个 Service 实现即可，符合开闭原则。
4. **缺点**：if-else 分支如果太多可以改成策略模式 + Map 分发 (如 `Map<String, LoginService>`)，目前只有两个分支 if-else 就够了。

Q4: token 过期怎么办？refresh 接口是干什么的？

A: JWT 设置了过期时间 (比如 30 分钟)，用户一直操作的话不能让他到点被踢下线。`refresh` 接口就是：用户每次请求时，如果 Redis 中 LoginUser 的剩余有效期不足 20 分钟，就自动续期到 30 分钟 (滑动过期)。这样只要用户一直在操作，就永远不会掉线；只有用户闲置超过 30 分钟才需要重新登录。

十、Elasticsearch IK 分词器与全文搜索

✦ 所属简历模块：③ Elasticsearch 全文搜索

项目背景：`travel-modules-search` 服务负责攻略、目的地、游记、用户四类数据的全文搜索。项目踩了一个典型的坑——**分词器名称写错导致搜索结果不相关** (搜"广州"出来"杭州")，后通过统一修正为 `ik_max_word` 解决。

10.1 踩坑实录：广州 → 杭州的故事

现象：ES 搜索"广州"，结果返回了大量关于"杭州"的攻略。

根因定位：

1. 打开 ES 索引 mapping 一看，`title` 字段 analyzer 写的是 `"ik-max-word"` (连字符)。
2. 但 IK 分词器正确的名称是 `ik_max_word` (下划线)。
3. analyzer 名称写错时，ES 不会报错，而是**静默回退到 standard 分词器**。
4. standard 分词器对中文是"逐字切分"，于是：
 - "广州" → 切出 `广`、`州`
 - "杭州" → 切出 `杭`、`州`
 - 两者都包含 `州` 字，所以搜"广州"匹配到了"杭州" (共享 `州`)。

修复范围：4 个 ES Service 全部修正 analyzer 名称：

文件	修复内容
StrategyESServiceImpl.java	<code>.analyzer("ik_max_word")</code> (title、subTitle、summary)
DestinationESServiceImpl.java	<code>.analyzer("ik_max_word")</code> (name、info)
INoteESServiceImpl.java	<code>.analyzer("ik_max_word")</code> (title、summary)
UserInfoESServiceImpl.java	<code>.analyzer("ik_max_word")</code> (nickname、info、city)

面试点：遇到 ES 搜索结果"明明关键字不一样却匹配了"，第一时间检查分词器名称是否正确，用 `GET /{index}/_mapping` 查看生效的 mapping，再用 `GET /{index}/_analyze` 测试实际分词效果。

10.2 IK 分词器的两种模式

模式	说明	适用场景
<code>ik_max_word</code>	最细粒度 拆分，尽可能多地切出词。如"中华人民共和国国歌"→ 中华人民共和国 / 中华人民 / 中华 / 华人 / 人民共和 / 人民 / 共和国 / 共和 / 国 / 国歌	索引阶段（写） ，切的词越多，召回率越高
<code>ik_smart</code>	最粗粒度 拆分，切出最长的合理词。如"中华人民共和国国歌"→ 中华人民共和国 / 国歌	查询阶段（读） ，拆的词更精准，精确匹配文档

本项目索引阶段用 `ik_max_word`（最大召回），查询阶段可以配合 `ik_smart` 或 `match_phrase` 短语匹配做精度控制。面试时被问"为什么 mapping 里只用了 `ik_max_word`？"可以回答：查询阶段如果用 `match` 查询且指定 `analyzer=ik_smart`，会在搜索时再用 `ik_smart` 拆查询词，二者是配合关系。

10.3 ES 索引初始化流程 (StrategyESServiceImpl.init)

这是典型的"全量重建索引"操作，暴露给管理后台的一键初始化按钮：

```

public void init() throws IOException {
    // ① 先从 strategy 服务拉全量数据 (Feign 远程调用)
    R<List<Strategy>> result = remoteStrategyService.list("inner");
    if (R.isError(result)) { throw new IOException("读取攻略数据失败"); }
    List<Strategy> data = result.getData();

    // ② 索引存在则先删除 (旧 mapping 不会自动更新)
    if (client.indices().exists(e -> e.index(INDEX_NAME)).value()) {
        client.indices().delete(d -> d.index(INDEX_NAME));
    }

    // ③ 建新索引 + mapping (指定 IK 分词器)
    CreateIndexRequest request = CreateIndexRequest.of(r -> r
        .index(INDEX_NAME)
        .settings(s -> s.numberOfShards("1").numberOfReplicas("1"))
        .mappings(m -> m
            .properties("id", p -> p.long_(l -> 1)) // id 不分词
            .properties("title", p -> p.text(t -> t.analyzer("ik_max_word"))) // 标题分词
            .properties("subTitle", p -> p.text(t -> t.analyzer("ik_max_word"))) // 副标题分词
            .properties("summary", p -> p.text(t -> t.analyzer("ik_max_word"))) // 摘要分词
        ));
    client.indices().create(request);

    // ④ Bulk 批量导入 (比一条一条 index 性能高 10 倍以上)
    BulkRequest.Builder bulkBuilder = new BulkRequest.Builder();
    for (Strategy s : data) {
        StrategyES es = new StrategyES();
        BeanUtils.copyProperties(es, s);
        bulkBuilder.operations(op -> op
            .index(idx -> idx
                .index(INDEX_NAME)
                .id(String.valueOf(es.getId())) // 文档 id = 攻略 id, 便于后续按 id 更新
                .document(es)));
    }
    client.bulk(bulkBuilder.refresh(Refresh.WaitFor).build());
}

```

面试点：

1. 为什么"存在就删除"? 因为 ES 的 mapping 一旦创建就不能修改字段类型/分词器, 只能删了重建 (可以新增字段, 但不能改已有字段)。
2. 为什么用 Bulk? 批量导入减少网络往返次数, 性能显著提升。
3. `refresh(Refresh.WaitFor)` 表示等数据 refresh 后再返回, 保证写完立即可搜。

10.4 面试题模板 (结合项目)

面试官问: ES 的中文分词你怎么用的? 有没有遇到过分词的坑?

答: 用的是 IK 分词器, 索引阶段用 `ik_max_word` (最细粒度拆分, 最大化召回), mapping 里对 title、subTitle、summary 等文本字段都指定了 analyzer。

踩过的坑: 早期把分词器名称写成了 `ik-max-word` (连字符), 而正确名称是 `ik_max_word` (下划线)。ES 不会报错, 只是静默回退到 standard 分词器, standard 对中文逐字拆分, 导致搜"广州"匹配到了"杭州"——因为两词共享了"州"字。排查方法是用 `GET /index/_analyze` 接口测试实际分词结果, 一看是逐字切分就知道分词器没生效。后来把 4 个 ES Service (Strategy/Destination/Note/UserInfo) 的 analyzer 名称全部修正为 `ik_max_word`, 重建索引后搜索就正常了。

索引初始化流程: 管理后台有一键初始化功能, 先 Feign 拉 DB 全量数据, 删旧索引 (因为 mapping 不能改), 建新索引指定分片副本和 IK mapping, 最后用 Bulk API 批量写入, 比单条写入性能提升明显。

10.5 面试可能追问的问题

Q1: ES mapping 创建后还能修改吗?

A: **已有的字段不能修改类型/分词器**，可以新增字段。改已有字段只能删索引重建。所以我项目的 init 方法是"先删再建再导数据"的全量重建流程。如果线上不能停服务，应该用**别名迁移法**：建新索引 → 导数据 → 把别名从旧索引切到新索引（零停机）。

Q2: ik_max_word 和 ik_smart 有什么区别? 怎么配合?

A:

- **ik_max_word**：最细粒度，索引阶段用，切的词越多，搜得到的概率越高（召回率高）。
- **ik_smart**：最粗粒度，查询阶段用，把查询词拆成最有意义的词（精准率高）。
- **最佳实践**：mapping 索引时用 ik_max_word（写得全），查询时 search request 指定 **analyzer=ik_smart**（拆得准），二者配合兼顾召回和精准。

Q3: Bulk 批量写入和单条写入性能差多少? 要注意什么?

A:

- **性能**：Bulk 是"一批一次网络请求 + 一次 fsync"，单条是"每条一次网络请求 + 一次 fsync"。实测 1000 条文档 Bulk 比单条快 5~20 倍。
- **注意事项**：
 1. 批次大小不是越大越好，建议 5-15MB 一批（默认 1000-5000 条），过大会内存溢出。
 2. 本项目用 **refresh(Refresh.WaitFor)**——写完强制 refresh 立即可搜，适合初始化场景；正常写业务时不要加，会影响性能。
 3. 要检查 **bulkResponse.errors()**，批量写入是部分成功部分失败，不会整体抛异常。

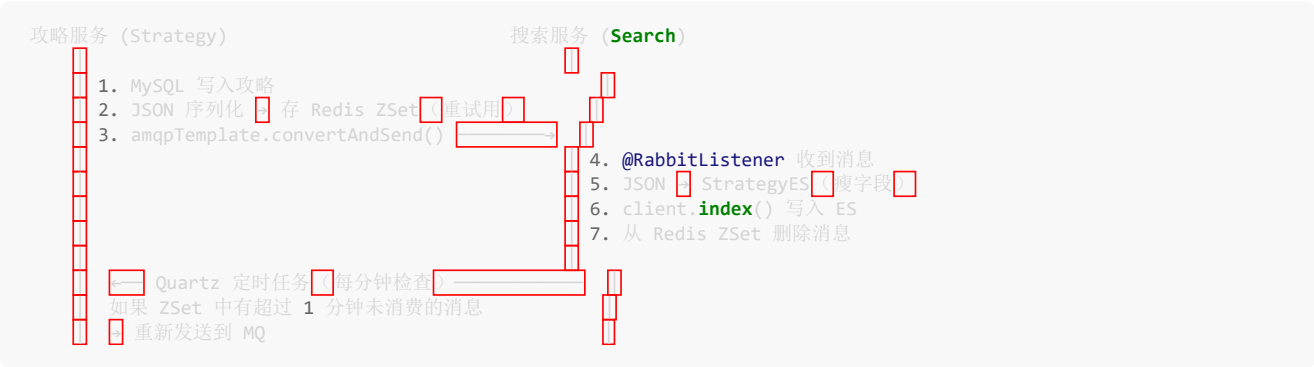
Q4: standard 分词器对中文是怎么处理的? 为什么会搜到不相关的结果?

A: standard 分词器是 ES 默认的，**对中文按单字切分**，"广州" → 广、州。这样任何带"州"字的城市（杭州、苏州、福州、郑州...）都会命中"广州"的查询，因为共享了"州"字。中文搜索必须显式配置中文分词器（IK、HanLP、Jieba 等），不能用默认的 standard。

10.5a RabbitMQ 异步同步 ES 索引

所属简历模块：③ Elasticsearch 全文搜索

整体流程



MQ 配置

核心文件: **RabbitConfig.java**


```
@Configuration
public class RabbitConfig {
    public static final String QUEUE_NAME = "travel-queue";
    public static final String EXCHANGE_NAME = "travel-exchange";
    public static final String ROUTING_KEY = "travel-routing-key";

    @Bean
    public Queue queue() {
        return new Queue(QUEUE_NAME, true, false, false); // durable=true
    }

    @Bean
    public Exchange exchange() {
        return new TopicExchange(EXCHANGE_NAME, true, false); // 持久化
    }

    @Bean
    public Binding binding(Queue queue, Exchange exchange) {
        return new Binding(QUEUE_NAME, Binding.DestinationType.QUEUE,
            EXCHANGE_NAME, ROUTING_KEY, null);
    }
}
```

组件	值	特性
队列	travel-queue	durable=true (持久化) , 重启不丢消息
交换机	travel-exchange	TopicExchange, 支持通配符路由
路由键	travel-routing-key	固定路由键

消息生产者 (Strategy 服务)

```
// StrategyServiceImpl.insert()
@Transactional
public int insert(Strategy strategy) {
    // 1. 业务逻辑: 填充主题名、分类名、目的地等
    // 2. 写 MySQL 主表
    baseMapper.insert(strategy);
    // 3. 写 MySQL 内容表 (大字段分离)
    strategyContentMapper.insert(strategyContent);
    // 4. JSON 序列化
    String message = JSON.toJSONString(strategy);
    // 5. 存入 Redis ZSet (score=时间戳, 用于重试)
    String key = RedisKeys.STRATEGY_RABBITMQ_ZSET.getPrefix();
    redisService.setCacheZSet(key, message, System.currentTimeMillis());
    // 6. 发送到 MQ
    amqpTemplate.convertAndSend(RabbitConfig.EXCHANGE_NAME,
        RabbitConfig.ROUTING_KEY, message);
    return ret;
}
```

关键点: 先写 MySQL, 再发 MQ。MySQL 和 MQ 之间没有事务保证, 但用 Redis ZSet 做了补偿。

消息消费者 (Search 服务)

```
// TravelRabbitListener.java
@Component
public class TravelRabbitListener {

    @Autowired
    private ElasticsearchClient client;
    @Autowired
    private RedisService redisService;

    @RabbitListener(queues = {RabbitConfig.QUEUE_NAME})
    public void receive(String message) throws Exception {
        // 1. JSON 反序列化为 Strategy
        Strategy strategy = JSON.parseObject(message, Strategy.class);

        // 2. 拷贝到搜索索引对象 (只取 4 个字段)
        StrategyES strategyES = new StrategyES();
        BeanUtils.copyProperties(strategy, strategyES);

        // 3. 写入 ES (按 _id 更新语义)
        client.index(i -> i
            .index(StrategyESServiceImpl.INDEX_NAME)
            .id(strategyES.getId().toString())
            .document(strategyES)
        );

        // 4. 消费成功, 从 Redis ZSet 删除
        String key = RedisKeys.STRATEGY_RABBITMQ_ZSET.getPrefix();
        redisService.deleteCacheZSetValue(key, message);
    }
}
```

Redis ZSet 重试机制 (伪死信队列)

```
// StrategyServiceImpl.checkRabbitMQMessage()
// Quartz 定时任务每分钟调用
public void checkRabbitMQMessage() {
    String key = RedisKeys.STRATEGY_RABBITMQ_ZSET.getPrefix();
    double time = System.currentTimeMillis() - 60000; // 1 分钟前

    // 查找 ZSet 中超过 1 分钟未消费的消息
    Set<String> set = redisService.rangeByScore(key, 0, time);

    if (set.isEmpty()) return;

    for (String message : set) {
        // 重新发送到 MQ
        amqpTemplate.convertAndSend(RabbitConfig.QUEUE_NAME, message);
        // 从 ZSet 删除
        redisService.deleteCacheZSetValue(key, message);
    }
}
```

重试流程:

1. 生产者发消息时, 同时存入 Redis ZSet (score=时间戳)
2. 消费者成功消费后, 从 ZSet 删除
3. Quartz 定时任务每分钟扫描 ZSet, 找出"发送超过 1 分钟仍未被消费"的消息
4. 重新发送到 MQ
5. 消费者按 `_id` 覆盖写入 ES, 天然幂等

为什么不用真正的死信队列？

对比	RabbitMQ DLX 死信队列	Redis ZSet 伪死信
可靠性	高（MQ 内部保证）	中（依赖 Redis 持久化）
复杂度	需要配置 DLX + DLQ + 绑定	简单，Redis ZSet + 定时扫描
重试控制	需要额外配置 TTL + 最大重试次数	灵活，自己控制扫描间隔和条件
适用场景	大规模、高可靠场景	中小规模、快速实现

项目选择 Redis ZSet 方案，是因为：已经有 Redis 基础设施，不想额外引入死信队列的复杂配置，够用就好。

面试题模板

"数据同步这块，我们用 **RabbitMQ 异步同步** MySQL 到 ES。攻略服务写入 MySQL 后，把攻略对象 JSON 序列化发送到 MQ，Search 服务的 `@RabbitListener` 监听到消息后，提取瘦字段写入 ES 索引。

这样做的好处是**解耦**——主业务（攻略服务）不依赖搜索服务，搜索服务挂了不影响攻略发布；搜索服务消费速度慢了也不会阻塞主业务。

为了保证消息不丢，发消息的同时会存一份到 Redis ZSet（score 是时间戳），消费者成功消费后从 ZSet 删除。Quartz 定时任务每分钟扫描 ZSet，发现超过 1 分钟没被消费的消息就重新发送。消费者按 `_id` 写入 ES，天然幂等，重复消费不会有问题。"

面试可能追问的问题

Q1：为什么不用 Canal 监听 Binlog 同步？

A：Canal 适合**已有的老系统**不想改代码的场景。我们是新系统，在业务代码里直接发 MQ 更可控——可以控制发什么内容、什么时候发。Canal 还需要额外部署和维护，增加运维成本。

Q2：MySQL 和 MQ 之间没有事务，消息丢了怎么办？

A：Redis ZSet 兜底。发 MQ 前先存 ZSet，消费成功后删 ZSet。定时任务扫描 ZSet 补发未消费的消息。虽然不是 100% 严格可靠，但对于搜索同步这个场景——就算丢了几条，大不了用 `esInit` 接口全量重建——是可以接受的最终一致性。

Q3：消费者消费失败了怎么办？

A：RabbitListener 抛异常后，消息会重新入队（默认行为）。消费者按 `_id` 写入 ES，重复消费是幂等的。如果持续失败，Redis ZSet 中的消息不会被删除，定时任务会补发。

Q4：为什么只有 insert 发 MQ，update 和 delete 没有？

A：当前版本确实只有 insert 发了 MQ。update 和 delete 的同步可以后续通过 Canal 监听 Binlog 来补，或者在 update/delete 方法里加同样的 MQ 发送逻辑。面试时可以主动说“这是一个已知的改进点”。

10.6 ES 瘦索引 + OpenFeign 远程补全 MySQL 完整数据

项目背景：`travel-modules-search` 服务的 ES 索引只存少量关键字段（id + 检索文本），完整数据通过 OpenFeign 远程调用各业务微服务（strategy/note/destination/member）从 MySQL 补全。这是微服务 + ES 搜索的经典架构。

10.6.1 什么是“瘦索引”？为什么这么设计？

瘦索引（Thin Index）：ES 索引里只存**用于检索的关键字段** + `id`，不存完整业务数据。

以攻略为例，`StrategyES`（ES 侧实体）只有 4 个字段：

```
@Data
public class StrategyES {
    private Long id;          // 主键, 用于回查 MySQL
    private String title;     // 用于检索
    private String subTitle;  // 用于检索
    private String summary;   // 用于检索
}
```

而 MySQL 完整实体 `Strategy` 有几十个字段 (coverUrl、viewnum、replynum、favorum、content 大字段等) , 其中 `content` 正文单独存在 `ta_strategy_content` 表。这些字段一个都不进 ES。

为什么用瘦索引而不是把所有字段都塞进 ES?

原因	说明
减小索引体积	content 等大字段会让 ES 索引膨胀数倍, 占用更多磁盘和 JVM 堆内存
降低同步成本	存的字段越少, MySQL 更新后要同步到 ES 的字段就越少, 双写一致性压力小
数据以 MySQL 为准	ES 只负责"检索定位", 完整数据仍由业务服务的 MySQL 提供, 职责单一
微服务解耦	Search 服务不需要知道各业务表的完整结构, 只需要 id + 检索字段

面试点: 被问"ES 里存了哪些字段? 为什么不把所有字段都存进去?" 就从上面 4 点答, 核心思路是"ES 只干检索的事, 完整数据交给业务库"。

10.6.2 Feign 远程补全的完整流程

搜索时的数据流 (`SearchServiceImpl.searchWithHighLight`) :



核心代码 (`SearchServiceImpl.getFullData`) :

```
private <T, K> T getFullData(String indexName, K esData, Class<T> clazz) {
    Long id = getId(esData); // 从 ES 瘦数据里提取 id
    try {
        Object data = null;
        switch (indexName) {
            case "strategy":
                data = remoteStrategyService.getOne(id, "inner").getData();
                break;
            case "note":
                data = remoteNoteService.getOne(id, "inner").getData();
                break;
            case "destination":
                data = remoteDestinationService.getOne(id, "inner").getData();
                break;
            case "userinfo":
                data = remoteUserInfoService.getOne(id, "inner").getData();
                break;
            default:
                // 没有对应远程服务, 就直接用 ES 瘦字段
                data = clazz.getDeclaredConstructor().newInstance();
                BeanUtils.copyProperties(esData, data);
                break;
        }
        return (T) data;
    } catch (Exception e) {
        return null; // 远程调用失败兜底
    }
}
```

4 个 Feign 接口都配置了 **FallbackFactory 降级工厂**，远程服务挂了会返回降级数据而不是抛异常，保证搜索页面不会因为某个业务服务故障而 500。

面试题：这是典型的"ES 定位 + DB 补全"模式。和"把所有字段存进 ES 直接返回"相比，瘦索引 + Feign 补全牺牲了一点查询性能（多一次远程调用），但换来了更小的索引、更简单的同步逻辑、更强的数据一致性。权衡是面试常考方向。

10.6.3 数据同步策略（怎么把 MySQL 数据同步到 ES 瘦索引？）

项目采用 **全量 + 增量** 两套机制：

同步方式	触发时机	实现方式
全量同步	管理后台一键初始化 / 项目首次启动	Feign 拉取 MySQL 全量 → 删旧索引 → 建新索引 → Bulk 批量写入
增量同步	业务数据新增/修改时实时同步	Strategy 服务发 RabbitMQ 消息 → Search 服务监听 → 写入 ES

增量同步的 MQ 消息体里只包含攻略对象，Search 服务收到后 `BeanUtils.copyProperties` 把瘦字段拷到 `StrategyES`，再 `client.index()` 写入 ES。

面试题：为什么用 MQ 而不是直接调接口？① 解耦（Strategy 服务不需要知道 Search 服务的存在）；② 异步（写操作不用等 ES 写完才返回）；③ 削峰（短时间大量更新也不会把 ES 打垮）。

10.7 多索引独立容错：单个索引异常不影响整体搜索

项目背景：综合搜索接口一次查 4 个索引（目的地/攻略/游记/用户），如果其中某个索引挂了（比如 userinfo 索引异常），不能让整个搜索接口报错返回。必须做到"单个索引降级，整体仍然可用"。

10.7.1 实现方式：每个索引独立 try-catch

`SearchServiceImpl.searchAll()` 方法的核心结构：

```
public Map<String, Object> searchAll(SearchQuery qo) {
    Map<String, Object> result = new LinkedHashMap<>();
    long total = 0;

    // 1. 目的地 — 独立 try-catch
    try {
        Page<Destination> destPage = searchWithHighLight("destination", ...);
        result.put("dests", destPage.getContent());
        total += destPage.getTotalElements();
    } catch (Exception e) {
        result.put("dests", new ArrayList<>()); // 异常只置空，不抛出
    }

    // 2. 攻略 — 独立 try-catch
    try {
        Page<Strategy> strategyPage = searchWithHighLight("strategy", ...);
        result.put("strategies", strategyPage.getContent());
        total += strategyPage.getTotalElements();
    } catch (Exception e) {
        result.put("strategies", new ArrayList<>());
    }

    // 3. 游记 — 独立 try-catch
    try {
        Page<Note> notePage = searchWithHighLight("note", ...);
        result.put("notes", notePage.getContent());
        total += notePage.getTotalElements();
    } catch (Exception e) {
        result.put("notes", new ArrayList<>());
    }

    // 4. 用户 — 独立 try-catch
    try {
        Page<UserInfo> userPage = searchWithHighLight("userinfo", ...);
        result.put("users", userPage.getContent());
        total += userPage.getTotalElements();
    } catch (Exception e) {
        result.put("users", new ArrayList<>());
    }

    result.put("total", total);
    return result;
}
```

- 关键点：
- 4 个索引分别在 4 个独立的 `try-catch` 块中。
 - 某个索引抛异常（索引不存在、ES 节点故障、分词器报错、Feign 超时等）→ **catch 住，放一个空列表**，继续执行下一个索引。
 - 用 `LinkedHashMap` 保证返回顺序：目的地 → 攻略 → 游记 → 用户。
 - `total` 只累加成功的索引结果。

10.7.2 面试延伸：还有哪些常见的容错/降级手段？

项目里用到的 + 可以扩展说的：

容错手段	项目中的体现
try-catch 独立降级	searchAll 中每个索引独立 try-catch，返回空列表
Feign FallbackFactory	4 个 RemoteXxxService 都有降级工厂，远程服务挂了返回 <code>R.fail()</code>
Sentinel 熔断	集成了 Sentinel，当异常比例/慢调用比例达到阈值时自动熔断
超时控制	Feign + Ribbon/Sentinel 配置了连接超时和读取超时，防止慢请求拖死线程

面试点：被问“你们的搜索接口怎么做容错的？”可以按这个顺序答：① 多索引各自 try-catch，单个失败返回空列表，不影响整体；② Feign 配了 FallbackFactory，远程服务挂了有降级数据；③ 集成了 Sentinel 做熔断限流，防止故障扩散。

10.8 面试题模板（瘦索引 + 容错）

面试官问：你们 ES 搜索是怎么设计的？存了哪些字段？为什么不把所有字段都存进去？

答：我们用的是瘦索引 + Feign 远程补全的架构。ES 索引里只存 `id` 和几个需要全文检索的文本字段（比如攻略的 title、subTitle、summary），像正文 content、浏览量、封面图这些字段都不进 ES。完整数据是搜索到 id 之后，再通过 OpenFeign 远程调用对应的业务微服务，从 MySQL 里查出来补全的。

为什么不把所有字段都存进 ES？ 主要 4 个原因：

- 1. 减小索引体积：content 这种大字段存进去索引会膨胀好几倍，磁盘和内存压力都大。
- 2. 降低同步成本：存的字段越少，MySQL 更新后要同步的字段就越少，双写一致性的压力小。
- 3. 职责分离：ES 只干“检索定位”的事，完整数据以 MySQL 为准，业务数据的权威源是各业务服务。
- 4. 微服务解耦：Search 服务不需要了解各业务表的完整结构，保持了模块间的独立性。

代价是每次搜索多了一次 Feign 远程调用，但因为是按主键 id 查询，MySQL 性能很快，整体延迟可接受。

面试官追问：一次搜多个索引，如果其中一个索引挂了怎么办？

答：我们的综合搜索一次查 4 个索引（目的地、攻略、游记、用户），每个索引的搜索逻辑都在独立的 `try-catch` 块里。某个索引异常（比如索引不存在、ES 分片故障），catch 住后往结果里放一个空列表，不会抛出异常，其他索引的结果照常返回。这样用户至少能看到其他模块的搜索结果，而不是整个页面报错。

除此之外，Feign 调用也配了 `FallbackFactory` 降级工厂，远程服务挂了会返回降级数据。再加上 Sentinel 熔断，三层容错保证搜索接口的可用性。

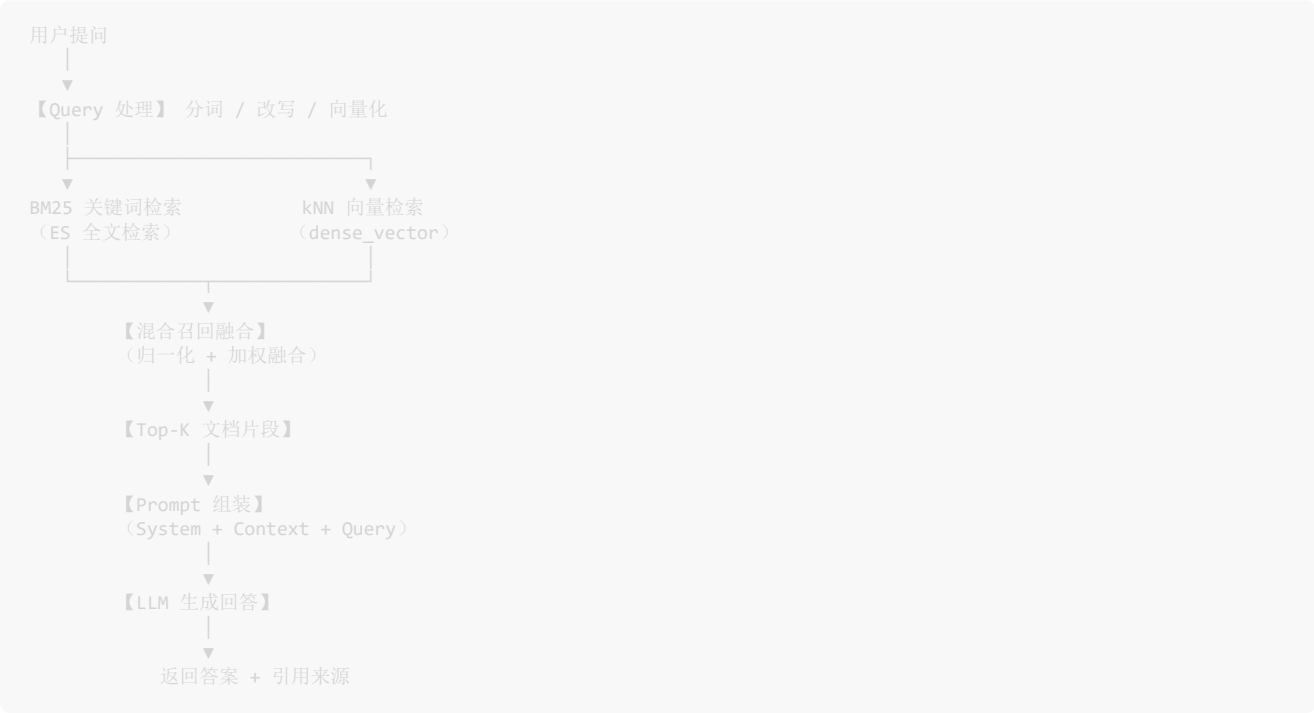
十一、RAG 智能问答与 ES 向量检索（混合检索）

所属简历模块：② AI 智能导购问答

项目背景：在 ES 全文检索基础上扩展 RAG（Retrieval-Augmented Generation，检索增强生成）智能问答能力。基于攻略、游记、目的地等旅游领域内容构建向量知识库，利用 ES 8.x 原生支持的 `dense_vector` 字段做 kNN 向量检索，与 BM25 关键词检索形成“双路召回”，融合后注入 LLM 生成自然语言回答。

⚠ 说明：本功能处于研发/调优阶段，面试时可讲方案设计与技术选型，避免说“已全量上线”。

11.1 RAG 整体架构流程



- 四大核心阶段：
- 1. 索引构建阶段：文档切分 → Embedding 向量化 → 写入 ES (dense_vector 字段 + 文本字段)
 - 2. 召回阶段：BM25 关键词检索 + kNN 向量检索，双路召回
 - 3. 融合排序阶段：两路结果归一化后加权融合，取 Top-N
 - 4. 生成阶段：将相关上下文拼入 Prompt，调用 LLM 生成自然语言回答

11.2 为什么用 ES 做向量检索，不用 Milvus / Pinecone？

这是写了 RAG 后面试官必问的第一个问题。从三个层面答：

维度	理由
技术栈复用	项目已经有 ES 8.x 在跑全文检索，再加一个 dense_vector 字段就能做向量检索，零基础设施成本
混合检索方便	BM25 和 kNN 都在同一个 ES 集群里，一次请求就能做 hybrid search，不用跨两个系统做结果合并
数据量匹配	旅游领域知识库规模在十万~百万级文档，ES kNN 完全够用。如果是亿级向量，再考虑专用向量数据库

面试话术：我们选 ES 做向量检索主要是因为项目已经有 ES 基础设施了，ES 8.x 原生支持 dense_vector 和 kNN 搜索，中小规模知识库完全够用。而且混合检索特别方便，BM25 和向量都在同一个 ES 里，不用跨系统合并结果，架构更简单。如果以后向量规模上来了（千万级以上），再考虑迁移到 Milvus 这类专用向量数据库。

11.3 ES dense_vector + kNN 检索原理

11.3.1 什么是 dense_vector？

ES 8.x 引入的字段类型，专门存高维向量。索引映射示例：

```
{
  "mappings": {
    "properties": {
      "content_vector": {
        "type": "dense_vector",
        "dims": 768,
        "similarity": "cosine"
      },
      "content": {
        "type": "text",
        "analyzer": "ik_max_word"
      }
    }
  }
}
```

关键参数：

- **dims**：向量维度，必须和 Embedding 模型输出一致（如 BGE-base 是 768 维）
- **similarity**：相似度计算方式，常用 **cosine**（余弦相似度）、**dot_product**（点积）、**l2_norm**（欧氏距离）

11.3.2 kNN 检索怎么用？

ES 8.x 支持在 **knn** 子句中直接做近似最近邻搜索：

```
{
  "knn": {
    "field": "content_vector",
    "query_vector": [0.1, 0.5, ..., 0.3],
    "k": 10,
    "num_candidates": 100
  }
}
```

- **k**：返回多少条最相似的结果
- **num_candidates**：每个分片提前筛选的候选数量，越大越准但越慢，通常设为 k 的 10~20 倍

11.3.3 混合检索 (Hybrid Search)

关键词检索 + 向量检索同时执行，ES 用 **rrf**（Reciprocal Rank Fusion，倒数排名融合）自动合并结果：

```
{
  "query": {
    "match": { "content": "三亚旅游攻略" } // BM25 关键词
  },
  "knn": {
    "field": "content_vector",
    "query_vector": [...], // 向量语义
    "k": 20,
    "num_candidates": 200
  },
  "rank": {
    "rrf": {
      "window_size": 50,
      "rank_constant": 60
    }
  },
  "size": 10
}
```

面试点：RRF 是一种经典的无权重融合算法，不需要人工调权重，公式是 $score = \sum 1 / (rank_constant + rank)$ 。两个召回源各自的排名越靠前，融合分越高。好处是不需要对两路分数做归一化，直接按排名融合，鲁棒性强。

11.4 文档切分策略（Chunking）

文档不能整段向量化——太长了语义混杂，也超过模型最大长度。需要先切分（Chunk）再向量化。

常见切分方式：

切分策略	原理	优点	缺点
固定长度切分	按 token/字符数切，如 512 token/块	简单、稳定	可能切断语义
按结构切分	按标题/段落/列表等 Markdown 结构切	语义完整	需要文档有规范结构
递归字符切分	优先按分隔符（\n\n → \n → 。 → ， → ）切，不行再按长度切	兼顾结构与长度	实现稍复杂
语义切分	用 Embedding 相似度判断边界	最智能	慢、成本高

项目实践：旅游攻略/游记类内容用递归字符切分 + overlap 最实用：

- 每块 500~800 token
- overlap 50~100 token（保证上下文连续，避免切在关键句中间）
- 优先按 `\n\n`（段落）切，再按 `。`（句号）切

面试点：为什么需要 overlap？因为如果切分刚好切断了一个完整语义（比如一句话被切成两半），那两块向量表达都会受影响。加一点重叠（一般 10%~20%），保证关键信息至少出现在一个完整块里。

11.5 面试必问：RAG 怎么优化？

面试官问“你们 RAG 做了哪些优化”，按以下维度答，覆盖全面又有层次：

11.5.1 召回层优化（最关键）

优化手段	说明
混合检索	BM25 + 向量双路召回，兼顾精确匹配与语义泛化
Query 改写	用户问题可能表述不清，先用 LLM 改写/扩展成更适合检索的查询
多路召回	除了 BM25 + 向量，还可以加同义词召回、拼音纠错、热门 Query 缓存
重排序（Rerank）	召回 Top-50 后用 Cross-Encoder 重排取 Top-5，精度显著提升
分段策略优化	调整 chunk 大小和 overlap，找到最佳粒度

11.5.2 生成层优化

优化手段	说明
Prompt 工程	System Prompt 明确“只根据给定信息回答，不知道就说不知道”，减少幻觉
引用溯源	返回答案时附上引用的文档来源，用户可验证，也提升可信度
对话历史管理	多轮对话时压缩/摘要历史，避免上下文过长

11.5.3 工程层优化

优化手段	说明
Embedding 缓存	相同问题/相似问题的向量缓存到 Redis，不用重复计算
流式输出	LLM 用 SSE 流式返回，降低首字延迟，用户体验好
超时控制	召回和生成都设超时，避免慢请求拖垮服务
并行调用	BM25 和向量检索并行发，Embedding 和检索并行，降低总时延

面试话术：RAG 优化我觉得最有收益的是三块：第一是混合检索，BM25 + 向量双路召回，比单一路召回准很多；第二是加 Rerank 重排，召回 Top-50 后用 Cross-Encoder 精排到 Top-5，精度提升明显；第三是 Prompt 工程，明确约束模型只能基于给定信息回答，能显著减少幻觉。工程上还要注意 Embedding 缓存、流式输出、超时控制这些细节，保证用户体验和系统稳定性。

11.6 面试题模板（横向迁移到电商/趣谷集）

面试官问：介绍一下你们的 RAG 智能问答项目。

答（电商场景版）：

我在趣谷集项目里负责 RAG 智能导购问答的检索模块开发。用户可以用自然语言问"夏天适合穿什么连衣裙"、"有没有 500 以内的运动鞋推荐"这类问题，系统基于商品信息、用户评价、帮助文档等知识库，结合大模型给出精准回答。

架构上，我们用的是 ES 8.x 作为向量检索引擎，因为项目已经有 ES 在跑商品全文搜索了，直接加 dense_vector 字段就能支持向量检索，复用成本低。整个 RAG 流程分四步：文档切分向量化 → 混合召回 → 融合排序 → LLM 生成。

检索层做了混合检索：BM25 关键词检索负责精确匹配（比如品牌名、型号），kNN 向量检索负责语义匹配（比如"夏天穿的裙子"和"夏季连衣裙"语义相近）。两路结果用 RRF 倒数排名融合，取 Top-20 送给重排模型精排，最后拿 Top-5 拼到 Prompt 里。

生成层用了 Prompt 工程约束模型只能基于给定的商品信息回答，不知道就说"暂时没有相关信息"，减少幻觉。回答里还会附上商品引用，用户可以点击跳转。

工程优化方面，做了 Embedding 结果缓存（相同问题不用重复算向量）、流式输出（SSE 降低首字延迟）、并行检索（BM25 和向量同时发请求）这些优化，保证接口响应速度和并发能力。

目前系统还在持续调优中，主要在做 chunk 粒度调优和 Rerank 模型选型这块。

11.7 常见追问与回答

Q1：ES 的 kNN 和 Milvus 比有什么优劣？

A：

- **ES 优势：**技术栈统一、混合检索方便（BM25 + 向量一站式）、运维成本低、中小数据量够用
- **Milvus 优势：**专门为向量检索设计，支持标量过滤 + 向量检索复合查询性能更好、支持十亿级向量、索引类型更丰富（HNSW / IVF / DiskANN)
- **选型建议：**已有 ES 且数据量 < 千万级，用 ES；纯向量检索、数据量大、对性能要求高，用 Milvus

Q2：向量维度越高越好吗？

A：不一定。维度越高，语义表达能力越强，但：

- 存储和计算成本线性增加
- 超过一定维度后边际收益递减（diminishing returns）
- 一般选择 768 维（base 模型）或 512 维（small 模型）是性价比最高的
- 实际选型要做 A/B 测试，看效果 vs 成本的 trade-off

Q3: RAG 和 Finetune (微调) 有什么区别? 什么时候用哪个?

A:

维度	RAG	Finetune
原理	检索相关文档注入上下文	在领域数据上继续训练模型
知识更新	简单, 更新知识库即可	需要重新训练, 成本高
幻觉控制	好 (有来源可验证)	差 (模型自己编)
适用场景	事实性问答、知识库问答	风格模仿、特定任务能力提升
成本	低	高 (数据+算力+时间)

结论: 企业内部知识库问答、商品问答这类需要"准确、可溯源、经常更新"的场景, 选 RAG。需要模型掌握某种特定风格或能力 (如写特定风格文案、代码生成) 且数据稳定的场景, 选微调。两者也可以结合用: RAG 管事实知识, 微调管回答风格。

Q4: 怎么评估 RAG 效果?

A: 分三层评估:

- **召回层:** Recall@K (相关文档在前 K 条里的比例)、MRR (第一个相关文档的排名倒数均值)
- **生成层:** 人工评测 (准确率/完整性/相关性)、RAGAS (自动评估框架, 含 faithfulness/answer relevancy 等指标)
- **业务层:** 用户满意度、追问率、答案采纳率

11.8 进阶方案: IP 导购 RAG 多域架构 (二次元电商场景)

这是 RAG 在垂直领域 (二次元电商) 的进阶设计, 面试主动提出来非常加分 —— 普通 RAG 谁都能说, 但"IP 人设 + 商品多域检索 + 角色化回答"是有真实业务思考的, 不是套模板。

简历描述 (推荐版)

AI 智能导购问答 (RAG) : 参与二次元电商 IP 智能导购问答系统开发。基于商品、评价、IP 人设等多源内容构建向量知识库, 结合关键词检索与向量语义检索实现混合召回; 针对二次元业务特点设计 IP 意图识别与角色化回答机制, 让模型以 IP 角色口吻回答商品相关问题, 提升导购体验与用户代入感。

说明: 这是**安全版**描述, 没有堆砌 RRF、Rerank 等容易被追问的术语, 但核心亮点 (多源知识库、混合召回、IP 意图识别、角色化回答) 都保留了。面试官如果追问细节, 再按下面的技术点展开。

11.8.1 为什么要做 IP 导购 RAG?

二次元电商的核心资产不是商品, 是 **IP (知识产权/角色)**。用户的问题往往不是"有什么 T 恤", 而是"初音未来有什么周边"、"这个角色推荐哪款"。普通商品 RAG 只能回答商品属性, IP 导购 RAG 还要:

- 理解用户说的是哪个 IP / 哪个角色
- 回答要符合 IP 人设 (不能让"傲娇"角色说软萌台词, 那叫 OOC)
- 把 IP 知识和商品信息融合在一起回答

这就从"商品检索问答"升级成了"**角色化导购助手**", 技术复杂度和业务价值都上了一个台阶。

11.8.2 多域知识库架构

不是一个大索引搞定, 而是**按业务域拆成多个 ES 索引**, 各自独立管理、独立加权:

索引	内容	作用	向量维度
ip_profile_index	IP 人设、角色关系、世界观、经典台词、性格标签	保证回答符合 IP 人设，避免 OOC	768
product_index	商品标题、详情、规格、价格、所属 IP/角色	回答商品相关问题	768
review_index	用户评价、晒单、买家问答	回答真实体验类问题 ("质量怎么样? ")	768
help_index	配送、退换货、会员规则、售后政策	服务/售后类问题	512

为什么要分索引？（面试可以主动讲）

- 1. 独立管理：IP 人设更新频率低，商品每天上下架，索引生命周期不一样
- 2. 独立加权：不同问题类型各域权重不同（问"是谁"→ IP 域权重高，问"多少钱"→ 商品域权重高）
- 3. 检索路由：先识别 IP，再在对应 IP 域内检索，避免全库瞎搜
- 4. 容错隔离：某个索引挂了不影响其他域（和搜索模块多索引容错一个思路）

11.8.3 IP 意图识别 + 检索路由

用户问"初音未来有什么周边"，不能全 4 个索引瞎搜，得先识别 IP，再限定域检索：



技术关键点：ES knn 的 pre-filter

ES 的 knn 查询支持 filter 子句，可以在向量检索前先按标量字段过滤：

```
{
  "knn": {
    "field": "content_vector",
    "query_vector": [...],
    "k": 10,
    "num_candidates": 100,
    "filter": {
      "term": { "ip_id": 1001 }
    }
  }
}
```

- 先过滤 ip_id = 1001 的文档，再在里面做向量近邻搜索
- 既精准，又不损失性能（比"先全量召回再过滤"高效得多）
- 这是标量 + 向量混合检索的典型用法，面试提出来很专业

11.8.4 角色化 Prompt 注入（IP 人设怎么用）

普通 RAG 的 Prompt 结构：

```
System: 你是一个客服助手
Context: {商品信息}
Question: {用户问题}
```

IP 导购 RAG 的 Prompt 结构：

```
System: 你是 {IP名称}, {性格描述}, 口头禅是"{经典台词}"。
        请以 {IP名称} 的口吻回答问题, 不要OOC(脱离人设)。
        回答只能基于下面提供的信息, 不知道就说"唔...这个人家也不太清楚呢~"。

IP 人设知识:
- {从 ip_profile_index 召回的人设片段}

商品信息:
- {从 product_index 召回的商品信息}

用户评价:
- {从 review_index 召回的评价摘要}

Question: {用户问题}
```

核心思路：把召回的 IP 人设片段注入 **System Prompt**，把商品和评价注入 **Context**，各司其职。System 管"你是谁、怎么说话"，Context 管"你知道什么事"。

面试点：IP 人设放 System Prompt 还是 Context? 答：放 System Prompt 更稳定 —— System 指令是模型的行为准则，权重更高，不容易被上下文带偏。但要注意人设不能太长，否则会挤占 Context 空间，所以召回 Top-3 最相关的人设片段就够了。

11.8.5 和通用 RAG 的对比（一句话差异化）

维度	通用商品 RAG	IP 导购 RAG（二次元电商）
知识库	单域（商品）	多域（IP + 商品 + 评价 + 帮助）
检索方式	直接混合检索	IP 意图识别 → 路由 → 分域检索 → 融合
回答风格	客观客服式	角色化（IP 口吻）
核心指标	准确率、响应速度	准确率 + 人设一致性 + 商品转化率
技术亮点	混合检索、Rerank	多域检索路由、pre-filter kNN、角色化 Prompt

11.9 IP 导购 RAG 面试题模板（趣谷集版）

面试官问：介绍一下你们的 RAG 智能问答项目。

答：

我在趣谷集二次元电商负责 IP 智能导购问答的检索模块设计。因为是二次元电商，IP 是核心资产，用户很多问题都是"某某 IP 有什么周边"、"这个角色推荐哪款"，所以我们的 RAG 不是简单的商品问答，而是 **IP 人设 + 商品 + 用户评价** 的多域知识问答。

知识库架构上，我分了四个域建 ES 索引：IP 人设库、商品库、用户评价库、帮助文档库。都用 ES 8.x 的 **dense_vector** 存向量。分开建的好处是各域可以独立管理、独立加权，也方便做检索路由和故障隔离。

检索层做了两层设计：第一层是 **IP 意图识别**，先判断用户问的是哪个 IP，然后用 ES 的 pre-filter kNN 在对应 IP 域内检索，避免全库乱搜，也提升了精准度；第二层是 **BM25 + 向量混合检索**，IP 人设和商品描述两路召回，用 RRF 倒数排名融合，再走 Rerank 重排序 Top-5。

生成层的特色是**角色化回答**。我把召回的 IP 人设片段注入 System Prompt，告诉模型"你是谁、什么性格、口头禅是什么"，然后把商品信息和用户评价作为 Context 注入，让模型以 IP 角色的口吻回答问题。这样用户代入感强，也比普通客服式回答更有辨识度。System 管人设，Context 管事实，职责分离，效果更稳定。

工程优化方面，做了 Embedding 结果缓存、SSE 流式输出、多路并行检索这些优化，保证接口响应速度。目前在重点调 IP 人设的一致性和回答的商品转化率，还有很多可以优化的方向。

11.10 IP 导购 RAG 核心技术点详解

本节逐个拆解简历描述里的 10 个技术点，每个技术点讲清楚「是什么、为什么、怎么实现、面试怎么说」。

① 多域向量知识库

是什么：不把所有数据塞到一个大索引里，而是**按业务领域拆成多个独立向量索引**（IP 人设库 / 商品库 / 评价库 / 帮助库），每个库存各自的文本 + 向量。

为什么这么做：

- **独立管理：**不同领域数据更新频率不一样（IP 人设很少改，商品每天上下架）
- **独立加权：**不同问题类型各域权重不同（问"是谁"→ IP 域权重高，问"多少钱"→ 商品域权重高）
- **故障隔离：**一个索引挂了不影响其他域（和搜索模块多索引容错一个思路）
- **检索路由：**识别到 IP 后，可以只在对应 IP 域内检索，避免全库瞎搜

面试怎么说："我们按业务域拆成了四个向量索引，IP 人设、商品、评价、帮助文档各自独立，方便分域加权、独立管理和故障隔离。"

② IP 意图识别

是什么：用户问"初音未来有什么周边"，系统首先要**识别出用户说的是哪个 IP / 哪个角色**，然后再去对应的 IP 域里搜。

实现方案（从轻到重）：

方案	原理	优点	缺点
关键词词典匹配	维护 IP 名称词典，直接字符串匹配	快、准、零成本	别称/谐音/拼写错误匹配不到
向量相似度匹配	把 IP 名称/别名向量化，用户问题算向量后找最相似的 IP	能理解语义变体	有一定误判率
LLM 做分类	让大模型判断"这句话提到了哪个 IP"	最智能	慢、成本高

项目实践：先用关键词词典兜底（命中了就直接用），没命中再走向量相似度，两层结合。

面试怎么说："IP 意图识别用关键词 + 向量双层方案，关键词命中直接用，没命中再走 Embedding 相似度匹配，兼顾速度和泛化能力。"

③ 检索路由

是什么：识别到 IP 后，**把检索限定在该 IP 的数据范围内**，不是全库搜。比如识别到"初音未来"，就只在 ip_id = 1001 的文档里做向量检索。

技术实现：ES knn 的 pre-filter

```
{
  "knn": {
    "field": "content_vector",
    "query_vector": [1, 2, 3],
    "k": 10,
    "filter": { "term": { "ip_id": 1001 } }
  }
}
```

filter 是在向量检索**之前**先做标量过滤，先把 ip_id ≠ 1001 的文档全部排除，再在剩下的里面找最近邻。

面试易错点："先过滤再检索" ≠ "先检索再过滤"。前者高效精准（叫 pre-filter），后者浪费算力还可能召回不足。ES 的 knn filter 是前者。

面试怎么说："用 ES 的 pre-filter knn 做检索路由，先按 ip_id 过滤再做向量近邻搜索，既精准又不损失性能。"

④ BM25

是什么：ES 默认的关键词相似度算分算法，是 TF-IDF 的改进版。核心思想是“**关键词匹配越密集、越稀有，得分越高**”。

核心概念：

- **TF (词频)**：词在文档中出现次数越多，分越高
- **IDF (逆文档频率)**：词在全索引中越稀有，分越高
- **长度归一化**：短文档中出现一次的权重比长文档中出现一次的权重高

BM25 擅长：精确匹配、专有名词、型号、品牌名。

BM25 不擅长：同义词、语义相近但用词不同（"裙子"和"连衣裙" BM25 认为不相关）。

面试怎么说："BM25 是 ES 默认的关键词检索算法，基于词频和逆文档频率算分，精确匹配好但不理解语义，所以要和向量检索结合。"

⑤ kNN 向量检索

是什么：k-Nearest Neighbors, k 近邻搜索。把用户问题转成向量，在向量空间里找**距离最近（语义最相似）的 k 个文档**。

向量相似度衡量：通常用**余弦相似度（cosine similarity）**。

- 余弦值越接近 1 → 两个向量方向越一致 → 语义越相似
- 余弦值接近 0 → 语义不相关

向量检索擅长：语义泛化、同义词、不同表述但意思一样。

向量检索不擅长：精确的数字/型号匹配。

所以 BM25 和向量是**互补**的，一个管精确匹配，一个管语义泛化，加在一起效果最好 → 这就是"混合召回"。

面试怎么说："kNN 向量检索是通过 Embedding 把文本映射到高维向量空间，用余弦相似度找语义最接近的文档，擅长同义词和语义泛化，和 BM25 互补。"

⑥ RRF 融合 (Reciprocal Rank Fusion)

是什么：倒数排名融合，一种**把多个召回源的结果合并排序**的算法。不需要对不同来源的分数做归一化，只看**排名**。

公式： `最终得分 =  $\sum 1 / (\text{rank\_constant} + \text{rank})$`

举个例子，rank_constant = 60：

文档	BM25 排名	向量排名	RRF 得分
A	第 1 名	第 3 名	$1/61 + 1/63 = 0.0323$
B	第 2 名	第 1 名	$1/62 + 1/61 = 0.0325$
C	第 5 名	第 2 名	$1/65 + 1/62 = 0.0315$

B 在向量排第 1、BM25 排第 2，综合得分最高，排第一。

为什么用 RRF 而不是加权求和？

- 加权求和需要先归一化两路的分数（BM25 分可能是 20，向量相似度可能是 0.9，量级差太远，不能直接加）
- RRF 只看排名，不管原始分数是多少，**不用调权重**，鲁棒性强，开箱即用

面试怎么说："混合检索的结果融合用 RRF 倒数排名融合，只看排名不看原始分数，不用做分数归一化，也不用调权重，鲁棒性强，开箱即用。"

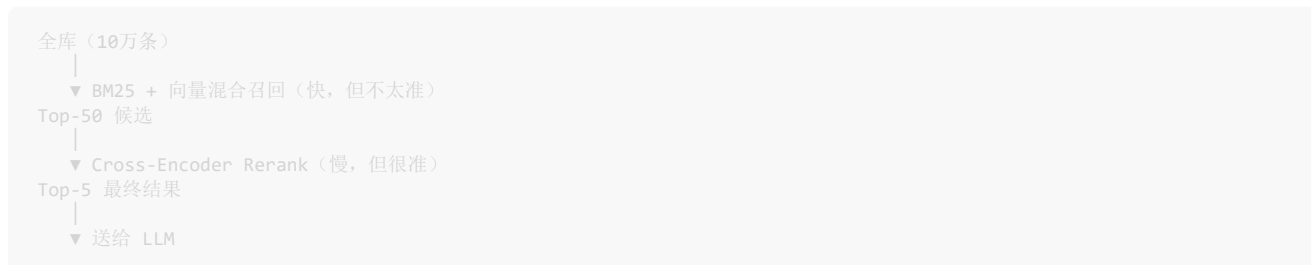
⑦ Rerank 重排

是什么：召回阶段先快速捞出 Top-50 个候选（粗筛），然后用一个**更精准但更慢的模型**重新计算相似度，取 Top-5（精排）。

为什么不直接用精排模型检索全库？

- 精排模型（如 Cross-Encoder）效果好，但慢，计算量大
- 全库精排不现实，所以先粗召回快速缩小范围，再精排

两阶段流程：



常见 Rerank 模型：bge-reranker、bce-reranker、Cohere Rerank 等。

面试点：RAG 效果提升最大的两个手段是「混合检索」和「加 Rerank」。加了 Rerank 之后，召回准确率通常能提升 10~30 个百分点，是性价比最高的优化手段之一。

面试怎么说："召回后加了一道 Rerank 重排，先粗召回 Top-50 再用 Cross-Encoder 精排 Top-5，在可接受的延迟内大幅提升召回精度。"

⑧ 角色化 Prompt (IP 人设注入)

是什么：不是让模型像"客服"一样客观回答，而是让模型**扮演某个 IP 角色，用角色的语气和性格回答问题**。

怎么实现：把 IP 人设信息放在 **System Prompt** 里：

你是初音未来，一个活泼开朗的虚拟歌姬，
说话喜欢用"呢~"、"哦~"结尾，
口头禅是"世界第一公主殿下！"。
请以初音未来的口吻，根据下面的信息回答用户问题。
注意：
1. 只能基于提供的商品信息回答，不知道就说"唔...这个人家也不太清楚呢~"
2. 不要脱离人设哦（不许OOC）

为什么放 System Prompt 而不是 Context？

- **System Prompt** 是模型的"行为准则"，权重最高，模型最听话 → 放人设
- **Context** 是"你知道的事实"，权重次之 → 放商品信息
- 职责分离，人设稳定，不容易被上下文带偏

面试怎么说："IP 人设注入 System Prompt，商品信息放 Context，System 管角色行为、Context 管回答事实，职责分离，人设一致性更好。"

⑨ 减少幻觉

是什么：幻觉 (Hallucination) 是大模型一本正经地胡说八道，编造假信息。RAG 的核心目标之一就是减少幻觉。

RAG 怎么减少幻觉 (三板斧)：

1. **强检索：**召回高质量、相关的上下文，让模型有正确的素材可以"念"
2. **强约束：**Prompt 里明确告诉模型"只能基于给定信息回答，不知道就说不知道"
3. **可溯源：**让模型回答时标注信息来源，用户可以验证（也方便做事实校验）

进阶手段：

- **Self-Check：**生成后用另一个模型检查答案和上下文是否一致
- **引用抽取：**自动从答案中抽取引用片段，和原文对比

面试点：RAG 不能 100% 消除幻觉，但可以显著降低。核心是"强检索 + 强约束 + 可溯源"三位一体。

面试怎么说："减少幻觉主要靠三板斧：检索要准、Prompt 要约束只能用给定信息、回答要可溯源。我们的做法是高召回率 + 明确的 Prompt 指令 + 引用标注，把幻觉压到可接受范围内。"

⑩ 代入感

是什么：用户和 IP 角色对话时，感觉"真的在和这个角色聊天"，而不是"一个客服套了个 IP 皮"。

怎么提升代入感：

- **人设一致性：**语气、用词、性格全程统一（傲娇角色不能突然变软萌）
- **经典元素：**适当加入角色口头禅、标志性动作、梗
- **知识边界：**角色不知道的就说不知道，符合角色设定（古代角色不知道手机是什么）

技术支撑：

- System Prompt 注入 IP 人设 → 保证说话风格
- IP 人设知识库召回 → 保证角色知识正确
- Rerank 精选 Top-3 人设片段 → 避免太长挤占 Context 空间

这是产品体验层面的指标，但技术上通过"System Prompt 人设注入 + IP 知识库召回"来支撑。面试可以提一句"人设一致性是 IP 导购 RAG 的核心指标之一"，体现你理解业务。

面试怎么说："代入感是 IP 导购和普通客服 RAG 最大的区别。我们通过 System Prompt 注入人设 + IP 知识库补充角色知识，让回答有角色味儿，用户感觉是真的在和喜欢的角色聊天。"

11.11 10 个技术点一句话速记

技术点	一句话解释
多域向量知识库	按业务域拆索引，各自管各自的，分而治之
IP 意图识别	先判断用户说的是哪个 IP / 角色
检索路由	识别到 IP 后，限定在该 IP 域内搜（pre-filter kNN）
BM25	ES 默认关键词匹配算法，精确但不会变通
kNN 向量检索	语义相似匹配，能理解同义词和泛化
RRF 融合	按排名倒数合并多路召回结果，不用调权重
Rerank 重排	先粗捞 50 条再精排 5 条，又快又准
角色化 Prompt	人设放 System、事实放 Context，职责分离
减少幻觉	强检索 + 强约束 + 可溯源，三板斧降幻觉
代入感	让用户觉得"真的在和这个角色聊天"

十二、MyBatis-Plus 分页与条件查询

★ 所属简历模块：⑤ 工程实践

项目背景：所有业务模块的列表查询都基于 MyBatis-Plus (MP)，涉及**分页插件配置**、**条件查询的空安全写法**、**分页方法选择**、**分页参数兼容**等核心技术点。项目踩过"分页 total 总为 0"和"无条件加了 NULL 条件导致查不到数据"两个经典坑。

12.1 MpConfig：分页插件配置

MP 3.4.0 之后引入了新的拦截器机制，分页插件通过 `MybatisPlusInterceptor` 注册，老版本的 `PaginationInterceptor` 已废弃。

```
@Configuration
public class MpConfig {
    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
        // 必须指定 DbType.MYSQL，否则 MP 不知道用哪种数据库方言
        interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.MYSQL));
        return interceptor;
    }
}
```

面试点：为什么要指定 `DbType.MYSQL`？因为不同数据库的分页 SQL 语法不一样：MySQL 是 `LIMIT offset,size`，Oracle 是 `ROWNUM`，PostgreSQL 也是 `LIMIT` 但方言细节不同。MP 需要根据 `DbType` 选择对应的分页 SQL 拼接器。如果不配 `DbType` 或配错了，分页 SQL 会拼接错误，出现 total 统计不准或语法报错。

12.2 条件查询的"空安全"写法（必问）

坑：不做空判断导致 `WHERE strategy_id = NULL`

错误代码（`StrategyCommentServiceImpl` 修复前）：

```
// ✗ 前端不传 strategyId, qo.getStrategyId() 是 null, 但 eq 仍然拼到 SQL
wrapper.eq(StrategyComment::getStrategyId, qo.getStrategyId());
// 生成 SQL: WHERE strategy_id = NULL
// 注意: MySQL 中 = NULL 永远为 false, 所以查不到任何数据!
```

正确写法（带条件参数的重载方法）：

```
// ✓ 第一个参数是 boolean 条件, true 才拼入 SQL, false 跳过
wrapper.eq(qo.getStrategyId() != null, StrategyComment::getStrategyId, qo.getStrategyId());
wrapper.eq(qo.getState() != null, StrategyComment::getState, qo.getState());
// 前端不传 strategyId → condition=false → 不拼 condition → 正确返回所有评论
```

MP 所有条件方法都支持这个"布尔条件"重载：

```
// 等于: condition 为 true 时才生效
wrapper.eq(condition, column, val)
// 模糊查询: keyword 有值时才 LIKE
wrapper.like(StringUtils.isNotEmpty(qo.getKeyword()), Note::getTitle, qo.getKeyword())
// IN 集合: 集合非空时才 IN
wrapper.in(CollectionUtils.isNotEmpty(ids), Note::getId, ids)
// BETWEEN: 两端都不为空时才范围查询
wrapper.between(min != null && max != null, Note::getAvgConsume, min, max)
```

面试一句话总结：永远用带 `boolean condition` 重载的条件方法，不要直接传可能为 `null` 的值。这是 MP 项目中最常见的低级

bug，也是面试官判断你是否真用过 MP 的试金石。

12.3 踩坑：ServiceImpl.page() vs baseMapper.selectPage()

现象： `NoteServiceImpl.queryPage` 用 MP 自带的 `page(page, wrapper)`（`ServiceImpl` 的 shortcut 方法），明明 DB 有数据，但 `page.getTotal()` 总返回 0。

根因： `ServiceImpl.page()` 在某些 MP 版本（如 3.4.2）中，对自定义 SQL、特殊条件包装时不会正确触发 COUNT 查询，导致 total 丢失。

修复： 改为调用 `baseMapper.selectPage(page, wrapper)`，这是 MyBatis 原生的分页查询入口，一定会被分页拦截器拦截并正确统计 total：

```
// ❌ 可能丢失 total
page(page, wrapper);           // ServiceImpl 快捷方法

// ✅ total 100% 正确
baseMapper.selectPage(page, wrapper); // MyBatis 原生查询入口
```

面试点： 记住一个原则——只要分页 total 不对，先把 `page()` 换成 `baseMapper.selectPage()` 试试。如果换了还不对，再排查分页插件配置、SQL 写法（比如多表 JOIN 时 COUNT 统计的是笛卡尔积，需要手写 countSQL）。

12.4 QueryObject：分页参数兼容设计

后台管理（Vue Admin）和前台（PC/Nuxt）对分页参数的命名不一致：

- 后台习惯用 `pageNum`（RuoYi 框架约定）
- 有些第三方前端习惯用 `currentPage`

`QueryObject` 做了双字段兼容，谁有值就用谁：

```
public class QueryObject implements Serializable {
    private int currentPage = -1; // 传了 currentPage，优先级高
    private int pageNum = -1;     // 传了 pageNum，兜底
    private int pageSize = 10;
    private String keyword;

    public int getCurrentPage() {
        if (currentPage != -1) { return currentPage; } // 优先 currentPage
        if (currentPage == -1 && pageNum != -1) { return pageNum; } // 兜底 pageNum
        return 1; // 默认第 1 页
    }
}
```

面试点： 这种“兼容不同前端参数名”的 Query 设计在多端项目中很常见。设计原则：默认值用“无效值标记”（如 -1），而不是 0 或 null，这样能明确区分“前端没传”和“前端传了 0”两种情况。

12.5 面试题模板（结合项目）

面试官问： MyBatis-Plus 条件查询时，参数为 null 怎么办？

答： 用 MP 条件方法的 boolean condition 重载。所有条件方法（eq/ne/like/in/between 等）都有一个带 boolean 首参的版本，condition 为 true 时才把条件拼进 SQL，为 false 就跳过。

比如我项目里攻略评论的分页查询，前端可能传 `strategyId`（查某篇攻略的评论）也可能不传（后台查所有评论），写法是：

```
wrapper.eq(qo.getStrategyId() != null, StrategyComment::getStrategyId, qo.getStrategyId());
wrapper.eq(qo.getState() != null, StrategyComment::getState, qo.getState());
```

如果直接写 `wrapper.eq(StrategyComment::getStrategyId, qo.getStrategyId())`，`strategyId` 为 `null` 时会生成 `WHERE strategy_id = NULL`，而 MySQL 中 `=NULL` 永远返回 `false`，所以一条也查不到。我们项目早期就踩过这个坑，前端显示“暂无数据”，排查了半天才发现是条件里带了 `NULL`，后来统一改成带 `condition` 参数的写法就没事了。

12.6 面试可能追问的问题

Q1: LambdaQueryWrapper 和 QueryWrapper 有什么区别？为什么推荐 Lambda？

A:

- `QueryWrapper` 写字段名是字符串：`wrapper.eq("strategy_id", 123)`，如果把列名写错（`strategy_id` 写成 `strategyid`）编译不报错，运行时才抛 SQL 异常，且数据库重构改列名时要全局搜字符串。
- `LambdaQueryWrapper` 用方法引用：`wrapper.eq(StrategyComment::getStrategyId, 123)`，字段名写错编译就通不过，且 IDE 重构时会自动跟着改。
- 所以项目里只用 `LambdaQueryWrapper`，不用普通 `QueryWrapper`。这也是“把错误从运行时提前到编译期”的最佳实践。

Q2: 分页插件为什么一定要配 DbType？不配会怎样？

A: MP 的分页插件要根据数据库类型拼接对应的分页 SQL:

| 数据库 | LIMIT 语法 |

|-----|-----|

| MySQL | `SELECT ... LIMIT ?, ?` |

| Oracle 11g- | `SELECT * FROM (SELECT t.*, ROWNUM rn FROM (...) WHERE ROWNUM <= ?) WHERE rn > ?` |

| PostgreSQL | `SELECT ... LIMIT ? OFFSET ?` |

| SQL Server | `SELECT ... OFFSET ? ROWS FETCH NEXT ? ROWS ONLY` |

不配 `DbType` 或配错 → MP 选错方言 → 拼接出的 SQL 在目标库上语法错误，轻则 `total` 不准，重则报错。我们用的是 `DbType.MYSQL`。

Q3: total 为 0 除了 baseMapper.selectPage 这个问题，还有哪些可能？

A: 排查顺序：

1. **拦截器配置**：有没有注册 `MybatisPlusInterceptor` + `PaginationInnerInterceptor`？老版本项目升级到 MP 3.4+ 最容易漏。
2. **方法选择**：是不是用了 `page()` 没触发 `count`？换成 `baseMapper.selectPage()`。
3. **自定义 SQL**：如果写了 XML 自定义 SQL（`selectPageByCondition`），MP 自动生成的 `countSQL` 可能不对（比如 JOIN 多张表），需要手写 `countId` 或 `@Select(flushCache=true)` 的 `count` 查询。
4. **SQL 本身无结果**：WHERE 条件太严真的没数据，先把同条件 SQL 粘到 MySQL 客户端执行验证。

Q4: QueryObject 为什么用 -1 作为默认值而不是 0 或 null？

A: 用“无效哨兵值”来区分“用户没传”和“用户传了值”：

- 如果用 0 作默认，就无法区分“前端没传 `currentPage`”和“前端明确传了 `currentPage=0`”（虽然 0 也是不合法的，但至少能识别用户意图）。
- 如果用 `Integer null`，需要到处判断 `!= null`，用 `int` + 哨兵值写法更简洁。
- 这种“无效值作默认”的模式在配置类、枚举中也常见，比如 `private int timeout = -1` 表示未设置，取值时再判断是否使用默认值。

十三、Quartz 定时任务调度 (job 模块)

★ 所属简历模块：④ Redis 高并发互动数据（定时回写）+ 附录 B（详解）

项目背景: `travel-modules-job` 负责平台的所有定时任务, 包括"Redis 统计数据异步刷回 DB" (StrategyTask、NoteTask)、"默认测试任务" (RyTask、TestTask) 等。核心特点是**任务配置存储在数据库 (sys_job 表) 中, 支持后台管理页面动态新增、修改、暂停、删除任务, 无需重启服务。**

13.1 整体架构



13.2 ScheduleUtils: 动态创建 Quartz 任务

核心方法 `createScheduleJob` 在服务启动时、后台新增/修改任务时调用, 把 DB 中的 `sys_job` 变成真正运行的 Quartz Job:

```
public static void createScheduleJob(Scheduler scheduler, SysJob job) throws SchedulerException {
    // ① 是否允许并发: 0=允许 (QuartzJobExecution), 1=禁止 (加了 @DisallowConcurrentExecution)
    Class<? extends Job> jobClass = "0".equals(job.getConcurrent())
        ? QuartzJobExecution.class
        : QuartzDisallowConcurrentExecution.class;

    // ② 构建 JobDetail (唯一标识 = TASK_CLASS_NAME + jobId)
    JobDetail jobDetail = JobBuilder.newJob(jobClass)
        .withIdentity(getJobKey(jobId, jobGroup)).build();
    jobDetail.getJobDataMap().put(TASK_PROPERTIES, job); // 把 sys_job 整个塞进去, 后续反射要用

    // ③ 构建 CronTrigger + 设置失火策略
    CronScheduleBuilder cb = CronScheduleBuilder.cronSchedule(job.getCronExpression());
    cb = handleCronScheduleMisfirePolicy(job, cb); // 4 种策略见下
    CronTrigger trigger = TriggerBuilder.newTrigger()
        .withIdentity(getTriggerKey(jobId, jobGroup))
        .withSchedule(cb).build();

    // ④ 注册前先删除旧任务 (防止重复/冲突)
    if (scheduler.checkExists(getJobKey(jobId, jobGroup))) {
        scheduler.deleteJob(getJobKey(jobId, jobGroup));
    }

    // ⑤ 没过期就注册; 状态为暂停则注册后立即 pause
    if (CronUtils.getNextExecution(job.getCronExpression()) != null) {
        scheduler.scheduleJob(jobDetail, trigger);
    }
    if (job.getStatus().equals(PAUSE)) {
        scheduler.pauseJob(getJobKey(jobId, jobGroup));
    }
}
```

失火策略 (Misfire Policy) 4 选 1:

策略	含义	适用场景
MISFIRE_DEFAULT	Quartz 智能判断 (默认)	通用
MISFIRE_IGNORE_MISFIRES	把错过的执行全部一次性补回来, 然后正常执行	统计任务, 错过的执行必须补跑
MISFIRE_FIRE_AND_PROCEED	立即执行一次错过的任务, 然后按 cron 继续	希望立刻补一次就好
MISFIRE_DO_NOTHING	什么都不做, 忽略错过的, 等下一次正常触发	定时拉取最新数据的任务, 错过没关系

面试题: 失火 (Misfire) 指"到了执行时间, Quartz 线程池忙没腾出空来, 或者服务宕机错过了", 需要明确补偿策略。我项目 Redis 持久化任务用 MISFIRE_IGNORE_MISFIRES (不能丢数据), 普通测试任务用 DO_NOTHING。

13.3 JobInvokeUtil: 反射调用目标 Bean 方法

Quartz Job 只负责调度时机, 真正的业务逻辑写在具体的 Task 类 (如 NoteTask、StrategyTask) 里。两者之间通过"字符串 → 反射"桥接:

```
sys_job.invokeTarget 字符串: "noteTask.statisHashMapPersist('param1', 123L, true)"
    |
    ▼ 解析
    beanName = "noteTask"
    methodName = "statisHashMapPersist"
    args = [("param1", String), (123, Long), (true, Boolean)]
    |
    ▼ 执行
    SpringUtils.getBean("noteTask") → 拿到 NoteTask Bean
    Method.invoke(noteTask, args) → 执行业务逻辑
```

核心代码:

```
public static void invokeMethod(SysJob sysJob) throws Exception {
    String invokeTarget = sysJob.getInvokeTarget();
    String beanName = getBeanName(invokeTarget); // noteTask
    String methodName = getMethodName(invokeTarget); // statisHashMapPersist
    List<Object[]> methodParams = getMethodParams(invokeTarget); // 解析参数值 + 参数类型

    // beanName 是类名路径 (含多个点) → 直接反射 new 实例; 否则从 Spring 容器拿 Bean
    if (!isValidClassName(beanName)) {
        Object bean = SpringUtils.getBean(beanName); // 推荐用法, Spring 管理的单例
        invokeMethod(bean, methodName, methodParams);
    } else {
        Object bean = Class.forName(beanName).getDeclaredConstructor().newInstance();
        invokeMethod(bean, methodName, methodParams);
    }
}
```

参数类型自动解析 (getMethodParams) :

参数写法	解析结果	类型
'hello' / "hello"	"hello"	String.class
true / false	true / false	Boolean.class
100L	100L	Long.class
3.14D	3.14D	Double.class
123 (无后缀)	123	Integer.class

面试坑点: invokeTarget 中写的 Bean 名必须和 Spring 容器中 Bean 的名称一致。比如任务类上是 @Service 默认 Bean 名是 noteTask (首字母小写类名), invokeTarget 就要写 noteTask.xxx ; 如果写了 @Service("noteTaskService") 自定义名称, invokeTarget 也必须对应改, 否则反射拿不到 Bean 抛异常。**这是项目实际踩过的坑**——Job 配置写的是类名, Bean 名和类名不一致时任务一直失败。

13.4 典型业务: Redis 统计数据刷 DB

以 NoteTask 为例, 这是 Quartz Job 跨服务调用 Note 模块的典型范例:

```
【sys_job 表】
invokeTarget: noteTask.statisHashMapPersist()
cronExpression: 0 */10 * * * ? ← 每 10 分钟

// travel-modules-job 中的 NoteTask (Job Bean 容器)
@Component("noteTask") // Bean 名 = invokeTarget 中的前缀
public class NoteTask {
    @Autowired
    private RemoteNoteService remoteNoteService; // Feign 调用 note 服务

    public void statisHashMapPersist() {
        // Job 模块只管"什么时候做", "怎么做"由业务模块自己暴露 Feign 接口
        R<?> r = remoteNoteService.statisHashMapPersist();
        if (R.isError(r)) { log.error("游记统计刷库失败: {}", r.getMsg()); }
    }
}
```

面试点: 为什么不把刷库逻辑直接写在 Job 模块里? 因为**职责分离**: Job 模块是"调度者" (只关心什么时候), Note/Strategy 模块是"执行者" (关心怎么刷、刷哪些数据)。跨服务通过 Feign 调用, Job 模块不需要引入 Note 的 Mapper、实体、表结构依赖, 耦合更低。

13.5 面试题模板（结合项目）

面试官问：Quartz 定时任务你是怎么用的？任务是硬编码写死的吗？

答：不是硬编码。我项目里任务配置是**存数据库**的（sys_job 表），管理后台可以新增、修改、暂停任务，不需要重启服务。

核心流程：服务启动时和后台改动任务时，`ScheduleUtils.createScheduleJob` 从 sys_job 表读取配置，根据任务是否允许并发选择 Job 类（`QuartzJobExecution` 允许并发，`QuartzDisallowConcurrentExecution` 加了 `@DisallowConcurrentExecution` 注解禁止同一任务重入），根据 cronExpression 构建 CronTrigger，设置失火策略，然后动态注册到 Quartz Scheduler 中。

到点执行时，Quartz Job 只做一件事——调用 `JobInvokeUtil.invokeMethod(sysJob)`，通过反射解析 invokeTarget 字符串（如 `noteTask.statisHashMapPersist()`），从 Spring 容器拿对应 Bean，用反射调用对应方法。参数支持 String、Boolean、Long、Double、Integer 五种类型自动解析。

业务上用在 Redis 统计数据的异步刷库：每 10 分钟触发一次 `NoteTask.statisHashMapPersist()`，通过 Feign 调用 note 服务把 Redis 中的游记计数批量 UPDATE 回 MySQL。Job 模块只做调度，业务逻辑留在各微服务里，职责分离。

13.6 面试可能追问的问题

Q1: Quartz 的 Job、JobDetail、Trigger、Scheduler 分别是什么？

A:

- **Job:** 任务的业务逻辑接口（`execute(JobExecutionContext)`），我们通过 `JobBuilder.newJob(JobClass)` 注册。
- **JobDetail:** Job 的配置 + 实例，包含 Job 类、身份标识（JobKey）、JobDataMap（参数容器）。同一个 Job 类可以注册多个不同的 JobDetail（比如同一个 ReportJob 用不同参数生成日报和月报）。
- **Trigger:** 触发器，定义“什么时候执行”。SimpleTrigger（简单重复）、CronTrigger（Cron 表达式）两种。
- **Scheduler:** 调度器，是 Quartz 的大脑，管理所有 JobDetail 和 Trigger，按时间安排触发执行。
- 一句话记忆：Scheduler 是调度中枢，Job 是“做什么”，Trigger 是“什么时候做”，JobDetail 是把 Job 和参数封装起来供 Scheduler 使用。

Q2: @DisallowConcurrentExecution 是干什么的？

A: 加在 Job 类上，禁止同一个 JobDetail 并发执行。例如“刷库任务”每 10 分钟一次，但如果某一次刷库执行了 15 分钟还没完成，默认情况下下一个 10 分钟到点又会开始一次，可能导致重复刷数据、死锁。加了 `@DisallowConcurrentExecution` 后，Quartz 会等前一次执行完才开始下一次，保证同一任务的实例串行化。

Q3: SpringTask (@Scheduled) 和 Quartz 有什么区别？你们怎么选的？

A:

| 维度 | SpringTask (@Scheduled) | Quartz |

|-----|-----|-----|

| 配置方式 | 注解写死在代码里 | DB/XML/注解都支持，可动态修改 |

| 动态修改 | 改代码 + 重启 | 管理后台改 DB，即时生效，不用重启 |

| 任务持久化 | 无 | 支持 JDBC JobStore，服务重启后任务自动恢复 |

| 失火策略 | 很简单（只有 fixedDelay/cron） | 4 种以上，灵活配置 |

| 集群部署 | 各节点各跑各的，会重复执行 | JDBC JobStore 集群模式，同一任务只跑一次 |

- 选 Quartz 是因为产品要求后台页面能动态管理任务（增删改暂停），且要求集群部署时不重复执行，SpringTask 满足不了。简单单机任务（比如每天 3 点清日志）用 SpringTask 更轻量。

Q4: invokeTarget 里的 Bean 名和 Spring 容器里对不上会怎样？

A: 这是项目里实际踩过的坑。JobInvokeUtil 用 `SpringUtils.getBean(beanName)` 拿 Bean，如果 Bean 名写错（类名是 NoteTask，Bean 名默认是 `noteTask`，但写成了 `NoteTask` 首字母大写），会抛 `NoSuchBeanDefinitionException`，任务一直失败，错误日志里会看到找不到 Bean。排查方法：在 sys_job 表检查 invokeTarget 前缀，和 `@Component("xxx")` 或默认首字母小写类名对比，必须完全一致。

十四、项目踩坑实录 (Bug 修复专题)

✦ 所属简历模块：附录 A — 踩坑实录

面试加分项：不是问什么答什么，而是主动讲“我踩过什么坑、怎么定位、怎么修复、学到了什么”，证明你有真实的开发经验而不是背题。本项目整理了 4 个最典型的实战修复案例。

14.1 坑一：攻略评论列表数据库有数据，前端显示“暂无数据”

现象：后台攻略评论管理页，接口 `GET /strategy/admin/strategyComments/list?pageNum=1&pageSize=10` 返回空数组，MySQL `ta_strategy_comment` 表中明明有 100+ 条评论。

定位过程：

1. 打开浏览器 DevTools → Network，看接口返回 `rows: []`，total 为 0。
2. 拿 SQL (MP 的 showSql 日志) 直接在 MySQL 执行，**居然也是 0 条！**
3. SQL 里有 `WHERE strategy_id = NULL AND state = NULL` ——这就是问题！
4. MySQL 中 `= NULL` 永远返回 false (应该用 `IS NULL`)，所以查不到任何数据。
5. 回到 `StrategyCommentServiceImpl.queryPage`，代码是 `wrapper.eq(StrategyComment::getStrategyId, qo.getStrategyId())`，但前端不传 `strategyId` 时 `qo.getStrategyId()` 就是 null，条件被拼了进去。

修复：两个条件都改为带 boolean condition 的空安全写法：

```
// ✖ 修复前
wrapper.eq(StrategyComment::getStrategyId, qo.getStrategyId());
wrapper.eq(StrategyComment::getState, qo.getState());

// ✔ 修复后 (StrategyCommentQuery 同步新增了 state 字段)
wrapper.eq(qo.getStrategyId() != null, StrategyComment::getStrategyId, qo.getStrategyId());
wrapper.eq(qo.getState() != null, StrategyComment::getState, qo.getState());
```

面试一句话：MySQL 的 `=` 和 `IS NULL` 是两回事，永远不要让 `= NULL` 出现在你的 SQL 里。用 MP condition 参数控制条件是否拼入。

14.2 坑二：搜索“广州”出来“杭州” (ES 分词器名称错误)

详细分析见第十章 10.1 节。面试时可以先讲这个故事，然后自然引出 IK 分词器、standard 逐字切分、mapping 创建后不能改等一系列知识点，是很好的串联题引子。

修复要点：

- 4 个 ES Service 的 analyzer 从 `ik-max-word` 改为 `ik_max_word`
- 管理后台点“重新初始化索引”删除旧索引重建
- 验证：`GET /strategy/_analyze { "analyzer": "ik_max_word", "text": "广州" }`，结果是 `["广州"]` (不再是 `["广", "州"]`)

14.3 坑三：首页游记"by 作者"头像不显示 + 分页 total 恒为 0

两个 Bug 同时出现在同一页面，分别对应两处代码缺陷。

Bug 3.1：作者头像/nickname 不显示

现象：首页热门游记列表，每张卡片底部显示“by 作者昵称 + 头像”，但头像加载失败（裂图），昵称空白。

定位：

1. 前端 `index.vue` 模板写的是 `{{ item.author.nickname }}` + `:src="item.author.headImgUrl"`，需要 `Note` 有一个 `author` 对象字段。
2. 打开 `Note.java`，果然没有 `author` 字段，只有 `authorId`、`authorNickname`、`authorHeadImgUrl`（还标了 `@TableField(exist = false)`，且分页查询时没填充值）。
3. `NoteServiceImpl.queryPage` 返回的 `Note` 列表里，没有任何地方去查 `UserInfo` 填充 `author` 信息。

修复三步：

1. `Note.java` 加 `author` 字段（非 DB 列）：

```
@TableName("ta_note")
public class Note {
    // ... 其他字段 ...
    @TableField(exist = false)    // 告诉 MP ta_note 表里没有 author 列
    private UserInfo author;
}
```

2. `NoteServiceImpl` 新增 `fillAuthorInfo` 方法（Feign 查 + 冗余字段兜底）：

```
private void fillAuthorInfo(List<Note> noteList) {
    for (Note note : noteList) {
        // ① 优先 Feign 远程查
        try {
            R<UserInfo> r = remoteUserInfoService.getOne(note.getAuthorId(), "inner");
            if (r != null && r.getCode() == R.SUCCESS) {
                note.setAuthor(r.getData());
                continue;
            }
        } catch (Exception ignored) {}
        // ② 兜底：用 Note 的冗余字段 (authorNickname、authorHeadImgUrl) 构造
        UserInfo fallback = new UserInfo();
        fallback.setNickname(note.getAuthorNickname() != null
            ? note.getAuthorNickname() : "匿名用户");
        fallback.setHeadImgUrl(note.getAuthorHeadImgUrl());
        note.setAuthor(fallback);
    }
}
```

3. `queryPage` 末尾调用填充方法：

```
baseMapper.selectPage(page, wrapper);    // 顺便修了 Bug 3.2
fillAuthorInfo(page.getRecords());      // ← 新增
return page;
```

设计亮点：不是所有系统都敢做“远程服务挂了也要正常展示”，但这个 fallback 机制体现了微服务的容错思维——**宁可展示退化的数据，也不要直接 500 让用户看白屏**。面试时主动提这个设计，会让面试官觉得你有生产环境思维。

Bug 3.2：分页 total 恒为 0，明明有数据显示“共 0 条”

修复：把 `page(page, wrapper)` 换成 `baseMapper.selectPage(page, wrapper)`，详见第十一章 11.3 节。

一个有趣的细节：Bug 3.1 和 3.2 是同一个方法里的两个问题，改完后前端页面同时恢复正常——头像能显示了，分页总数也对了。排查时要注意“一个接口可能有多个独立缺陷”，不要修了一个问题就以为全部搞定了，一定要验证所有显示字段。

14.4 坑四：R 响应判断方式错用（微服务 Feign 调用）

现象：Feign 调用返回成功了，但 `if (r.isSuccess())` 报错说找不到方法，或者条件永远不成立。

根因：项目里返回的统一响应类 `R`（common-core 模块），只有两个字段：

```
public class R<T> {
    public static final int SUCCESS = 200;
    private int code;
    private String msg;
    private T data;

    // 正确的判断方式: 用 getCode() 和常量比较
    public static boolean isError(R<?> r) { return r == null || r.getCode() != SUCCESS; }
}
```

错误写法 1: 用不存在的方法

```
if (r.checkSuccess()) { ... } // ✗ R 类根本没有 checkSuccess 方法
if (r.isSuccess()) { ... } // ✗ 也没有 isSuccess 方法
```

错误写法 2: 和字符串比较

```
if ("success".equals(r.getMsg())) { ... } // ✗ msg 是中文提示, 不是状态
```

正确写法 (项目统一约定) :

```
// ✔ 方案 A: 用 getCode 和常量
if (r != null && r.getCode() == R.SUCCESS && r.getData() != null) {
    author = r.getData();
}

// ✔ 方案 B: 用工具方法
if (!R.isError(r)) {
    author = r.getData();
}
```

面试点: 判断响应状态永远用 **code 和固定常量比较**, 不要用 msg (会变、可能是中文), 更不要用 data 是否为 null ("查询成功但数据为空"是正常业务情况)。这是工程中代码健壮性的基础素养。

14.5 面试时讲 Bug 的万能公式

面试官问: 你在项目中遇到过什么难点/坑, 怎么解决的?

可以按这个结构讲 (以"广州出来杭州"为例) :

- ① 背景是什么（一两句话）
 - 项目做了 ES 全文搜索，攻略、目的地、游记四类数据都建了索引。
- ② 现象是什么（用户能看到的问题）
 - 产品反馈搜"广州"出来的全是"杭州"的攻略，完全不对。
- ③ 怎么定位的（你的排查思路）
 - 先在 ES 里用 `_analyze` 接口测试"广州"的实际分词效果，发现是逐字切成了"广"和"州"——说明 IK 分词器没生效。
 - 再看 `_mapping`，发现 `analyzer` 写的是 `"ik-max-word"`（连字符），但正确名称应该是 `"ik_max_word"`（下划线）。
 - 名称错了 ES 不报错，静默回退到 `standard` 逐字切分，所以"广州"和"杭州"都匹配到了"州"字。
- ④ 怎么修的（具体改了什么）
 - 4 个 ES Service 的索引 `mapping` 全部修正为 `ik_max_word`，管理后台点初始化索引删除旧 `mapping` 重建，验证分词正确。
- ⑤ 学到了什么（拔高到方法论）
 - 对中间件的配置要"验证生效"而不是"写了就算"，ES 要查 `_mapping` 看实际配置，Redis 要 TTL 看实际过期时间；
 - 字符串配置项很容易写错字符却不报错，最好封装常量或枚举，避免"ik-max-word"和"ik_max_word"这种肉眼难以区分的拼写错误。

面试官听到最后⑤ 会加分——说明你不是只修了个 bug，而是把经验提炼成了方法论，可以避免团队再踩同样的坑。四个坑都按这个结构准备，面试时挑 1~2 个最有代表性的讲即可。

十五、后续补充备忘

✦ 所属简历模块：附录 D

以下方向在项目代码中有体现，面试时可以继续深化：

1. **策略排行模块 (StrategyRank)**：每月 1 号定时任务生成历史快照（`statis_time` 标记），按月归档热门榜数据，支持查看"2026 年 7 月热门攻略"而不被当前数据覆盖。（表设计 + 定时任务联动）
2. **攻略主题分组 (StrategyTheme)**：SQL 用 `GROUP_CONCAT` 一次性查出主题 → 攻略列表，避免 N+1 查询。（MySQL `GROUP_CONCAT` + MyBatis `resultMap`）
3. **Banner 模块 (travel-modules-adv)**：仅查询启用状态（`state=0`）、按 `seq` 升序排列、`LIMIT 5` 的典型前台列表查询模式。
4. **Feign 降级工厂 (FallbackFactory)**：Sentinel 熔断时返回降级数据而不是抛异常，前台服务不会 500。
5. **@TableField(exist = false)**：这个注解在项目中大量使用，`Note.author`、`Note.authorNickname`/`authorHeadImgUrl`、各种 VO 扩展字段——面试被问"MP 怎么忽略非表字段"就举这几个例子。